**Sri Sivasubramaniya Nadar College of Engineering, Kalavakkam**
**(An Autonomous Institution, Affiliated to Anna University, Chennai)**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

<table>
<tr><td>Course Code</td><td>UCS2411</td><td>Course Name</td><td colspan="6">Database Lab</td></tr>
<tr><td rowspan="2">Course Type</td><td rowspan="2">Lab</td><td rowspan="2">Course Category</td><td rowspan="2">Professional Core (PC)</td><td>L</td><td>T</td><td>P</td><td>E</td><td>C</td></tr>
<tr><td>0</td><td>0</td><td>3</td><td>0</td><td>1.5</td></tr>
<tr><td>Regulation</td><td colspan="2">2021</td><td>Academic Year & Batch</td><td colspan="5">2023 - 24 & 2022 - 26</td></tr>
<tr><td colspan="3">Degree and Branch</td><td colspan="6">B.E. Computer Science & Engineering</td></tr>
<tr><td colspan="2">Semester</td><td>IV</td><td>K6, CO3</td><td colspan="5">PI: 1.3.1, 1.4.1, 2.1.3, 2.3.1, 2.3.2, 3.4.2, 4.1.2, 5.2.2, 9.3.1, 10.1.3, 11.3.1, 13.3.1 & 13.3.2</td></tr>
<tr><td colspan="2">Assignment No.</td><td>10</td><td>Assigned Date</td><td colspan="2">24-05-2024</td><td>Due Date</td><td colspan="2">07-06-2024</td></tr>
<tr><td colspan="3">Name of the Faculty</td><td colspan="6">Dr. N. Sujaudeen & Dr. P. Mirunalini</td></tr>
<tr><td colspan="9">Title: Mini Project Implementation</td></tr>
</table>

**Sri Sivasubramaniya College of Engineering**

Department of Computer Science

# National Examination Management System (NEMS)

UCS3402 - Database Management Systems

**Submitted By:**

Swathi B - 3122245001177

Vedika Chandra Mouli - 3122245001194

Visithra.J - 3122245001200

**Submission Date:**

March 3, 2026

## PROBLEM STATEMENT:

In traditional examination management systems, most processes such as student registration, exam scheduling, seat allocation, invigilator assignment, evaluation, result processing, grievance handling, and security monitoring are handled using fragmented and semi-manual approaches. This leads to inefficiencies, delays in processing, data inconsistency, lack of real-time monitoring, and increased chances of human errors and malpractice.

There is also a lack of integrated systems that can ensure secure handling of examination data while providing transparency, traceability, and automated validation across all stages of the examination lifecycle. Critical issues such as duplicate seat allocation, invigilator workload imbalance, delayed result processing, unresolved grievances, and insufficient monitoring of malpractice incidents further reduce the reliability of the system.

Therefore, there is a need for a centralized, structured, and secure database-driven Examination Management System that integrates all core modules including student registration, exam scheduling, invigilator management, attendance tracking, secure evaluation, result processing, grievance handling, and security monitoring. The proposed system aims to ensure data integrity, improve operational efficiency, enhance transparency, and provide real-time monitoring and analytics for effective decision-making in examination administration.

## OBJECTIVES OF THE PROJECT:

1. To design a centralized database system for managing student registration, examination scheduling, invigilator assignment, evaluation, results, grievances, and security monitoring.

2. To eliminate manual processing and reduce human errors in examination management activities by automating key operations.

3. To ensure data consistency and integrity across all modules through well-defined relational database structures and constraints.

4. To provide efficient seat allocation and attendance tracking mechanisms for smooth conduct of examinations.

5. To enable secure and fair evaluation of answer scripts through structured evaluation and moderation processes.

6. To generate accurate and timely results along with rank computation, performance analysis, and statistical reporting.

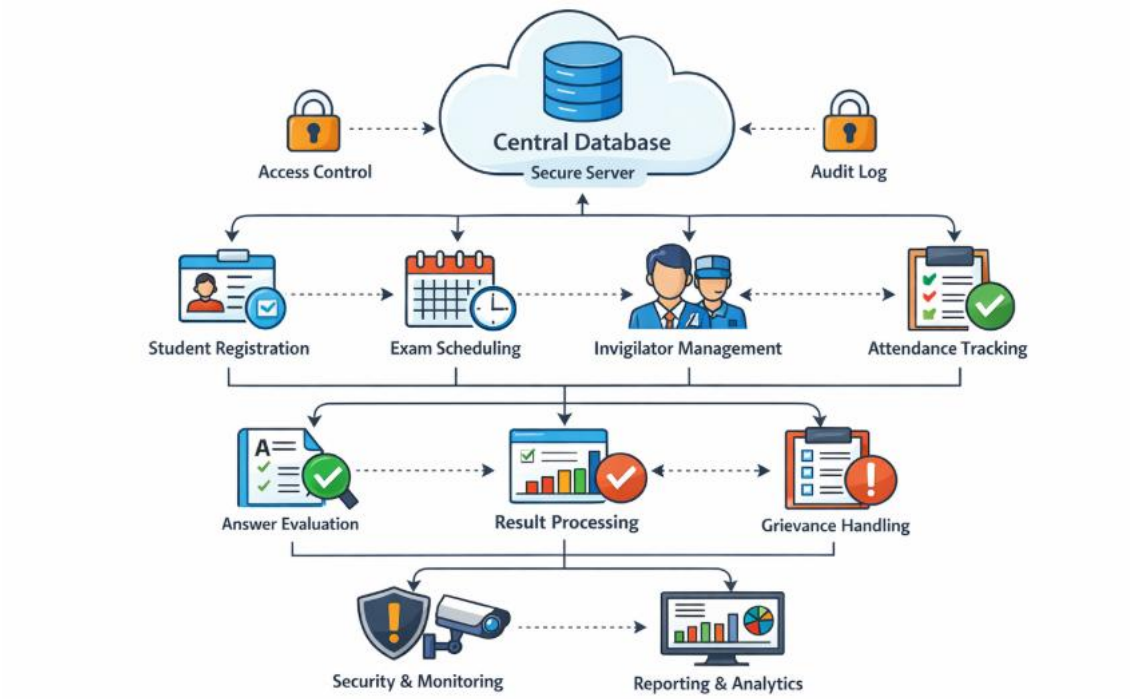
7. To implement a grievance handling system for transparent submission, tracking, and resolution of student complaints.

8. To incorporate security and monitoring features for detecting malpractice, ensuring system integrity, and maintaining audit trails.

9. To improve administrative efficiency by providing real-time reporting and analytics for decision-making.

10. To develop a scalable system that can be extended to support large-scale examinations conducted at institutional or national level.

**SYSTEM ARCHITECTURE:**




**CORE MODULES PRESENT:**

**MODULE 1: Master Reference Data Module**

**MODULE 2: Student Registration Module**

**MODULE 3: Exam Scheduling Module**

**MODULE 4: Invigilator Management Module**

**MODULE 5: Student Distribution and Attendance Module**

**MODULE 6: Secure Evaluation Module**

**MODULE 7: Result Processing Module**

**MODULE 8: GrievanceHandlingModule**

**MODULE 9: SecurityandMonitoringModule**

## IMPLEMENTATION OF MODULE 1 – MASTER REFERENCE DATA MODULE

### (i) DESIGN AND IMPLEMENTATION OF ENTITES WITH DATA INITIALIZATION:

**This module focuses on the creation of core reference entities and the insertion of foundational data required for system initialization.**

```
SQL> CREATE TABLE REGION (
  2   Region_ID NUMBER PRIMARY KEY,
  3   Region_Code VARCHAR2(10) UNIQUE NOT NULL,
  4   Region_Name VARCHAR2(100) NOT NULL,
  5   City VARCHAR2(100) NOT NULL,
  6   District VARCHAR2(100) NOT NULL,
  7   State VARCHAR2(100) NOT NULL,
  8   Country VARCHAR2(100) NOT NULL,
  9   Pincode VARCHAR2(10) NOT NULL,
 10   Timezone VARCHAR2(50) NOT NULL
 11 );

Table created.

SQL> CREATE TABLE EXAM (
  2   Exam_ID NUMBER PRIMARY KEY,
  3   Exam_Code VARCHAR2(20) UNIQUE NOT NULL,
  4   Exam_Name VARCHAR2(200) NOT NULL,
  5   Conducting_Body VARCHAR2(200) NOT NULL,
  6   Exam_Mode VARCHAR2(50) NOT NULL,
  7   Exam_Level VARCHAR2(50) NOT NULL,
  8   Year NUMBER(4) NOT NULL,
  9   Total_Marks NUMBER NOT NULL CHECK (Total_Marks > 0)
 10 );

Table created.
```

```
SQL> CREATE TABLE EXAM_PHASE (
  2    Phase_ID NUMBER PRIMARY KEY,
  3    Exam_ID NUMBER NOT NULL,
  4    Phase_Number NUMBER NOT NULL,
  5    Phase_Name VARCHAR2(100) NOT NULL,
  6    Registration_Start_Date DATE NOT NULL,
  7    Registration_End_Date DATE NOT NULL,
  8    FOREIGN KEY (Exam_ID) REFERENCES EXAM(Exam_ID)
  9  );

Table created.

SQL> CREATE TABLE SUBJECT (
  2    Subject_ID NUMBER PRIMARY KEY,
  3    Exam_ID NUMBER NOT NULL,
  4    Subject_Code VARCHAR2(20) UNIQUE NOT NULL,
  5    Subject_Name VARCHAR2(150) NOT NULL,
  6    Duration_Minutes NUMBER NOT NULL CHECK (Duration_Minutes > 0),
  7    Max_Marks NUMBER NOT NULL CHECK (Max_Marks > 0),
  8    FOREIGN KEY (Exam_ID) REFERENCES EXAM(Exam_ID)
  9  );

Table created.

SQL> CREATE TABLE EXAM_PATTERN (
  2    Pattern_ID NUMBER PRIMARY KEY,
  3    Subject_ID NUMBER NOT NULL,
  4    Total_Questions NUMBER NOT NULL,
  5    Marks_Per_Question NUMBER(5,2) NOT NULL,
  6    Negative_Marking NUMBER(5,2) DEFAULT 0,
  7    Section_Count NUMBER NOT NULL,
  8    Pass_Percentage NUMBER(5,2) NOT NULL,
  9    FOREIGN KEY (Subject_ID) REFERENCES SUBJECT(Subject_ID)
```

```
 10  );
```

Table created.

```
SQL> CREATE TABLE CATEGORY (
  2     Category_ID NUMBER PRIMARY KEY,
  3     Category_Code VARCHAR2(20) UNIQUE NOT NULL,
  4     Category_Name VARCHAR2(100) NOT NULL,
  5     Relaxation_Percentage NUMBER(5,2) DEFAULT 0
  6  );
```

Table created.

```
SQL> CREATE TABLE EXAM_CENTER (
  2     Center_ID NUMBER PRIMARY KEY,
  3     Center_Code VARCHAR2(20) UNIQUE NOT NULL,
  4     Center_Name VARCHAR2(200) NOT NULL,
  5     Region_ID NUMBER NOT NULL,
  6     Address_Line1 VARCHAR2(200) NOT NULL,
  7     Address_Line2 VARCHAR2(200),
  8     City VARCHAR2(100) NOT NULL,
  9     Pincode VARCHAR2(10) NOT NULL,
 10     Center_Type VARCHAR2(50) NOT NULL,
 11     FOREIGN KEY (Region_ID) REFERENCES REGION(Region_ID)
 12  );
```

Table created.

```
SQL> CREATE TABLE ROOM (
  2     Room_ID NUMBER PRIMARY KEY,
  3     Room_Code VARCHAR2(20) UNIQUE NOT NULL,
  4     Center_ID NUMBER NOT NULL,
  5     Floor_Number NUMBER NOT NULL,
  6     Capacity NUMBER NOT NULL,
```

```
  7    Has_CCTV NUMBER(1) DEFAULT 0,
  8    Has_Biometric NUMBER(1) DEFAULT 0,
  9    Power_Backup_Available NUMBER(1) DEFAULT 0,
 10    Room_Type VARCHAR2(50) NOT NULL,
 11    Total_Rooms NUMBER NOT NULL,
 12    FOREIGN KEY (Center_ID) REFERENCES EXAM_CENTER(Center_ID)
 13  );

Table created.

SQL> CREATE TABLE HOLIDAY_CALENDAR (
  2    Holiday_ID NUMBER PRIMARY KEY,
  3    Region_ID NUMBER NOT NULL,
  4    Holiday_Date DATE NOT NULL,
  5    Holiday_Name VARCHAR2(150) NOT NULL,
  6    Is_National NUMBER(1) DEFAULT 0,
  7    FOREIGN KEY (Region_ID) REFERENCES REGION(Region_ID)
  8  );

Table created.

SQL> CREATE TABLE EXAM_CYCLE (
  2    Exam_Cycle_ID NUMBER PRIMARY KEY,
  3    Exam_ID NUMBER NOT NULL,
  4    Phase_ID NUMBER NOT NULL,
  5    Cycle_Name VARCHAR2(150) NOT NULL,
  6    Start_Date DATE NOT NULL,
  7    End_Date DATE NOT NULL,
  8    Is_Active NUMBER(1) DEFAULT 1,
  9    FOREIGN KEY (Exam_ID) REFERENCES EXAM(Exam_ID),
 10    FOREIGN KEY (Phase_ID) REFERENCES EXAM_PHASE(Phase_ID)
 11  );

Table created.
```

```
SQL> INSERT INTO REGION VALUES (1,'R1','South','Chennai','Chennai','TN','India','600001','IST');

1 row created.

SQL> INSERT INTO REGION VALUES (2,'R2','North','Delhi','Delhi','Delhi','India','110001','IST');

1 row created.

SQL> INSERT INTO REGION VALUES (3,'R3','West','Mumbai','Mumbai','MH','India','400001','IST');

1 row created.

SQL> INSERT INTO REGION VALUES (4,'R4','East','Kolkata','Kolkata','WB','India','700001','IST');

1 row created.

SQL> INSERT INTO REGION VALUES (5,'R5','Central','Bhopal','Bhopal','MP','India','462001','IST');

1 row created.

SQL> INSERT INTO REGION VALUES (6,'R6','South2','Bangalore','Bangalore','KA','India','560001','IST');

1 row created.

SQL> INSERT INTO REGION VALUES (7,'R7','North2','Chandigarh','Chandigarh','PB','India','160001','IST');

1 row created.

SQL> INSERT INTO REGION VALUES (8,'R8','West2','Ahmedabad','Ahmedabad','GJ','India','380001','IST');

1 row created.

SQL> INSERT INTO REGION VALUES (9,'R9','East2','Patna','Patna','BR','India','800001','IST');
```

1 row created.

SQL> INSERT INTO REGION VALUES (10,'R10','NE','Guwahati','Kamrup','AS','India','781001','IST');

1 row created.

SQL> INSERT INTO CATEGORY VALUES (1,'GEN','General',0);

1 row created.

SQL> INSERT INTO CATEGORY VALUES (2,'OBC','OBC',27);

1 row created.

SQL> INSERT INTO CATEGORY VALUES (3,'SC','SC',15);

1 row created.

SQL> INSERT INTO CATEGORY VALUES (4,'ST','ST',7.5);

1 row created.

SQL> INSERT INTO CATEGORY VALUES (5,'EWS','EWS',10);

1 row created.

SQL> INSERT INTO CATEGORY VALUES (6,'PWD','PWD',5);

1 row created.

SQL> INSERT INTO CATEGORY VALUES (7,'DEF','Defense',3);

1 row created.

```
SQL> INSERT INTO CATEGORY VALUES (8,'SPORT','Sports',2);

1 row created.

SQL> INSERT INTO CATEGORY VALUES (9,'MIN','Minority',1);

1 row created.

SQL> INSERT INTO CATEGORY VALUES (10,'INT','International',0);

1 row created.

SQL> INSERT INTO EXAM VALUES (1,'E1','Exam1','Govt','ONLINE','UG',2026,300);

1 row created.

SQL> INSERT INTO EXAM VALUES (2,'E2','Exam2','Govt','ONLINE','PG',2026,400);

1 row created.

SQL> INSERT INTO EXAM VALUES (3,'E3','Exam3','Private','OFFLINE','UG',2026,200);

1 row created.

SQL> INSERT INTO EXAM VALUES (4,'E4','Exam4','UPSC','OFFLINE','PG',2026,500);

1 row created.

SQL> INSERT INTO EXAM VALUES (5,'E5','Exam5','NTA','ONLINE','UG',2026,720);

1 row created.

SQL> INSERT INTO EXAM VALUES (6,'E6','Exam6','CLAT','ONLINE','UG',2026,150);
```

1 row created.

SQL> INSERT INTO EXAM VALUES (7,'E7','Exam7','IBPS','ONLINE','UG',2026,200);

1 row created.

SQL> INSERT INTO EXAM VALUES (8,'E8','Exam8','SSC','ONLINE','UG',2026,200);

1 row created.

SQL> INSERT INTO EXAM VALUES (9,'E9','Exam9','RRB','ONLINE','UG',2026,100);

1 row created.

SQL> INSERT INTO EXAM VALUES (10,'E10','Exam10','NDA','OFFLINE','UG',2026,300);

1 row created.

SQL> INSERT INTO EXAM_PHASE VALUES (1,1,1,'Phase1',DATE '2026-01-01',DATE '2026-02-01');

1 row created.

SQL> INSERT INTO EXAM_PHASE VALUES (2,2,1,'Phase1',DATE '2026-01-01',DATE '2026-02-01');

1 row created.

SQL> INSERT INTO EXAM_PHASE VALUES (3,3,1,'Phase1',DATE '2026-01-01',DATE '2026-02-01');

1 row created.

SQL> INSERT INTO EXAM_PHASE VALUES (4,4,1,'Phase1',DATE '2026-01-01',DATE '2026-02-01');

1 row created.

```
SQL> INSERT INTO EXAM_PHASE VALUES (5,5,1,'Phase1',DATE '2026-01-01',DATE '2026-02-01');

1 row created.

SQL> INSERT INTO EXAM_PHASE VALUES (6,6,1,'Phase1',DATE '2026-01-01',DATE '2026-02-01');

1 row created.

SQL> INSERT INTO EXAM_PHASE VALUES (7,7,1,'Phase1',DATE '2026-01-01',DATE '2026-02-01');

1 row created.

SQL> INSERT INTO EXAM_PHASE VALUES (8,8,1,'Phase1',DATE '2026-01-01',DATE '2026-02-01');

1 row created.

SQL> INSERT INTO EXAM_PHASE VALUES (9,9,1,'Phase1',DATE '2026-01-01',DATE '2026-02-01');

1 row created.

SQL> INSERT INTO EXAM_PHASE VALUES (10,10,1,'Phase1',DATE '2026-01-01',DATE '2026-02-01');

1 row created.

SQL> INSERT INTO SUBJECT VALUES (1,1,'S1','Math',180,100);

1 row created.

SQL> INSERT INTO SUBJECT VALUES (2,1,'S2','Physics',180,100);

1 row created.

SQL> INSERT INTO SUBJECT VALUES (3,1,'S3','Chemistry',180,100);
```

1 row created.

SQL> INSERT INTO SUBJECT VALUES (4,2,'S4','AdvMath',180,200);

1 row created.

SQL> INSERT INTO SUBJECT VALUES (5,3,'S5','Aptitude',120,100);

1 row created.

SQL> INSERT INTO SUBJECT VALUES (6,4,'S6','GS',180,250);

1 row created.

SQL> INSERT INTO SUBJECT VALUES (7,5,'S7','Biology',180,360);

1 row created.

SQL> INSERT INTO SUBJECT VALUES (8,6,'S8','Legal',120,150);

1 row created.

SQL> INSERT INTO SUBJECT VALUES (9,7,'S9','Quant',120,100);

1 row created.

SQL> INSERT INTO SUBJECT VALUES (10,8,'S10','Reasoning',120,100);

1 row created.

SQL> INSERT INTO EXAM_PATTERN VALUES (1,1,50,2,0.5,2,40);

1 row created.

```
SQL> INSERT INTO EXAM_PATTERN VALUES (2,2,50,2,0.5,2,40);

1 row created.

SQL> INSERT INTO EXAM_PATTERN VALUES (3,3,50,2,0.5,2,40);

1 row created.

SQL> INSERT INTO EXAM_PATTERN VALUES (4,4,100,2,0.25,3,50);

1 row created.

SQL> INSERT INTO EXAM_PATTERN VALUES (5,5,60,1,0.25,2,35);

1 row created.

SQL> INSERT INTO EXAM_PATTERN VALUES (6,6,80,3,0.5,4,50);

1 row created.

SQL> INSERT INTO EXAM_PATTERN VALUES (7,7,90,4,1,3,60);

1 row created.

SQL> INSERT INTO EXAM_PATTERN VALUES (8,8,70,2,0.5,2,40);

1 row created.

SQL> INSERT INTO EXAM_PATTERN VALUES (9,9,60,2,0.25,2,35);

1 row created.

SQL> INSERT INTO EXAM_PATTERN VALUES (10,10,50,2,0.5,2,40);
```

```
1 row created.

SQL> -- List centers by region
SQL> SELECT r.Region_Name, c.Center_Name
  2  FROM REGION r
  3  JOIN EXAM_CENTER c ON r.Region_ID = c.Region_ID;

no rows selected

SQL> INSERT INTO EXAM_CENTER VALUES (1,'C1','Center1',1,'Addr1','A2','Chennai','600001','GovT');

1 row created.

SQL> INSERT INTO EXAM_CENTER VALUES (2,'C2','Center2',2,'Addr1','A2','Delhi','110001','GovT');

1 row created.

SQL> INSERT INTO EXAM_CENTER VALUES (3,'C3','Center3',3,'Addr1','A2','Mumbai','400001','Private');

1 row created.

SQL> INSERT INTO EXAM_CENTER VALUES (4,'C4','Center4',4,'Addr1','A2','Kolkata','700001','GovT');

1 row created.

SQL> INSERT INTO EXAM_CENTER VALUES (5,'C5','Center5',5,'Addr1','A2','Bhopal','462001','Private');

1 row created.

SQL> INSERT INTO EXAM_CENTER VALUES (6,'C6','Center6',6,'Addr1','A2','Bangalore','560001','Private');

1 row created.
```

```
SQL> INSERT INTO EXAM_CENTER VALUES (7,'C7','Center7',7,'Addr1','A2','Chandigarh','160001','GovT');

1 row created.

SQL> INSERT INTO EXAM_CENTER VALUES (8,'C8','Center8',8,'Addr1','A2','Ahmedabad','380001','Private');

1 row created.

SQL> INSERT INTO EXAM_CENTER VALUES (9,'C9','Center9',9,'Addr1','A2','Patna','800001','GovT');

1 row created.

SQL> INSERT INTO EXAM_CENTER VALUES (10,'C10','Center10',10,'Addr1','A2','Guwahati','781001','Private');

1 row created.

SQL> INSERT INTO ROOM VALUES (1,'R1',1,1,30,1,1,1,'Exam',10);

1 row created.

SQL> INSERT INTO ROOM VALUES (2,'R2',1,2,25,1,0,1,'Exam',10);

1 row created.

SQL> INSERT INTO ROOM VALUES (3,'R3',2,1,30,1,1,1,'Exam',10);

1 row created.

SQL> INSERT INTO ROOM VALUES (4,'R4',3,1,40,1,1,1,'Exam',10);

1 row created.

SQL> INSERT INTO ROOM VALUES (5,'R5',4,1,35,1,0,1,'Exam',10);
```

1 row created.

SQL> INSERT INTO ROOM VALUES (6,'R6',5,1,30,0,1,1,'Exam',10);

1 row created.

SQL> INSERT INTO ROOM VALUES (7,'R7',6,2,45,1,1,1,'Exam',10);

1 row created.

SQL> INSERT INTO ROOM VALUES (8,'R8',7,1,30,1,0,1,'Exam',10);

1 row created.

SQL> INSERT INTO ROOM VALUES (9,'R9',8,1,25,1,1,1,'Exam',10);

1 row created.

SQL> INSERT INTO ROOM VALUES (10,'R10',9,1,20,1,1,1,'Exam',10);

1 row created.

SQL> INSERT INTO HOLIDAY_CALENDAR VALUES (1,1,DATE '2026-01-26','Republic Day',1);

1 row created.

SQL> INSERT INTO HOLIDAY_CALENDAR VALUES (2,2,DATE '2026-08-15','Independence Day',1);

1 row created.

SQL> INSERT INTO HOLIDAY_CALENDAR VALUES (3,3,DATE '2026-10-02','Gandhi Jayanti',1);

1 row created.

```
SQL> INSERT INTO HOLIDAY_CALENDAR VALUES (4,4,DATE '2026-12-25','Christmas',1);

1 row created.

SQL> INSERT INTO HOLIDAY_CALENDAR VALUES (5,5,DATE '2026-11-01','State Day',0);

1 row created.

SQL> INSERT INTO HOLIDAY_CALENDAR VALUES (6,6,DATE '2026-01-14','Pongal',0);

1 row created.

SQL> INSERT INTO HOLIDAY_CALENDAR VALUES (7,7,DATE '2026-04-13','Baisakhi',0);

1 row created.

SQL> INSERT INTO HOLIDAY_CALENDAR VALUES (8,8,DATE '2026-10-24','Diwali',1);

1 row created.

SQL> INSERT INTO HOLIDAY_CALENDAR VALUES (9,9,DATE '2026-03-29','Holi',1);

1 row created.

SQL> INSERT INTO HOLIDAY_CALENDAR VALUES (10,10,DATE '2026-09-10','Regional Fest',0);

1 row created.

SQL> INSERT INTO EXAM_CYCLE VALUES (1,1,1,'Cycle1',DATE '2026-01-01',DATE '2026-06-01',1);

1 row created.

SQL> INSERT INTO EXAM_CYCLE VALUES (2,2,2,'Cycle2',DATE '2026-01-01',DATE '2026-06-01',1);
```

1 row created.

SQL> INSERT INTO EXAM_CYCLE VALUES (3,3,3,'Cycle3',DATE '2026-01-01',DATE '2026-06-01',1);

1 row created.

SQL> INSERT INTO EXAM_CYCLE VALUES (4,4,4,'Cycle4',DATE '2026-01-01',DATE '2026-06-01',1);

1 row created.

SQL> INSERT INTO EXAM_CYCLE VALUES (5,5,5,'Cycle5',DATE '2026-01-01',DATE '2026-06-01',1);

1 row created.

SQL> INSERT INTO EXAM_CYCLE VALUES (6,6,6,'Cycle6',DATE '2026-01-01',DATE '2026-06-01',1);

1 row created.

SQL> INSERT INTO EXAM_CYCLE VALUES (7,7,7,'Cycle7',DATE '2026-01-01',DATE '2026-06-01',1);

1 row created.

SQL> INSERT INTO EXAM_CYCLE VALUES (8,8,8,'Cycle8',DATE '2026-01-01',DATE '2026-06-01',1);

1 row created.

SQL> INSERT INTO EXAM_CYCLE VALUES (9,9,9,'Cycle9',DATE '2026-01-01',DATE '2026-06-01',1);

1 row created.

SQL> INSERT INTO EXAM_CYCLE VALUES (10,10,10,'Cycle10',DATE '2026-01-01',DATE '2026-06-01',1);

1 row created.

## (ii) APPLIED SQL QUERIES AND USE CASES – MASTER REFERENCE DATA MODULE:

1. **Count number of centers per region**
2. **Total seating capacity per center**
3. **Find rooms with full facilities (CCTV + Biometric + Power backup)**
4. **Exams conducted by a specific body**
5. **Subjects under each exam (with total marks)**
6. **Total marks distribution per exam**
7. **Find subjects with highest duration**
8. **Exam patterns with high negative marking**
9. **Pass percentage analysis**
10. **Active exam cycles**
11. **Exams currently open for registration**
12. **Holidays in a region**
13. **Check if a date is a holiday**
14. **Centers in regions with holidays on a specific date**
15. **Average room capacity per center**
16. **Regions with no exam centers**
17. **Total rooms per region**
18. **Exams with multiple phases**
19. **Top regions by infrastructure**

```
SQL> -- Find all exam centers in a given state

SQL> -- Used by admin to list centers for allocation planning

SQL> SELECT Center_Name, City, Pincode

  2  FROM EXAM_CENTER ec

  3  JOIN REGION r ON ec.Region_ID = r.Region_ID

  4  WHERE r.State = 'TN';

SELECT Center_Name, City, Pincode

                *

ERROR at line 1:

ORA-00918: column ambiguously defined
```

```
SQL> Count number of centers per region

SP2-0734: unknown command beginning "Count numb..." - rest of line ignored.

SQL> -- Used to analyze infrastructure distribution

SQL> SELECT r.Region_Name, COUNT(ec.Center_ID) AS total_centers

  2  FROM REGION r

  3  LEFT JOIN EXAM_CENTER ec ON r.Region_ID = ec.Region_ID

  4  GROUP BY r.Region_Name

  5  ORDER BY total_centers DESC;


REGION_NAME

--------------------------------------------------------------------------------

TOTAL_CENTERS

-------------

South

         1


North

         1


West

         1



REGION_NAME

--------------------------------------------------------------------------------

TOTAL_CENTERS

-------------

East

         1


NE

         1


South2
```

```
         1



REGION_NAME
--------------------------------------------------------------------------------
TOTAL_CENTERS
-------------
North2
         1

West2
         1

East2
         1



REGION_NAME
--------------------------------------------------------------------------------
TOTAL_CENTERS
-------------
Central
         1



10 rows selected.

SQL> -- Find all exam centers in a given state
SQL> SELECT ec.Center_Name, ec.City, ec.Pincode
  2  FROM EXAM_CENTER ec
  3  JOIN REGION r ON ec.Region_ID = r.Region_ID
  4  WHERE r.State = 'TN';

CENTER_NAME
```

```
--------------------------------------------------------------------------------
CITY
--------------------------------------------------------------------------------
PINCODE
----------
Center1
Chennai
600001


SQL> -- Total seating capacity per center
SQL> -- Used to decide student allocation
SQL> SELECT ec.Center_Name,
  2       SUM(rm.Capacity) AS total_capacity
  3  FROM EXAM_CENTER ec
  4  JOIN ROOM rm ON ec.Center_ID = rm.Center_ID
  5  GROUP BY ec.Center_Name
  6  ORDER BY total_capacity DESC;

CENTER_NAME
--------------------------------------------------------------------------------
TOTAL_CAPACITY
--------------
Center1
         55

Center6
         45

Center3
         40


CENTER_NAME
```

```
--------------------------------------------------------------------------------
TOTAL_CAPACITY
--------------
Center4
        35

Center2
        30

Center5
        30


CENTER_NAME
--------------------------------------------------------------------------------
TOTAL_CAPACITY
--------------
Center7
        30

Center8
        25

Center9
        20


9 rows selected.

SQL> -- Find rooms with full facilities (CCTV + Biometric + Power backup)
SQL> -- Used for high-security exams
SQL> SELECT Room_Code, Center_ID
  2  FROM ROOM
  3  WHERE Has_CCTV = 1
```

```
  4   AND Has_Biometric = 1
  5   AND Power_Backup_Available = 1;

ROOM_CODE          CENTER_ID
-------------------- ----------
R1                          1
R3                          2
R4                          3
R7                          6
R9                          8
R10                          9

6 rows selected.

SQL> -- Exams conducted by a specific body
SQL> -- Used for filtering exams (e.g., UPSC, NTA)
SQL> SELECT Exam_Name, Year
  2  FROM EXAM
  3  WHERE Conducting_Body = 'UPSC';

EXAM_NAME
--------------------------------------------------------------------------------
    YEAR
----------
Exam4
    2026


SQL> -- Subjects under each exam (with total marks)
SQL> -- Used for exam structure display
SQL> SELECT e.Exam_Name, s.Subject_Name, s.Max_Marks
  2  FROM EXAM e
  3  JOIN SUBJECT s ON e.Exam_ID = s.Exam_ID
  4  ORDER BY e.Exam_Name;
```

EXAM_NAME

---

SUBJECT_NAME

---

MAX_MARKS

---

Exam1

Math

100

Exam1

Physics

100

EXAM_NAME

---

SUBJECT_NAME

---

MAX_MARKS

---

Exam1

Chemistry

100

Exam2

AdvMath

EXAM_NAME

---

SUBJECT_NAME

---

MAX_MARKS

---

200

Exam3

Aptitude

100

Exam4

EXAM_NAME

---

SUBJECT_NAME

---

MAX_MARKS

---

GS

250

Exam5

Biology

360

EXAM_NAME

---

SUBJECT_NAME

---

MAX_MARKS

---

Exam6

Legal

150

Exam7

```
Quant
     100


EXAM_NAME
--------------------------------------------------------------------------------
SUBJECT_NAME
--------------------------------------------------------------------------------
 MAX_MARKS
----------


Exam8
Reasoning
     100



10 rows selected.


SQL> -- Total marks distribution per exam
SQL> -- Validate if subject marks match exam total
SQL> SELECT e.Exam_Name,
  2       SUM(s.Max_Marks) AS subject_total,
  3       e.Total_Marks
  4  FROM EXAM e
  5  JOIN SUBJECT s ON e.Exam_ID = s.Exam_ID
  6  GROUP BY e.Exam_Name, e.Total_Marks;


EXAM_NAME
--------------------------------------------------------------------------------
SUBJECT_TOTAL TOTAL_MARKS
------------- -----------
Exam1
        300        300


Exam2
```

```
        200       400


Exam3
        100       200



EXAM_NAME
--------------------------------------------------------------------------------
SUBJECT_TOTAL TOTAL_MARKS
------------- -----------
Exam4
        250       500


Exam5
        360       720


Exam6
        150       150



EXAM_NAME
--------------------------------------------------------------------------------
SUBJECT_TOTAL TOTAL_MARKS
------------- -----------
Exam7
        100       200


Exam8
        100       200



8 rows selected.


SQL> -- Find subjects with highest duration
```

```
SQL> -- Used for scheduling decisions
SQL> SELECT Subject_Name, Duration_Minutes
  2  FROM SUBJECT
  3  ORDER BY Duration_Minutes DESC;

SUBJECT_NAME
--------------------------------------------------------------------------------
DURATION_MINUTES
----------------
Chemistry
         180

Physics
         180

Biology
         180


SUBJECT_NAME
--------------------------------------------------------------------------------
DURATION_MINUTES
----------------
AdvMath
         180

Math
         180

GS
         180


SUBJECT_NAME
```

```
--------------------------------------------------------------------------------
DURATION_MINUTES
----------------
Quant
        120

Legal
        120

Reasoning
        120


SUBJECT_NAME
--------------------------------------------------------------------------------
DURATION_MINUTES
----------------
Aptitude
        120


10 rows selected.

SQL> -- Exam patterns with high negative marking
SQL> -- Used to identify difficult exams
SQL> SELECT s.Subject_Name, ep.Negative_Marking
  2  FROM EXAM_PATTERN ep
  3  JOIN SUBJECT s ON ep.Subject_ID = s.Subject_ID
  4  WHERE ep.Negative_Marking > 0.5;

SUBJECT_NAME
--------------------------------------------------------------------------------
NEGATIVE_MARKING
----------------
```

```
Biology
          1



SQL> -- Pass percentage analysis
SQL> -- Used to evaluate exam difficulty
SQL> SELECT s.Subject_Name, ep.Pass_Percentage
  2  FROM EXAM_PATTERN ep
  3  JOIN SUBJECT s ON ep.Subject_ID = s.Subject_ID
  4  ORDER BY ep.Pass_Percentage;

SUBJECT_NAME
--------------------------------------------------------------------------------
PASS_PERCENTAGE
---------------
Aptitude
         35

Quant
         35

Chemistry
         40



SUBJECT_NAME
--------------------------------------------------------------------------------
PASS_PERCENTAGE
---------------
Reasoning
         40

Legal
         40
```

```
Physics
        40


SUBJECT_NAME
--------------------------------------------------------------------------------
PASS_PERCENTAGE
---------------
Math
        40

AdvMath
        50

GS
        50


SUBJECT_NAME
--------------------------------------------------------------------------------
PASS_PERCENTAGE
---------------
Biology
        60


10 rows selected.

SQL> -- Active exam cycles
SQL> -- Used in live system dashboard
SQL> SELECT Cycle_Name, Start_Date, End_Date
  2  FROM EXAM_CYCLE
  3  WHERE Is_Active = 1;
```

CYCLE_NAME

---

START_DAT END_DATE

--- ---

Cycle1

01-JAN-26 01-JUN-26

Cycle2

01-JAN-26 01-JUN-26

Cycle3

01-JAN-26 01-JUN-26

CYCLE_NAME

---

START_DAT END_DATE

--- ---

Cycle4

01-JAN-26 01-JUN-26

Cycle5

01-JAN-26 01-JUN-26

Cycle6

01-JAN-26 01-JUN-26

CYCLE_NAME

---

START_DAT END_DATE

--- ---

Cycle7

01-JAN-26 01-JUN-26

Cycle8

01-JAN-26 01-JUN-26

Cycle9

01-JAN-26 01-JUN-26

CYCLE_NAME

---

START_DAT END_DATE

--- ---

Cycle10

01-JAN-26 01-JUN-26

10 rows selected.

```
SQL> -- Exams currently open for registration
SQL> -- Critical real-world query (portal usage)
SQL> SELECT e.Exam_Name, ep.Phase_Name
  2  FROM EXAM e
  3  JOIN EXAM_PHASE ep ON e.Exam_ID = ep.Exam_ID
  4  WHERE SYSDATE BETWEEN ep.Registration_Start_Date AND ep.Registration_End_Date;
```

no rows selected

```
SQL> -- Holidays in a region
SQL> -- Used to avoid scheduling exams
SQL> SELECT Holiday_Name, Holiday_Date
  2  FROM HOLIDAY_CALENDAR
  3  WHERE Region_ID = 1;
```

```
HOLIDAY_NAME
--------------------------------------------------------------------------------
HOLIDAY_D
---------
Republic Day
26-JAN-26


SQL> -- Check if a date is a holiday
SQL> -- Used before scheduling an exam
SQL> SELECT COUNT(*) AS is_holiday
  2  FROM HOLIDAY_CALENDAR
  3  WHERE Holiday_Date = DATE '2026-01-26';

IS_HOLIDAY
----------
         1

SQL> -- Centers in regions with holidays on a specific date
SQL> -- Avoid scheduling in affected regions
SQL> SELECT ec.Center_Name, r.Region_Name
  2  FROM EXAM_CENTER ec
  3  JOIN REGION r ON ec.Region_ID = r.Region_ID
  4  JOIN HOLIDAY_CALENDAR hc ON r.Region_ID = hc.Region_ID
  5  WHERE hc.Holiday_Date = DATE '2026-01-26';

CENTER_NAME
--------------------------------------------------------------------------------
REGION_NAME
--------------------------------------------------------------------------------
Center1
South
```

```
SQL> -- Average room capacity per center
SQL> -- Infrastructure quality analysis
SQL> SELECT ec.Center_Name,
  2       AVG(rm.Capacity) AS avg_capacity
  3  FROM EXAM_CENTER ec
  4  JOIN ROOM rm ON ec.Center_ID = rm.Center_ID
  5  GROUP BY ec.Center_Name;

CENTER_NAME
--------------------------------------------------------------------------------
AVG_CAPACITY
------------
Center1
      27.5

Center2
        30

Center3
        40


CENTER_NAME
--------------------------------------------------------------------------------
AVG_CAPACITY
------------
Center4
        35

Center5
        30

Center6
        45
```

```
CENTER_NAME
--------------------------------------------------------------------------------
AVG_CAPACITY
------------
Center7
        30

Center8
        25

Center9
        20



9 rows selected.

SQL> -- Regions with no exam centers
SQL> -- Identify missing infrastructure
SQL> SELECT r.Region_Name
  2  FROM REGION r
  3  LEFT JOIN EXAM_CENTER ec ON r.Region_ID = ec.Region_ID
  4  WHERE ec.Center_ID IS NULL;

no rows selected

SQL> -- Total rooms per region
SQL> -- Resource planning
SQL> SELECT r.Region_Name,
  2       COUNT(rm.Room_ID) AS total_rooms
  3  FROM REGION r
  4  JOIN EXAM_CENTER ec ON r.Region_ID = ec.Region_ID
  5  JOIN ROOM rm ON ec.Center_ID = rm.Center_ID
```

```
  6  GROUP BY r.Region_Name;

REGION_NAME
--------------------------------------------------------------------------------
TOTAL_ROOMS
-----------
South
         2

North
         1

West
         1


REGION_NAME
--------------------------------------------------------------------------------
TOTAL_ROOMS
-----------
East
         1

Central
         1

South2
         1


REGION_NAME
--------------------------------------------------------------------------------
TOTAL_ROOMS
-----------
```

```
North2
        1

West2
        1

East2
        1


9 rows selected.

SQL> -- Exams with multiple phases
SQL> -- Multi-phase exam detection
SQL> SELECT Exam_ID, COUNT(*) AS phase_count
  2  FROM EXAM_PHASE
  3  GROUP BY Exam_ID
  4  HAVING COUNT(*) > 1;

no rows selected

SQL> -- Top regions by infrastructure (centers + rooms)
SQL> -- Strategic planning query
SQL> SELECT r.Region_Name,
  2       COUNT(DISTINCT ec.Center_ID) AS centers,
  3       COUNT(rm.Room_ID) AS rooms
  4  FROM REGION r
  5  LEFT JOIN EXAM_CENTER ec ON r.Region_ID = ec.Region_ID
  6  LEFT JOIN ROOM rm ON ec.Center_ID = rm.Center_ID
  7  GROUP BY r.Region_Name
  8  ORDER BY rooms DESC;

REGION_NAME
--------------------------------------------------------------------------------
```

CENTERS ROOMS

---------- ----------

South

1 2

West

1 1

East

1 1

REGION_NAME

---

CENTERS ROOMS

---------- ----------

North

1 1

Central

1 1

South2

1 1

REGION_NAME

---

CENTERS ROOMS

---------- ----------

North2

1 1

West2

```
      1        1


East2

      1        1



REGION_NAME

--------------------------------------------------------------------------------

  CENTERS     ROOMS

---------- ----------

NE

      1        0



10 rows selected.
```

## IMPLEMENTATION OF MODULE 2 - STUDENT REGISTRATION MODULE

### (i) DESIGN AND IMPLEMENTATION OF ENTITIES WITH DATA INITIALIZATION:

**This module focuses on the creation and management of student-related entities, including the registration of student records and the insertion of essential data required to maintain accurate and structured student information within the system.**

```
SQL> CREATE TABLE STUDENT (
  2    Student_ID NUMBER PRIMARY KEY,
  3    First_Name VARCHAR2(100) NOT NULL,
  4    Middle_Name VARCHAR2(100),
  5    Last_Name VARCHAR2(100) NOT NULL,
  6    Gender CHAR(1) CHECK (Gender IN ('M','F','O')) NOT NULL,
  7    DOB DATE NOT NULL,
  8    Email VARCHAR2(150) UNIQUE NOT NULL,
  9    Phone_Number VARCHAR2(15) NOT NULL,
 10    Alternate_Phone VARCHAR2(15),
```

```
11    Nationality VARCHAR2(100) NOT NULL,
12    Aadhaar_Number CHAR(12) UNIQUE NOT NULL,
13    Category_ID NUMBER,
14    PwD_Flag NUMBER(1) DEFAULT 0,
15    Domicile_Region_ID NUMBER,
16    FOREIGN KEY (Category_ID) REFERENCES CATEGORY(Category_ID),
17    FOREIGN KEY (Domicile_Region_ID) REFERENCES REGION(Region_ID)
18  );
```

Table created.

```
SQL> CREATE TABLE STUDENT_ADDRESS (
 2    Address_ID NUMBER PRIMARY KEY,
 3    Student_ID NUMBER NOT NULL,
 4    Address_Type VARCHAR2(50) NOT NULL,
 5    Address_Line1 VARCHAR2(200) NOT NULL,
 6    Address_Line2 VARCHAR2(200),
 7    Region_ID NUMBER NOT NULL,
 8    FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
 9    FOREIGN KEY (Region_ID) REFERENCES REGION(Region_ID)
10  );
```

Table created.

```
SQL> CREATE TABLE STUDENT_ACADEMIC_PROFILE (
 2    Academic_Profile_ID NUMBER PRIMARY KEY,
 3    Student_ID NUMBER NOT NULL,
 4    Qualification_Level VARCHAR2(100) NOT NULL,
 5    Board_University_Name VARCHAR2(200) NOT NULL,
 6    Institution_Name VARCHAR2(200) NOT NULL,
 7    Year_of_Passing NUMBER(4) NOT NULL,
 8    Marks_Obtained NUMBER(6,2) NOT NULL,
 9    Maximum_Marks NUMBER(6,2) NOT NULL,
10    CGPA NUMBER(4,2),
```

```
11     Stream VARCHAR2(100),
12     Is_Primary_Qualification NUMBER(1) DEFAULT 0,
13     FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID)
14  );
```

Table created.

```
SQL> CREATE TABLE STUDENT_DOCUMENT (
 2     Document_ID NUMBER PRIMARY KEY,
 3     Student_ID NUMBER NOT NULL,
 4     Document_Type VARCHAR2(100) NOT NULL,
 5     File_Path VARCHAR2(500) NOT NULL,
 6     File_Size NUMBER NOT NULL,
 7     Upload_Date TIMESTAMP NOT NULL,
 8     Verification_Status VARCHAR2(50) NOT NULL,
 9     Verified_By NUMBER,
10     Verification_Date TIMESTAMP,
11     Rejection_Reason CLOB,
12     FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID)
13  );
```

Table created.

```
SQL> CREATE TABLE REGISTRATION (
 2     Registration_ID NUMBER PRIMARY KEY,
 3     Student_ID NUMBER NOT NULL,
 4     Exam_ID NUMBER NOT NULL,
 5     Application_Number VARCHAR2(50) UNIQUE NOT NULL,
 6     Registration_Date TIMESTAMP NOT NULL,
 7     Attempt_Number NUMBER NOT NULL,
 8     Is_First_Attempt NUMBER(1) DEFAULT 1,
 9     Status VARCHAR2(50) NOT NULL,
10     Eligibility_Status VARCHAR2(50) NOT NULL,
11     Fee_Category VARCHAR2(50) NOT NULL,
```

```
  Total_Fee NUMBER(10,2) NOT NULL,
  Payment_Status VARCHAR2(50) NOT NULL,
  Admit_Card_Generated_Flag NUMBER(1) DEFAULT 0,
  Registration_IP_Address VARCHAR2(45),
  Registration_Device_ID VARCHAR2(100),
  FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
  FOREIGN KEY (Exam_ID) REFERENCES EXAM(Exam_ID)
);
```

Table created.

```
SQL> CREATE TABLE SUBJECT_SELECTION (
  Registration_ID NUMBER NOT NULL,
  Subject_ID NUMBER NOT NULL,
  Preference_Order NUMBER NOT NULL,
  Is_Optional NUMBER(1) DEFAULT 0,
  PRIMARY KEY (Registration_ID, Subject_ID),
  FOREIGN KEY (Registration_ID) REFERENCES REGISTRATION(Registration_ID),
  FOREIGN KEY (Subject_ID) REFERENCES SUBJECT(Subject_ID)
);
```

Table created.

```
SQL> CREATE TABLE CENTER_PREFERENCE (
  Registration_ID NUMBER NOT NULL,
  Center_ID NUMBER NOT NULL,
  Preference_Rank NUMBER NOT NULL,
  PRIMARY KEY (Registration_ID, Center_ID),
  FOREIGN KEY (Registration_ID) REFERENCES REGISTRATION(Registration_ID),
  FOREIGN KEY (Center_ID) REFERENCES EXAM_CENTER(Center_ID)
);
```

Table created.

```
SQL> CREATE TABLE PAYMENT_TRANSACTION (
  2    Transaction_ID NUMBER PRIMARY KEY,
  3    Registration_ID NUMBER NOT NULL,
  4    Payment_Gateway_Name VARCHAR2(100) NOT NULL,
  5    Payment_Mode VARCHAR2(50) NOT NULL,
  6    Transaction_Reference_Number VARCHAR2(100) UNIQUE NOT NULL,
  7    Amount_Paid NUMBER(10,2) NOT NULL,
  8    Payment_Date TIMESTAMP NOT NULL,
  9    Payment_Status VARCHAR2(50) NOT NULL,
 10    Failure_Reason CLOB,
 11    Refund_Status VARCHAR2(50),
 12    Refund_Date TIMESTAMP,
 13    FOREIGN KEY (Registration_ID) REFERENCES REGISTRATION(Registration_ID)
 14  );

Table created.

SQL> CREATE TABLE BIOMETRIC_PROFILE (
  2    Student_ID NUMBER PRIMARY KEY,
  3    Face_Embedding_Blob BLOB,
  4    Fingerprint_Hash VARCHAR2(256),
  5    Iris_Hash VARCHAR2(256),
  6    Capture_Device_ID VARCHAR2(100) NOT NULL,
  7    Capture_Date TIMESTAMP NOT NULL,
  8    Last_Updated TIMESTAMP NOT NULL,
  9    Verification_Status VARCHAR2(50) NOT NULL,
 10    FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID)
 11  );

Table created.

SQL> CREATE TABLE REGISTRATION_AUDIT_LOG (
  2    Log_ID NUMBER PRIMARY KEY,
  3    Registration_ID NUMBER NOT NULL,
```

```
  4   Action_Type VARCHAR2(100) NOT NULL,
  5   Action_Timestamp TIMESTAMP NOT NULL,
  6   Action_By NUMBER NOT NULL,
  7   IP_Address VARCHAR2(45),
  8   Device_Info VARCHAR2(200),
  9   Remarks CLOB,
 10   FOREIGN KEY (Registration_ID) REFERENCES REGISTRATION(Registration_ID)
 11 );
```

Table created.

```
SQL> CREATE TABLE ATTEMPT_HISTORY (
  2   Student_ID NUMBER NOT NULL,
  3   Exam_ID NUMBER NOT NULL,
  4   Attempt_Number NUMBER NOT NULL,
  5   Result_Status VARCHAR2(50) NOT NULL,
  6   Score NUMBER(8,2),
  7   Rank NUMBER,
  8   PRIMARY KEY (Student_ID, Exam_ID, Attempt_Number),
  9   FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
 10   FOREIGN KEY (Exam_ID) REFERENCES EXAM(Exam_ID)
 11 );
```

Table created.

```
SQL> INSERT INTO STUDENT VALUES (1,'A','B','C','M',DATE '2000-01-
01','a1@mail.com','9001',NULL,'India','111111111111',1,0,1);
```

1 row created.

```
SQL> INSERT INTO STUDENT VALUES (2,'A','B','C','F',DATE '2000-01-
02','a2@mail.com','9002',NULL,'India','222222222222',2,0,2);
```

1 row created.

```
SQL> INSERT INTO STUDENT VALUES (3,'A','B','C','M',DATE '2000-01-
03','a3@mail.com','9003',NULL,'India','333333333333',3,0,3);

1 row created.

SQL> INSERT INTO STUDENT VALUES (4,'A','B','C','F',DATE '2000-01-
04','a4@mail.com','9004',NULL,'India','444444444444',1,1,1);

1 row created.

SQL> INSERT INTO STUDENT VALUES (5,'A','B','C','M',DATE '2000-01-
05','a5@mail.com','9005',NULL,'India','555555555555',2,0,2);

1 row created.

SQL> INSERT INTO STUDENT VALUES (6,'A','B','C','F',DATE '2000-01-
06','a6@mail.com','9006',NULL,'India','666666666666',3,0,3);

1 row created.

SQL> INSERT INTO STUDENT VALUES (7,'A','B','C','M',DATE '2000-01-
07','a7@mail.com','9007',NULL,'India','777777777777',4,0,4);

1 row created.

SQL> INSERT INTO STUDENT VALUES (8,'A','B','C','F',DATE '2000-01-
08','a8@mail.com','9008',NULL,'India','888888888888',5,0,5);

1 row created.

SQL> INSERT INTO STUDENT VALUES (9,'A','B','C','M',DATE '2000-01-
09','a9@mail.com','9009',NULL,'India','999999999999',1,0,1);

1 row created.
```

```
SQL> INSERT INTO STUDENT VALUES (10,'A','B','C','F',DATE '2000-01-
10','a10@mail.com','9010',NULL,'India','101010101010',2,0,2);

1 row created.

SQL> INSERT INTO STUDENT_ADDRESS VALUES (1,1,'HOME','Addr1',NULL,1);

1 row created.

SQL> INSERT INTO STUDENT_ADDRESS VALUES (2,2,'HOME','Addr1',NULL,2);

1 row created.

SQL> INSERT INTO STUDENT_ADDRESS VALUES (3,3,'HOME','Addr1',NULL,3);

1 row created.

SQL> INSERT INTO STUDENT_ADDRESS VALUES (4,4,'HOME','Addr1',NULL,1);

1 row created.

SQL> INSERT INTO STUDENT_ADDRESS VALUES (5,5,'HOME','Addr1',NULL,2);

1 row created.

SQL> INSERT INTO STUDENT_ADDRESS VALUES (6,6,'HOME','Addr1',NULL,3);

1 row created.

SQL> INSERT INTO STUDENT_ADDRESS VALUES (7,7,'HOME','Addr1',NULL,4);

1 row created.

SQL> INSERT INTO STUDENT_ADDRESS VALUES (8,8,'HOME','Addr1',NULL,5);
```

```
1 row created.

SQL> INSERT INTO STUDENT_ADDRESS VALUES (9,9,'HOME','Addr1',NULL,1);

1 row created.

SQL> INSERT INTO STUDENT_ADDRESS VALUES (10,10,'HOME','Addr1',NULL,2);

1 row created.

SQL> INSERT INTO STUDENT_ACADEMIC_PROFILE VALUES
(1,1,'12th','CBSE','School1',2018,450,500,NULL,'Science',1);

1 row created.

SQL> INSERT INTO STUDENT_ACADEMIC_PROFILE VALUES
(2,2,'12th','CBSE','School2',2018,440,500,NULL,'Science',1);

1 row created.

SQL> INSERT INTO STUDENT_ACADEMIC_PROFILE VALUES
(3,3,'12th','CBSE','School3',2018,430,500,NULL,'Science',1);

1 row created.

SQL> INSERT INTO STUDENT_ACADEMIC_PROFILE VALUES
(4,4,'12th','CBSE','School4',2018,420,500,NULL,'Science',1);

1 row created.

SQL> INSERT INTO STUDENT_ACADEMIC_PROFILE VALUES
(5,5,'12th','CBSE','School5',2018,410,500,NULL,'Science',1);

1 row created.
```

```
SQL> INSERT INTO STUDENT_ACADEMIC_PROFILE VALUES
(6,6,'12th','CBSE','School6',2018,400,500,NULL,'Science',1);

1 row created.

SQL> INSERT INTO STUDENT_ACADEMIC_PROFILE VALUES
(7,7,'12th','CBSE','School7',2018,390,500,NULL,'Science',1);

1 row created.

SQL> INSERT INTO STUDENT_ACADEMIC_PROFILE VALUES
(8,8,'12th','CBSE','School8',2018,380,500,NULL,'Science',1);

1 row created.

SQL> INSERT INTO STUDENT_ACADEMIC_PROFILE VALUES
(9,9,'12th','CBSE','School9',2018,370,500,NULL,'Science',1);

1 row created.

SQL> INSERT INTO STUDENT_ACADEMIC_PROFILE VALUES
(10,10,'12th','CBSE','School10',2018,360,500,NULL,'Science',1);

1 row created.

SQL> INSERT INTO STUDENT_DOCUMENT VALUES
(1,1,'AADHAAR','/docs/a1.pdf',50000,SYSTIMESTAMP,'VERIFIED',101,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO STUDENT_DOCUMENT VALUES
(2,2,'AADHAAR','/docs/a2.pdf',50000,SYSTIMESTAMP,'VERIFIED',102,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO STUDENT_DOCUMENT VALUES
(3,3,'AADHAAR','/docs/a3.pdf',50000,SYSTIMESTAMP,'PENDING',NULL,NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO STUDENT_DOCUMENT VALUES
(4,4,'AADHAAR','/docs/a4.pdf',50000,SYSTIMESTAMP,'VERIFIED',104,SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO STUDENT_DOCUMENT VALUES
(5,5,'AADHAAR','/docs/a5.pdf',50000,SYSTIMESTAMP,'REJECTED',105,SYSTIMESTAMP,'Blurred');
```

1 row created.

```
SQL> INSERT INTO STUDENT_DOCUMENT VALUES
(6,6,'AADHAAR','/docs/a6.pdf',50000,SYSTIMESTAMP,'VERIFIED',106,SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO STUDENT_DOCUMENT VALUES
(7,7,'AADHAAR','/docs/a7.pdf',50000,SYSTIMESTAMP,'PENDING',NULL,NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO STUDENT_DOCUMENT VALUES
(8,8,'AADHAAR','/docs/a8.pdf',50000,SYSTIMESTAMP,'VERIFIED',108,SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO STUDENT_DOCUMENT VALUES
(9,9,'AADHAAR','/docs/a9.pdf',50000,SYSTIMESTAMP,'VERIFIED',109,SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO STUDENT_DOCUMENT VALUES
(10,10,'AADHAAR','/docs/a10.pdf',50000,SYSTIMESTAMP,'PENDING',NULL,NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO REGISTRATION VALUES
(1,1,1,'APP001',SYSTIMESTAMP,1,1,'SUBMITTED','ELIGIBLE','GEN',1000,'PAID',1,'192.168.1.1','DEV1');
```

1 row created.

```
SQL> INSERT INTO REGISTRATION VALUES
(2,2,1,'APP002',SYSTIMESTAMP,1,1,'SUBMITTED','ELIGIBLE','OBC',800,'PAID',1,'192.168.1.2','DEV2');
```

1 row created.

```
SQL> INSERT INTO REGISTRATION VALUES
(3,3,1,'APP003',SYSTIMESTAMP,1,1,'SUBMITTED','ELIGIBLE','SC',500,'FAILED',0,'192.168.1.3','DEV3');
```

1 row created.

```
SQL> INSERT INTO REGISTRATION VALUES
(4,4,2,'APP004',SYSTIMESTAMP,1,1,'SUBMITTED','ELIGIBLE','GEN',1000,'PAID',1,'192.168.1.4','DEV4');
```

1 row created.

```
SQL> INSERT INTO REGISTRATION VALUES
(5,5,2,'APP005',SYSTIMESTAMP,2,0,'SUBMITTED','ELIGIBLE','OBC',800,'PAID',1,'192.168.1.5','DEV5');
```

1 row created.

```
SQL> INSERT INTO REGISTRATION VALUES
(6,6,3,'APP006',SYSTIMESTAMP,1,1,'SUBMITTED','ELIGIBLE','ST',500,'PAID',1,'192.168.1.6','DEV6');
```

1 row created.

```
SQL> INSERT INTO REGISTRATION VALUES
(7,7,3,'APP007',SYSTIMESTAMP,1,1,'SUBMITTED','INELIGIBLE','EWS',700,'FAILED',0,'192.168.1.7','DEV7');
```

1 row created.

```
SQL> INSERT INTO REGISTRATION VALUES
(8,8,4,'APP008',SYSTIMESTAMP,1,1,'SUBMITTED','ELIGIBLE','GEN',1000,'PAID',1,'192.168.1.8','DEV8');

1 row created.

SQL> INSERT INTO REGISTRATION VALUES
(9,9,4,'APP009',SYSTIMESTAMP,2,0,'SUBMITTED','ELIGIBLE','SC',500,'PAID',1,'192.168.1.9','DEV9');

1 row created.

SQL> INSERT INTO REGISTRATION VALUES
(10,10,5,'APP010',SYSTIMESTAMP,1,1,'SUBMITTED','ELIGIBLE','OBC',800,'PAID',1,'192.168.1.10','DEV10');

1 row created.

SQL> INSERT INTO SUBJECT_SELECTION VALUES (1,1,1,0);

1 row created.

SQL> INSERT INTO SUBJECT_SELECTION VALUES (2,2,1,0);

1 row created.

SQL> INSERT INTO SUBJECT_SELECTION VALUES (3,3,1,0);

1 row created.

SQL> INSERT INTO SUBJECT_SELECTION VALUES (4,4,1,0);

1 row created.

SQL> INSERT INTO SUBJECT_SELECTION VALUES (5,5,1,1);

1 row created.
```

```
SQL> INSERT INTO SUBJECT_SELECTION VALUES (6,6,1,0);

1 row created.

SQL> INSERT INTO SUBJECT_SELECTION VALUES (7,7,1,0);

1 row created.

SQL> INSERT INTO SUBJECT_SELECTION VALUES (8,8,1,0);

1 row created.

SQL> INSERT INTO SUBJECT_SELECTION VALUES (9,9,1,0);

1 row created.

SQL> INSERT INTO SUBJECT_SELECTION VALUES (10,10,1,0);

1 row created.

SQL> INSERT INTO CENTER_PREFERENCE VALUES (1,1,1);

1 row created.

SQL> INSERT INTO CENTER_PREFERENCE VALUES (2,2,1);

1 row created.

SQL> INSERT INTO CENTER_PREFERENCE VALUES (3,3,1);

1 row created.

SQL> INSERT INTO CENTER_PREFERENCE VALUES (4,4,1);
```

1 row created.

SQL> INSERT INTO CENTER_PREFERENCE VALUES (5,5,1);

1 row created.

SQL> INSERT INTO CENTER_PREFERENCE VALUES (6,6,1);

1 row created.

SQL> INSERT INTO CENTER_PREFERENCE VALUES (7,7,1);

1 row created.

SQL> INSERT INTO CENTER_PREFERENCE VALUES (8,8,1);

1 row created.

SQL> INSERT INTO CENTER_PREFERENCE VALUES (9,9,1);

1 row created.

SQL> INSERT INTO CENTER_PREFERENCE VALUES (10,10,1);

1 row created.

SQL> INSERT INTO PAYMENT_TRANSACTION VALUES
(1,1,'RAZORPAY','CARD','TXN1',1000,SYSTIMESTAMP,'SUCCESS',NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO PAYMENT_TRANSACTION VALUES
(2,2,'RAZORPAY','UPI','TXN2',800,SYSTIMESTAMP,'SUCCESS',NULL,NULL,NULL);

1 row created.

```
SQL> INSERT INTO PAYMENT_TRANSACTION VALUES
(3,3,'PAYTM','UPI','TXN3',500,SYSTIMESTAMP,'FAILED','Insufficient Balance',NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO PAYMENT_TRANSACTION VALUES
(4,4,'RAZORPAY','CARD','TXN4',1000,SYSTIMESTAMP,'SUCCESS',NULL,NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO PAYMENT_TRANSACTION VALUES
(5,5,'PAYTM','UPI','TXN5',800,SYSTIMESTAMP,'SUCCESS',NULL,NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO PAYMENT_TRANSACTION VALUES
(6,6,'RAZORPAY','CARD','TXN6',500,SYSTIMESTAMP,'SUCCESS',NULL,NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO PAYMENT_TRANSACTION VALUES
(7,7,'PAYTM','UPI','TXN7',700,SYSTIMESTAMP,'FAILED','Timeout',NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO PAYMENT_TRANSACTION VALUES
(8,8,'RAZORPAY','CARD','TXN8',1000,SYSTIMESTAMP,'SUCCESS',NULL,NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO PAYMENT_TRANSACTION VALUES
(9,9,'RAZORPAY','UPI','TXN9',500,SYSTIMESTAMP,'SUCCESS',NULL,NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO PAYMENT_TRANSACTION VALUES
(10,10,'PAYTM','UPI','TXN10',800,SYSTIMESTAMP,'SUCCESS',NULL,NULL,NULL);
```

1 row created.

```
SQL> INSERT INTO BIOMETRIC_PROFILE VALUES
(1,NULL,'hash1','iris1','DEV1',SYSTIMESTAMP,SYSTIMESTAMP,'VERIFIED');
```

1 row created.

```
SQL> INSERT INTO BIOMETRIC_PROFILE VALUES
(2,NULL,'hash2','iris2','DEV2',SYSTIMESTAMP,SYSTIMESTAMP,'VERIFIED');
```

1 row created.

```
SQL> INSERT INTO BIOMETRIC_PROFILE VALUES
(3,NULL,'hash3','iris3','DEV3',SYSTIMESTAMP,SYSTIMESTAMP,'PENDING');
```

1 row created.

```
SQL> INSERT INTO BIOMETRIC_PROFILE VALUES
(4,NULL,'hash4','iris4','DEV4',SYSTIMESTAMP,SYSTIMESTAMP,'VERIFIED');
```

1 row created.

```
SQL> INSERT INTO BIOMETRIC_PROFILE VALUES
(5,NULL,'hash5','iris5','DEV5',SYSTIMESTAMP,SYSTIMESTAMP,'REJECTED');
```

1 row created.

```
SQL> INSERT INTO BIOMETRIC_PROFILE VALUES
(6,NULL,'hash6','iris6','DEV6',SYSTIMESTAMP,SYSTIMESTAMP,'VERIFIED');
```

1 row created.

```
SQL> INSERT INTO BIOMETRIC_PROFILE VALUES
(7,NULL,'hash7','iris7','DEV7',SYSTIMESTAMP,SYSTIMESTAMP,'PENDING');

1 row created.

SQL> INSERT INTO BIOMETRIC_PROFILE VALUES
(8,NULL,'hash8','iris8','DEV8',SYSTIMESTAMP,SYSTIMESTAMP,'VERIFIED');

1 row created.

SQL> INSERT INTO BIOMETRIC_PROFILE VALUES
(9,NULL,'hash9','iris9','DEV9',SYSTIMESTAMP,SYSTIMESTAMP,'VERIFIED');

1 row created.

SQL> INSERT INTO BIOMETRIC_PROFILE VALUES
(10,NULL,'hash10','iris10','DEV10',SYSTIMESTAMP,SYSTIMESTAMP,'VERIFIED');

1 row created.

SQL> INSERT INTO REGISTRATION_AUDIT_LOG VALUES (1,1,'CREATE',SYSTIMESTAMP,101,'IP1','Device1',NULL);

1 row created.

SQL> INSERT INTO REGISTRATION_AUDIT_LOG VALUES (2,2,'CREATE',SYSTIMESTAMP,102,'IP2','Device2',NULL);

1 row created.

SQL> INSERT INTO REGISTRATION_AUDIT_LOG VALUES (3,3,'PAYMENT_FAIL',SYSTIMESTAMP,103,'IP3','Device3','Retry
needed');

1 row created.

SQL> INSERT INTO REGISTRATION_AUDIT_LOG VALUES (4,4,'CREATE',SYSTIMESTAMP,104,'IP4','Device4',NULL);
```

1 row created.

SQL> INSERT INTO REGISTRATION_AUDIT_LOG VALUES (5,5,'UPDATE',SYSTIMESTAMP,105,'IP5','Device5',NULL);

1 row created.

SQL> INSERT INTO REGISTRATION_AUDIT_LOG VALUES (6,6,'CREATE',SYSTIMESTAMP,106,'IP6','Device6',NULL);

1 row created.

SQL> INSERT INTO REGISTRATION_AUDIT_LOG VALUES (7,7,'FAIL',SYSTIMESTAMP,107,'IP7','Device7','Eligibility issue');

1 row created.

SQL> INSERT INTO REGISTRATION_AUDIT_LOG VALUES (8,8,'CREATE',SYSTIMESTAMP,108,'IP8','Device8',NULL);

1 row created.

SQL> INSERT INTO REGISTRATION_AUDIT_LOG VALUES (9,9,'UPDATE',SYSTIMESTAMP,109,'IP9','Device9',NULL);

1 row created.

SQL> INSERT INTO REGISTRATION_AUDIT_LOG VALUES (10,10,'CREATE',SYSTIMESTAMP,110,'IP10','Device10',NULL);

1 row created.

SQL> INSERT INTO ATTEMPT_HISTORY VALUES (1,1,1,'PASS',250,100);

1 row created.

SQL> INSERT INTO ATTEMPT_HISTORY VALUES (2,1,1,'PASS',240,120);

1 row created.

```
SQL> INSERT INTO ATTEMPT_HISTORY VALUES (3,1,1,'FAIL',100,NULL);

1 row created.

SQL> INSERT INTO ATTEMPT_HISTORY VALUES (4,2,1,'PASS',300,80);

1 row created.

SQL> INSERT INTO ATTEMPT_HISTORY VALUES (5,2,2,'PASS',320,60);

1 row created.

SQL> INSERT INTO ATTEMPT_HISTORY VALUES (6,3,1,'PASS',150,90);

1 row created.

SQL> INSERT INTO ATTEMPT_HISTORY VALUES (7,3,1,'FAIL',80,NULL);

1 row created.

SQL> INSERT INTO ATTEMPT_HISTORY VALUES (8,4,1,'PASS',400,50);

1 row created.

SQL> INSERT INTO ATTEMPT_HISTORY VALUES (9,4,2,'PASS',420,30);

1 row created.

SQL> INSERT INTO ATTEMPT_HISTORY VALUES (10,5,1,'PASS',500,20);

1 row created.
```

## (ii) APPLIED SQL QUERIES AND REAL-WORLD USE CASES – STUDENT REGISTRARTION MODULE

1. **Get full profile of a student (basic + category + region)**
2. **Students who are PwD**
3. **Students with multiple addresses**
4. **Primary academic qualification of each student**
5. **Students with low academic performance (<50%)**
6. **Documents pending verification**
7. **Rejected documents with reasons**
8. **Total registrations per exam**
9. **Students with failed payments**
10. **Successful payments with transaction details**
11. **Students who selected optional subjects**
12. **Center preference list for a student**
13. **Students who attempted exam multiple times**
14. **Latest attempt result of each student**
15. **Students whose biometric verification failed**
16. **Audit log of a registration**
17. **Registrations where admit card not generated**
18. **Students eligible but not paid**
19. **Average fee paid per exam**
20. **Top scorers per exam**

```
SQL> -- Fetch complete student profile with category and domicile region

SQL> SELECT s.Student_ID,

  2       s.First_Name,

  3       s.Last_Name,

  4       c.Category_Name,

  5       r.Region_Name

  6  FROM STUDENT s

  7  JOIN CATEGORY c ON s.Category_ID = c.Category_ID

  8  JOIN REGION r ON s.Domicile_Region_ID = r.Region_ID

  9  WHERE s.Student_ID = 1;
```

```
STUDENT_ID
----------
FIRST_NAME
--------------------------------------------------------------------------------
LAST_NAME
--------------------------------------------------------------------------------
CATEGORY_NAME
--------------------------------------------------------------------------------
REGION_NAME
--------------------------------------------------------------------------------
         1
A
C


STUDENT_ID
----------
FIRST_NAME
--------------------------------------------------------------------------------
LAST_NAME
--------------------------------------------------------------------------------
CATEGORY_NAME
--------------------------------------------------------------------------------
REGION_NAME
--------------------------------------------------------------------------------
General
South



SQL> -- List all students marked as PwD
SQL> SELECT s.Student_ID,
  2         s.First_Name,
  3         s.Last_Name
  4  FROM STUDENT s
  5  WHERE s.PwD_Flag = 1;
```

```
STUDENT_ID
----------
FIRST_NAME
--------------------------------------------------------------------------------
LAST_NAME
--------------------------------------------------------------------------------
         4
A
C



SQL> -- Identify students having more than one address
SQL> SELECT sa.Student_ID,
  2       COUNT(sa.Address_ID) AS address_count
  3  FROM STUDENT_ADDRESS sa
  4  GROUP BY sa.Student_ID
  5  HAVING COUNT(sa.Address_ID) > 1;

no rows selected

SQL> -- Get primary qualification details
SQL> SELECT sap.Student_ID,
  2       sap.Qualification_Level,
  3       sap.Marks_Obtained,
  4       sap.Maximum_Marks
  5  FROM STUDENT_ACADEMIC_PROFILE sap
  6  WHERE sap.Is_Primary_Qualification = 1;

STUDENT_ID
----------
QUALIFICATION_LEVEL
--------------------------------------------------------------------------------
MARKS_OBTAINED MAXIMUM_MARKS
```

---------- ----------

1

12th

450 500

2

12th

440 500

STUDENT_ID

---

QUALIFICATION_LEVEL

---

MARKS_OBTAINED MAXIMUM_MARKS

---------- ----------

3

12th

430 500

4

12th

STUDENT_ID

---

QUALIFICATION_LEVEL

---

MARKS_OBTAINED MAXIMUM_MARKS

---------- ----------

420 500

5

12th

410 500

6

STUDENT_ID

---

QUALIFICATION_LEVEL

---

MARKS_OBTAINED MAXIMUM_MARKS

--- ---

12th

400 500

7

12th

390 500

STUDENT_ID

---

QUALIFICATION_LEVEL

---

MARKS_OBTAINED MAXIMUM_MARKS

--- ---

8

12th

380 500

9

12th

370 500

STUDENT_ID

---

QUALIFICATION_LEVEL

```
--------------------------------------------------------------------------------
MARKS_OBTAINED MAXIMUM_MARKS
-------------- -------------


      10
12th
        360         500



10 rows selected.


SQL> -- Find students scoring below 50%
SQL> SELECT sap.Student_ID,
  2       sap.Marks_Obtained,
  3       sap.Maximum_Marks
  4  FROM STUDENT_ACADEMIC_PROFILE sap
  5  WHERE (sap.Marks_Obtained / sap.Maximum_Marks) * 100 < 50;


no rows selected


SQL> -- List documents that are not yet verified
SQL> SELECT sd.Document_ID,
  2       sd.Student_ID,
  3       sd.Document_Type
  4  FROM STUDENT_DOCUMENT sd
  5  WHERE sd.Verification_Status = 'PENDING';


DOCUMENT_ID STUDENT_ID
----------- ----------
DOCUMENT_TYPE
--------------------------------------------------------------------------------
        3        3
AADHAAR
```

```
       7        7
AADHAAR


      10       10
AADHAAR



SQL> -- Fetch rejected documents and reasons
SQL> SELECT sd.Student_ID,
  2       sd.Document_Type,
  3       sd.Rejection_Reason
  4  FROM STUDENT_DOCUMENT sd
  5  WHERE sd.Verification_Status = 'REJECTED';

STUDENT_ID
----------
DOCUMENT_TYPE
--------------------------------------------------------------------------------
REJECTION_REASON
--------------------------------------------------------------------------------
       5
AADHAAR
Blurred



SQL> -- Count how many students registered for each exam
SQL> SELECT r.Exam_ID,
  2       COUNT(r.Registration_ID) AS total_registrations
  3  FROM REGISTRATION r
  4  GROUP BY r.Exam_ID;

   EXAM_ID TOTAL_REGISTRATIONS
---------- --------------------
       1                    3
```

```
      2            2
      3            2
      4            2
      5            1

SQL> -- Identify registrations with failed payments
SQL> SELECT r.Registration_ID,
  2       r.Student_ID,
  3       r.Payment_Status
  4  FROM REGISTRATION r
  5  WHERE r.Payment_Status = 'FAILED';

REGISTRATION_ID STUDENT_ID PAYMENT_STATUS
--------------- ---------- --------------------------------------------------
              3          3 FAILED
              7          7 FAILED

SQL> -- Fetch successful payment transactions
SQL> SELECT pt.Transaction_ID,
  2       pt.Registration_ID,
  3       pt.Amount_Paid
  4  FROM PAYMENT_TRANSACTION pt
  5  WHERE pt.Payment_Status = 'SUCCESS';

TRANSACTION_ID REGISTRATION_ID AMOUNT_PAID
-------------- --------------- -----------
             1               1        1000
             2               2         800
             4               4        1000
             5               5         800
             6               6         500
             8               8        1000
             9               9         500
            10              10         800
```

```
8 rows selected.

SQL> -- Find students opting for optional subjects
SQL> SELECT ss.Registration_ID,
  2       ss.Subject_ID
  3  FROM SUBJECT_SELECTION ss
  4  WHERE ss.Is_Optional = 1;

REGISTRATION_ID SUBJECT_ID
--------------- ----------
              5          5

SQL> -- Fetch preferred centers for a registration
SQL> SELECT cp.Registration_ID,
  2       cp.Center_ID,
  3       cp.Preference_Rank
  4  FROM CENTER_PREFERENCE cp
  5  WHERE cp.Registration_ID = 1
  6  ORDER BY cp.Preference_Rank;

REGISTRATION_ID  CENTER_ID PREFERENCE_RANK
--------------- ---------- ---------------
              1          1               1

SQL> -- Identify students with multiple attempts
SQL> SELECT ah.Student_ID,
  2       ah.Exam_ID,
  3       COUNT(*) AS attempt_count
  4  FROM ATTEMPT_HISTORY ah
  5  GROUP BY ah.Student_ID, ah.Exam_ID
  6  HAVING COUNT(*) > 1;

no rows selected
```

```
SQL> -- Get latest attempt result per student per exam
SQL> SELECT ah.Student_ID,
  2       ah.Exam_ID,
  3       ah.Attempt_Number,
  4       ah.Result_Status
  5  FROM ATTEMPT_HISTORY ah
  6  WHERE (ah.Student_ID, ah.Exam_ID, ah.Attempt_Number) IN (
  7    SELECT ah2.Student_ID,
  8           ah2.Exam_ID,
  9           MAX(ah2.Attempt_Number)
 10    FROM ATTEMPT_HISTORY ah2
 11    GROUP BY ah2.Student_ID, ah2.Exam_ID
 12  );

STUDENT_ID    EXAM_ID ATTEMPT_NUMBER
---------- ---------- --------------
RESULT_STATUS
--------------------------------------------------
       1          1              1
PASS

       2          1              1
PASS

       3          1              1
FAIL


STUDENT_ID    EXAM_ID ATTEMPT_NUMBER
---------- ---------- --------------
RESULT_STATUS
--------------------------------------------------
       4          2              1
```

PASS

5 2 2

PASS

6 3 1

PASS

STUDENT_ID EXAM_ID ATTEMPT_NUMBER

---------- ---------- ----------

RESULT_STATUS

---

7 3 1

FAIL

8 4 1

PASS

9 4 2

PASS

STUDENT_ID EXAM_ID ATTEMPT_NUMBER

---------- ---------- ----------

RESULT_STATUS

---

10 5 1

PASS

10 rows selected.

SQL> -- Identify students with failed biometric verification

```
SQL> SELECT bp.Student_ID,
  2       bp.Verification_Status
  3  FROM BIOMETRIC_PROFILE bp
  4  WHERE bp.Verification_Status = 'REJECTED';

STUDENT_ID VERIFICATION_STATUS
---------- --------------------------------------------------
         5 REJECTED

SQL> -- Track all actions performed on a registration
SQL> SELECT ral.Registration_ID,
  2       ral.Action_Type,
  3       ral.Action_Timestamp
  4  FROM REGISTRATION_AUDIT_LOG ral
  5  WHERE ral.Registration_ID = 1
  6  ORDER BY ral.Action_Timestamp;

REGISTRATION_ID
---------------
ACTION_TYPE
--------------------------------------------------------------------------------
ACTION_TIMESTAMP
---------------------------------------------------------------------------
             1
CREATE
14-APR-26 05.00.47.503000 PM


SQL> -- Find registrations pending admit card generation
SQL> SELECT r.Registration_ID,
  2       r.Student_ID
  3  FROM REGISTRATION r
  4  WHERE r.Admit_Card_Generated_Flag = 0;
```

```
REGISTRATION_ID STUDENT_ID
--------------- ----------
              3          3
              7          7

SQL> -- Identify eligible students who haven't completed payment
SQL> SELECT r.Student_ID,
  2         r.Eligibility_Status,
  3         r.Payment_Status
  4  FROM REGISTRATION r
  5  WHERE r.Eligibility_Status = 'ELIGIBLE'
  6    AND r.Payment_Status <> 'PAID';

STUDENT_ID ELIGIBILITY_STATUS
---------- ----------------------------------------------------
PAYMENT_STATUS
----------------------------------------------------
         3 ELIGIBLE
FAILED


SQL> -- Calculate average fee collected per exam
SQL> SELECT r.Exam_ID,
  2         AVG(r.Total_Fee) AS avg_fee
  3  FROM REGISTRATION r
  4  GROUP BY r.Exam_ID;

   EXAM_ID    AVG_FEE
---------- ----------
         1 766.666667
         2        900
         3        600
         4        750
         5        800
```

```
SQL> -- Get highest scorer for each exam
SQL> SELECT ah.Exam_ID,
  2       MAX(ah.Score) AS highest_score
  3  FROM ATTEMPT_HISTORY ah
  4  GROUP BY ah.Exam_ID;

   EXAM_ID HIGHEST_SCORE
---------- -------------
         1           250
         2           320
         3           150
         4           420
         5           500

SQL> spool off;
```

## IMPLEMENTATION OF MODULE 3 - EXAM SCHEDULING MODULE

### (i) DESIGN AND IMPLEMENTATION OF ENTITIES WITH DATA INITIALIZATION:

**This module focuses on the creation and management of exam scheduling related entities, including the definition of exam details, allocation of time slots and venues, and the insertion of essential data required to ensure accurate planning and organization of examinations within the system.**

```
SQL> CREATE TABLE TIME_SLOT (
  2     Time_Slot_ID NUMBER PRIMARY KEY,
  3     Slot_Name VARCHAR2(100) NOT NULL,
  4     Start_Time TIMESTAMP NOT NULL,
  5     End_Time TIMESTAMP NOT NULL,
  6     Is_Active NUMBER(1) DEFAULT 1,
  7     Max_Concurrent_Sessions NUMBER NOT NULL
  8  );
```

Table created.

```
SQL> CREATE TABLE EXAM_SCHEDULE (
  2   Schedule_ID NUMBER PRIMARY KEY,
  3   Subject_ID NUMBER NOT NULL,
  4   Exam_Date DATE NOT NULL,
  5   Time_Slot_ID NUMBER NOT NULL,
  6   Schedule_Type VARCHAR2(50) NOT NULL,
  7   Duration_Minutes NUMBER NOT NULL,
  8   Buffer_Time_Minutes NUMBER DEFAULT 0,
  9   Total_Registered_Count NUMBER DEFAULT 0,
 10   Total_Allocated_Count NUMBER DEFAULT 0,
 11   Created_At TIMESTAMP NOT NULL,
 12   Last_Modified_At TIMESTAMP NOT NULL,
 13   Approval_Status VARCHAR2(50) NOT NULL,
 14   FOREIGN KEY (Subject_ID) REFERENCES SUBJECT(Subject_ID),
 15   FOREIGN KEY (Time_Slot_ID) REFERENCES TIME_SLOT(Time_Slot_ID)
 16 );
```

Table created.

```
SQL> CREATE TABLE SEATING_PATTERN (
  2   Pattern_ID NUMBER PRIMARY KEY,
  3   Room_ID NUMBER NOT NULL,
  4   Pattern_Type VARCHAR2(50) NOT NULL,
  5   Total_Rows NUMBER NOT NULL,
  6   Total_Columns NUMBER NOT NULL,
  7   Seats_Per_Row NUMBER NOT NULL,
  8   Aisle_Position VARCHAR2(50),
  9   Seat_Numbering_Scheme VARCHAR2(50) NOT NULL,
 10   Capacity_Calculated NUMBER NOT NULL,
 11   Created_At TIMESTAMP NOT NULL,
 12   FOREIGN KEY (Room_ID) REFERENCES ROOM(Room_ID)
```

```
13 );
```

Table created.

```
SQL> CREATE TABLE ROOM_ALLOCATION (
 2   Allocation_ID NUMBER PRIMARY KEY,
 3   Schedule_ID NUMBER NOT NULL,
 4   Room_ID NUMBER NOT NULL,
 5   Seating_Pattern_ID NUMBER NOT NULL,
 6   Allocated_Capacity NUMBER NOT NULL,
 7   Blocked_Seats NUMBER DEFAULT 0,
 8   Emergency_Buffer_Seats NUMBER DEFAULT 0,
 9   Allocation_Status VARCHAR2(50) NOT NULL,
10   Allocation_Timestamp TIMESTAMP NOT NULL,
11   Allocated_By NUMBER NOT NULL,
12   FOREIGN KEY (Schedule_ID) REFERENCES EXAM_SCHEDULE(Schedule_ID),
13   FOREIGN KEY (Room_ID) REFERENCES ROOM(Room_ID),
14   FOREIGN KEY (Seating_Pattern_ID) REFERENCES SEATING_PATTERN(Pattern_ID)
15 );
```

Table created.

```
SQL> CREATE TABLE CENTER_DAY_STATUS (
 2   Center_ID NUMBER NOT NULL,
 3   Exam_Date DATE NOT NULL,
 4   Is_Holiday NUMBER(1) DEFAULT 0,
 5   Holiday_Reason VARCHAR2(200),
 6   Is_Maintenance_Day NUMBER(1) DEFAULT 0,
 7   Maintenance_Description CLOB,
 8   Approved_By NUMBER,
 9   Approved_At TIMESTAMP,
10   PRIMARY KEY (Center_ID, Exam_Date),
11   FOREIGN KEY (Center_ID) REFERENCES EXAM_CENTER(Center_ID)
12 );
```

Table created.

```
SQL> CREATE TABLE STUDENT_SEAT_ALLOCATION (
  2    Allocation_ID NUMBER PRIMARY KEY,
  3    Registration_ID NUMBER NOT NULL,
  4    Schedule_ID NUMBER NOT NULL,
  5    Room_ID NUMBER NOT NULL,
  6    Seat_Number VARCHAR2(20) NOT NULL,
  7    Row_Number NUMBER NOT NULL,
  8    Column_Number NUMBER NOT NULL,
  9    Allocation_Status VARCHAR2(50) NOT NULL,
 10    Allocation_Timestamp TIMESTAMP NOT NULL,
 11    FOREIGN KEY (Registration_ID) REFERENCES REGISTRATION(Registration_ID),
 12    FOREIGN KEY (Schedule_ID) REFERENCES EXAM_SCHEDULE(Schedule_ID),
 13    FOREIGN KEY (Room_ID) REFERENCES ROOM(Room_ID)
 14  );
```

Table created.

```
SQL> CREATE TABLE SCHEDULE_CONFLICT_LOG (
  2    Conflict_ID NUMBER PRIMARY KEY,
  3    Schedule_ID NUMBER NOT NULL,
  4    Conflict_Details CLOB NOT NULL,
  5    Detected_At TIMESTAMP NOT NULL,
  6    Resolved_Flag NUMBER(1) DEFAULT 0,
  7    Resolved_By NUMBER,
  8    Resolved_At TIMESTAMP,
  9    FOREIGN KEY (Schedule_ID) REFERENCES EXAM_SCHEDULE(Schedule_ID)
 10  );
```

Table created.

```
SQL> CREATE TABLE CENTER_CAPACITY_AGGREGATION (
```

```
  2    Center_ID NUMBER NOT NULL,
  3    Exam_Date DATE NOT NULL,
  4    Session_ID NUMBER NOT NULL,
  5    Total_Rooms NUMBER NOT NULL,
  6    Total_Capacity NUMBER NOT NULL,
  7    Allocated_Count NUMBER DEFAULT 0,
  8    Available_Seats NUMBER NOT NULL,
  9    Capacity_Utilization_Percentage NUMBER(5,2),
 10    Last_Updated TIMESTAMP NOT NULL,
 11    PRIMARY KEY (Center_ID, Exam_Date, Session_ID),
 12    FOREIGN KEY (Center_ID) REFERENCES EXAM_CENTER(Center_ID)
 13  );
```

Table created.

```
SQL> INSERT INTO TIME_SLOT VALUES (1,'Morning',TIMESTAMP '2026-05-01 09:00:00',TIMESTAMP '2026-05-01
12:00:00',1,5);
```

1 row created.

```
SQL> INSERT INTO TIME_SLOT VALUES (2,'Afternoon',TIMESTAMP '2026-05-01 14:00:00',TIMESTAMP '2026-05-01
17:00:00',1,5);
```

1 row created.

```
SQL> INSERT INTO TIME_SLOT VALUES (3,'Morning2',TIMESTAMP '2026-05-02 09:00:00',TIMESTAMP '2026-05-02
12:00:00',1,5);
```

1 row created.

```
SQL> INSERT INTO TIME_SLOT VALUES (4,'Afternoon2',TIMESTAMP '2026-05-02 14:00:00',TIMESTAMP '2026-05-02
17:00:00',1,5);
```

1 row created.

```
SQL> INSERT INTO TIME_SLOT VALUES (5,'Morning3',TIMESTAMP '2026-05-03 09:00:00',TIMESTAMP '2026-05-03
12:00:00',1,5);

1 row created.

SQL> INSERT INTO TIME_SLOT VALUES (6,'Afternoon3',TIMESTAMP '2026-05-03 14:00:00',TIMESTAMP '2026-05-03
17:00:00',1,5);

1 row created.

SQL> INSERT INTO TIME_SLOT VALUES (7,'Morning4',TIMESTAMP '2026-05-04 09:00:00',TIMESTAMP '2026-05-04
12:00:00',1,5);

1 row created.

SQL> INSERT INTO TIME_SLOT VALUES (8,'Afternoon4',TIMESTAMP '2026-05-04 14:00:00',TIMESTAMP '2026-05-04
17:00:00',1,5);

1 row created.

SQL> INSERT INTO TIME_SLOT VALUES (9,'Morning5',TIMESTAMP '2026-05-05 09:00:00',TIMESTAMP '2026-05-05
12:00:00',1,5);

1 row created.

SQL> INSERT INTO TIME_SLOT VALUES (10,'Afternoon5',TIMESTAMP '2026-05-05 14:00:00',TIMESTAMP '2026-05-05
17:00:00',1,5);

1 row created.

SQL> INSERT INTO EXAM_SCHEDULE VALUES (1,1,DATE '2026-05-
01',1,'REGULAR',180,10,100,80,SYSTIMESTAMP,SYSTIMESTAMP,'APPROVED');

1 row created.

SQL> INSERT INTO EXAM_SCHEDULE VALUES (2,2,DATE '2026-05-
01',2,'REGULAR',180,10,100,80,SYSTIMESTAMP,SYSTIMESTAMP,'APPROVED');
```

1 row created.

```
SQL> INSERT INTO EXAM_SCHEDULE VALUES (3,3,DATE '2026-05-
02',3,'REGULAR',180,10,100,80,SYSTIMESTAMP,SYSTIMESTAMP,'APPROVED');
```

1 row created.

```
SQL> INSERT INTO EXAM_SCHEDULE VALUES (4,4,DATE '2026-05-
02',4,'REGULAR',180,10,100,80,SYSTIMESTAMP,SYSTIMESTAMP,'APPROVED');
```

1 row created.

```
SQL> INSERT INTO EXAM_SCHEDULE VALUES (5,5,DATE '2026-05-
03',5,'REGULAR',180,10,100,80,SYSTIMESTAMP,SYSTIMESTAMP,'APPROVED');
```

1 row created.

```
SQL> INSERT INTO EXAM_SCHEDULE VALUES (6,6,DATE '2026-05-
03',6,'REGULAR',180,10,100,80,SYSTIMESTAMP,SYSTIMESTAMP,'APPROVED');
```

1 row created.

```
SQL> INSERT INTO EXAM_SCHEDULE VALUES (7,7,DATE '2026-05-
04',7,'REGULAR',180,10,100,80,SYSTIMESTAMP,SYSTIMESTAMP,'APPROVED');
```

1 row created.

```
SQL> INSERT INTO EXAM_SCHEDULE VALUES (8,8,DATE '2026-05-
04',8,'REGULAR',180,10,100,80,SYSTIMESTAMP,SYSTIMESTAMP,'APPROVED');
```

1 row created.

```
SQL> INSERT INTO EXAM_SCHEDULE VALUES (9,9,DATE '2026-05-
05',9,'REGULAR',180,10,100,80,SYSTIMESTAMP,SYSTIMESTAMP,'APPROVED');
```

1 row created.

SQL> INSERT INTO EXAM_SCHEDULE VALUES (10,10,DATE '2026-05-05',10,'REGULAR',180,10,100,80,SYSTIMESTAMP,SYSTIMESTAMP,'APPROVED');

1 row created.

SQL> INSERT INTO SEATING_PATTERN VALUES (1,1,'GRID',5,6,6,NULL,'NUM',30,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SEATING_PATTERN VALUES (2,2,'GRID',5,5,5,NULL,'NUM',25,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SEATING_PATTERN VALUES (3,3,'GRID',6,5,5,NULL,'NUM',30,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SEATING_PATTERN VALUES (4,4,'GRID',8,5,5,NULL,'NUM',40,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SEATING_PATTERN VALUES (5,5,'GRID',7,5,5,NULL,'NUM',35,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SEATING_PATTERN VALUES (6,6,'GRID',6,5,5,NULL,'NUM',30,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SEATING_PATTERN VALUES (7,7,'GRID',9,5,5,NULL,'NUM',45,SYSTIMESTAMP);

1 row created.

```
SQL> INSERT INTO SEATING_PATTERN VALUES (8,8,'GRID',6,5,5,NULL,'NUM',30,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SEATING_PATTERN VALUES (9,9,'GRID',5,5,5,NULL,'NUM',25,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SEATING_PATTERN VALUES (10,10,'GRID',4,5,5,NULL,'NUM',20,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ROOM_ALLOCATION VALUES (1,1,1,1,30,2,2,'ALLOCATED',SYSTIMESTAMP,101);

1 row created.

SQL> INSERT INTO ROOM_ALLOCATION VALUES (2,2,2,2,25,1,2,'ALLOCATED',SYSTIMESTAMP,102);

1 row created.

SQL> INSERT INTO ROOM_ALLOCATION VALUES (3,3,3,3,30,2,2,'ALLOCATED',SYSTIMESTAMP,103);

1 row created.

SQL> INSERT INTO ROOM_ALLOCATION VALUES (4,4,4,4,40,3,2,'ALLOCATED',SYSTIMESTAMP,104);

1 row created.

SQL> INSERT INTO ROOM_ALLOCATION VALUES (5,5,5,5,35,2,2,'ALLOCATED',SYSTIMESTAMP,105);

1 row created.

SQL> INSERT INTO ROOM_ALLOCATION VALUES (6,6,6,6,30,2,2,'ALLOCATED',SYSTIMESTAMP,106);
```

1 row created.

SQL> INSERT INTO ROOM_ALLOCATION VALUES (7,7,7,7,45,3,2,'ALLOCATED',SYSTIMESTAMP,107);

1 row created.

SQL> INSERT INTO ROOM_ALLOCATION VALUES (8,8,8,8,30,2,2,'ALLOCATED',SYSTIMESTAMP,108);

1 row created.

SQL> INSERT INTO ROOM_ALLOCATION VALUES (9,9,9,9,25,1,2,'ALLOCATED',SYSTIMESTAMP,109);

1 row created.

SQL> INSERT INTO ROOM_ALLOCATION VALUES (10,10,10,10,20,1,2,'ALLOCATED',SYSTIMESTAMP,110);

1 row created.

SQL> INSERT INTO CENTER_DAY_STATUS VALUES (1,DATE '2026-05-01',0,NULL,0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO CENTER_DAY_STATUS VALUES (2,DATE '2026-05-01',1,'Holiday',0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO CENTER_DAY_STATUS VALUES (3,DATE '2026-05-02',0,NULL,1,'Maintenance',101,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO CENTER_DAY_STATUS VALUES (4,DATE '2026-05-02',0,NULL,0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO CENTER_DAY_STATUS VALUES (5,DATE '2026-05-03',0,NULL,0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO CENTER_DAY_STATUS VALUES (6,DATE '2026-05-03',0,NULL,0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO CENTER_DAY_STATUS VALUES (7,DATE '2026-05-04',0,NULL,0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO CENTER_DAY_STATUS VALUES (8,DATE '2026-05-04',1,'Festival',0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO CENTER_DAY_STATUS VALUES (9,DATE '2026-05-05',0,NULL,0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO CENTER_DAY_STATUS VALUES (10,DATE '2026-05-05',0,NULL,0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO STUDENT_SEAT_ALLOCATION VALUES (1,1,1,1,'A1',1,1,'ALLOCATED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO STUDENT_SEAT_ALLOCATION VALUES (2,2,2,2,'A2',1,2,'ALLOCATED',SYSTIMESTAMP);

1 row created.

```
SQL> INSERT INTO STUDENT_SEAT_ALLOCATION VALUES (3,3,3,3,'A3',1,3,'ALLOCATED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO STUDENT_SEAT_ALLOCATION VALUES (4,4,4,4,'A4',1,4,'ALLOCATED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO STUDENT_SEAT_ALLOCATION VALUES (5,5,5,5,'A5',1,5,'ALLOCATED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO STUDENT_SEAT_ALLOCATION VALUES (6,6,6,6,'A6',1,6,'ALLOCATED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO STUDENT_SEAT_ALLOCATION VALUES (7,7,7,7,'A7',1,7,'ALLOCATED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO STUDENT_SEAT_ALLOCATION VALUES (8,8,8,8,'A8',1,8,'ALLOCATED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO STUDENT_SEAT_ALLOCATION VALUES (9,9,9,9,'A9',1,9,'ALLOCATED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO STUDENT_SEAT_ALLOCATION VALUES (10,10,10,10,'A10',1,10,'ALLOCATED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SCHEDULE_CONFLICT_LOG VALUES (1,1,'Overlap issue',SYSTIMESTAMP,0,NULL,NULL);
```

```
1 row created.

SQL> INSERT INTO SCHEDULE_CONFLICT_LOG VALUES (2,2,'Room conflict',SYSTIMESTAMP,1,101,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SCHEDULE_CONFLICT_LOG VALUES (3,3,'Timing clash',SYSTIMESTAMP,0,NULL,NULL);

1 row created.

SQL> INSERT INTO SCHEDULE_CONFLICT_LOG VALUES (4,4,'Holiday clash',SYSTIMESTAMP,1,102,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SCHEDULE_CONFLICT_LOG VALUES (5,5,'Resource issue',SYSTIMESTAMP,0,NULL,NULL);

1 row created.

SQL> INSERT INTO SCHEDULE_CONFLICT_LOG VALUES (6,6,'Room shortage',SYSTIMESTAMP,1,103,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SCHEDULE_CONFLICT_LOG VALUES (7,7,'Overlap',SYSTIMESTAMP,0,NULL,NULL);

1 row created.

SQL> INSERT INTO SCHEDULE_CONFLICT_LOG VALUES (8,8,'System error',SYSTIMESTAMP,1,104,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SCHEDULE_CONFLICT_LOG VALUES (9,9,'Capacity issue',SYSTIMESTAMP,0,NULL,NULL);

1 row created.
```

```
SQL> INSERT INTO SCHEDULE_CONFLICT_LOG VALUES (10,10,'Duplicate
schedule',SYSTIMESTAMP,1,105,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO CENTER_CAPACITY_AGGREGATION VALUES (1,DATE '2026-05-
01',1,10,300,250,50,83.33,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO CENTER_CAPACITY_AGGREGATION VALUES (2,DATE '2026-05-
01',1,10,250,200,50,80.00,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO CENTER_CAPACITY_AGGREGATION VALUES (3,DATE '2026-05-
02',1,10,300,260,40,86.67,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO CENTER_CAPACITY_AGGREGATION VALUES (4,DATE '2026-05-
02',1,10,400,350,50,87.50,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO CENTER_CAPACITY_AGGREGATION VALUES (5,DATE '2026-05-
03',1,10,350,300,50,85.71,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO CENTER_CAPACITY_AGGREGATION VALUES (6,DATE '2026-05-
03',1,10,300,250,50,83.33,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO CENTER_CAPACITY_AGGREGATION VALUES (7,DATE '2026-05-
04',1,10,450,400,50,88.89,SYSTIMESTAMP);
```

1 row created.

```
SQL> INSERT INTO CENTER_CAPACITY_AGGREGATION VALUES (8,DATE '2026-05-
04',1,10,300,260,40,86.67,SYSTIMESTAMP);
```

1 row created.

```
SQL> INSERT INTO CENTER_CAPACITY_AGGREGATION VALUES (9,DATE '2026-05-
05',1,10,250,200,50,80.00,SYSTIMESTAMP);
```

1 row created.

```
SQL> INSERT INTO CENTER_CAPACITY_AGGREGATION VALUES (10,DATE '2026-05-
05',1,10,200,150,50,75.00,SYSTIMESTAMP);
```

1 row created.

## APPLIED SQL QUERIES AND BUSINESS USE CASES – EXAM SCHEDULING MODULE

1. **Complete Exam Schedule with Subject and Time Slot Details**
2. **Schedules Pending Approval**
3. **Schedules Falling on Holidays**
4. **Schedules on Maintenance Days**
5. **Total Allocated Capacity per Schedule**
6. **Over Allocation Beyond Seating Capacity**
7. **Unallocated Rooms Identification**
8. **Student Count per Schedule Allocation**
9. **Duplicate Seat Assignment Detection**
10. **Unassigned Student Registrations**
11. **Schedule Conflict Detection for Time Slots**
12. **Unresolved Schedule Conflicts**
13. **Center Wise Capacity Utilization Analysis**
14. **Low Utilization Centers Identification**
15. **Overbooked Schedule Detection**
16. **Room Wise Allocation Status**
17. **Detailed Student Seating Arrangement**
18. **Schedules Without Room Allocation**
19. **Peak Load Time Slot Analysis**

## 20. Allocation Activity by Administrator

```
SQL> -- View complete exam schedule with subject and time slot
SQL> SELECT es.Schedule_ID,
  2       es.Exam_Date,
  3       es.Schedule_Type,
  4       s.Subject_Name,
  5       ts.Slot_Name,
  6       ts.Start_Time,
  7       ts.End_Time
  8  FROM EXAM_SCHEDULE es
  9  JOIN SUBJECT s ON es.Subject_ID = s.Subject_ID
 10  JOIN TIME_SLOT ts ON es.Time_Slot_ID = ts.Time_Slot_ID;

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE
----------- --------- --------------------------------------------------
SUBJECT_NAME
--------------------------------------------------------------------------------
SLOT_NAME
--------------------------------------------------------------------------------
START_TIME
-----------------------------------------------------------------------------
END_TIME
-----------------------------------------------------------------------------
         1 01-MAY-26 REGULAR
Math
Morning

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE
----------- --------- --------------------------------------------------
SUBJECT_NAME
--------------------------------------------------------------------------------
SLOT_NAME
```

---

START_TIME

---

END_TIME

---

01-MAY-26 09.00.00.000000 AM

01-MAY-26 12.00.00.000000 PM

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

--- --- ---

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

2 01-MAY-26 REGULAR

Physics

Afternoon

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

--- --- ---

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

01-MAY-26 02.00.00.000000 PM

01-MAY-26 05.00.00.000000 PM

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

3 02-MAY-26 REGULAR

Chemistry

Morning2

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

02-MAY-26 09.00.00.000000 AM

02-MAY-26 12.00.00.000000 PM

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

4 02-MAY-26 REGULAR

AdvMath

Afternoon2

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

02-MAY-26 02.00.00.000000 PM

02-MAY-26 05.00.00.000000 PM

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

5 03-MAY-26 REGULAR

Aptitude

Morning3

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

03-MAY-26 09.00.00.000000 AM

03-MAY-26 12.00.00.000000 PM

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

6 03-MAY-26 REGULAR

GS

Afternoon3

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

03-MAY-26 02.00.00.000000 PM

03-MAY-26 05.00.00.000000 PM

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

7 04-MAY-26 REGULAR

Biology

Morning4

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

04-MAY-26 09.00.00.000000 AM

04-MAY-26 12.00.00.000000 PM

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

--- --- ---

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

8 04-MAY-26 REGULAR

Legal

Afternoon4

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

--- --- ---

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

04-MAY-26 02.00.00.000000 PM

04-MAY-26 05.00.00.000000 PM

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

9 05-MAY-26 REGULAR

Quant

Morning5

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

---

START_TIME

---

END_TIME

---

05-MAY-26 09.00.00.000000 AM

05-MAY-26 12.00.00.000000 PM

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE

---------- --------- ----------

SUBJECT_NAME

---

SLOT_NAME

```
--------------------------------------------------------------------------------
START_TIME
-------------------------------------------------------------------------------
END_TIME
-------------------------------------------------------------------------------
        10 05-MAY-26 REGULAR
Reasoning
Afternoon5

SCHEDULE_ID EXAM_DATE SCHEDULE_TYPE
----------- --------- --------------------------------------------------
SUBJECT_NAME
--------------------------------------------------------------------------------
SLOT_NAME
--------------------------------------------------------------------------------
START_TIME
-------------------------------------------------------------------------------
END_TIME
-------------------------------------------------------------------------------
05-MAY-26 02.00.00.000000 PM
05-MAY-26 05.00.00.000000 PM



10 rows selected.


SQL> -- Identify schedules pending approval
SQL> SELECT es.Schedule_ID,
  2       es.Subject_ID,
  3       es.Exam_Date
  4  FROM EXAM_SCHEDULE es
  5  WHERE es.Approval_Status <> 'APPROVED';


no rows selected
```

```
SQL> -- Find schedules falling on holidays
SQL> SELECT es.Schedule_ID,
  2       es.Exam_Date,
  3       cds.Center_ID
  4  FROM EXAM_SCHEDULE es
  5  JOIN CENTER_DAY_STATUS cds
  6    ON es.Exam_Date = cds.Exam_Date
  7  WHERE cds.Is_Holiday = 1;

SCHEDULE_ID EXAM_DATE  CENTER_ID
----------- --------- ----------
          1 01-MAY-26          2
          2 01-MAY-26          2
          7 04-MAY-26          8
          8 04-MAY-26          8

SQL> -- Find schedules on maintenance days
SQL> SELECT es.Schedule_ID,
  2       es.Exam_Date,
  3       cds.Center_ID
  4  FROM EXAM_SCHEDULE es
  5  JOIN CENTER_DAY_STATUS cds
  6    ON es.Exam_Date = cds.Exam_Date
  7  WHERE cds.Is_Maintenance_Day = 1;

SCHEDULE_ID EXAM_DATE  CENTER_ID
----------- --------- ----------
          3 02-MAY-26          3
          4 02-MAY-26          3

SQL> -- Calculate allocated seating capacity for each schedule
SQL> SELECT ra.Schedule_ID,
  2       SUM(ra.Allocated_Capacity) AS total_capacity
  3  FROM ROOM_ALLOCATION ra
```

```
  4  GROUP BY ra.Schedule_ID;

SCHEDULE_ID TOTAL_CAPACITY
----------- --------------
          1           30
          2           25
          3           30
          4           40
          5           35
          6           30
          7           45
          8           30
          9           25
         10           20

10 rows selected.

SQL> -- Detect over-allocation beyond seating pattern capacity
SQL> SELECT ra.Allocation_ID,
  2       ra.Allocated_Capacity,
  3       sp.Capacity_Calculated
  4  FROM ROOM_ALLOCATION ra
  5  JOIN SEATING_PATTERN sp
  6    ON ra.Seating_Pattern_ID = sp.Pattern_ID
  7  WHERE ra.Allocated_Capacity > sp.Capacity_Calculated;

no rows selected

SQL> -- Identify unused rooms
SQL> SELECT r.Room_ID,
  2       r.Room_Code
  3  FROM ROOM r
  4  WHERE r.Room_ID NOT IN (
  5     SELECT ra.Room_ID FROM ROOM_ALLOCATION ra
```

```
  6 );

no rows selected

SQL> -- Count how many students allocated per schedule
SQL> SELECT ssa.Schedule_ID,
  2       COUNT(ssa.Allocation_ID) AS total_students
  3  FROM STUDENT_SEAT_ALLOCATION ssa
  4  GROUP BY ssa.Schedule_ID;

SCHEDULE_ID TOTAL_STUDENTS
----------- --------------
          1              1
          2              1
          3              1
          4              1
          5              1
          6              1
          7              1
          8              1
          9              1
         10              1

10 rows selected.

SQL> -- Detect duplicate seat numbers in same room and schedule
SQL> SELECT ssa.Schedule_ID,
  2       ssa.Room_ID,
  3       ssa.Seat_Number,
  4       COUNT(*) AS duplicate_count
  5  FROM STUDENT_SEAT_ALLOCATION ssa
  6  GROUP BY ssa.Schedule_ID, ssa.Room_ID, ssa.Seat_Number
  7  HAVING COUNT(*) > 1;
```

```
no rows selected

SQL> -- Identify registrations without seat allocation
SQL> SELECT r.Registration_ID
  2  FROM REGISTRATION r
  3  WHERE r.Registration_ID NOT IN (
  4    SELECT ssa.Registration_ID FROM STUDENT_SEAT_ALLOCATION ssa
  5  );

no rows selected

SQL> -- Find multiple exams scheduled in same time slot and date
SQL> SELECT es.Exam_Date,
  2       es.Time_Slot_ID,
  3       COUNT(*) AS conflict_count
  4  FROM EXAM_SCHEDULE es
  5  GROUP BY es.Exam_Date, es.Time_Slot_ID
  6  HAVING COUNT(*) > 1;

no rows selected

SQL> -- Fetch unresolved schedule conflicts
SQL> SELECT scl.Conflict_ID,
  2       scl.Schedule_ID,
  3       scl.Conflict_Details
  4  FROM SCHEDULE_CONFLICT_LOG scl
  5  WHERE scl.Resolved_Flag = 0;

CONFLICT_ID SCHEDULE_ID
----------- -----------
CONFLICT_DETAILS
--------------------------------------------------------------------------------
         1           1
Overlap issue
```

3 3

Timing clash

5 5

Resource issue

CONFLICT_ID SCHEDULE_ID

---------- ----------

CONFLICT_DETAILS

---

7 7

Overlap

9 9

Capacity issue

SQL> -- Analyze capacity usage for each center

SQL> SELECT cca.Center_ID,

2 cca.Total_Capacity,

3 cca.Allocated_Count,

4 cca.Capacity_Utilization_Percentage

5 FROM CENTER_CAPACITY_AGGREGATION cca;

| CENTER_ID | TOTAL_CAPACITY | ALLOCATED_COUNT | CAPACITY_UTILIZATION_PERCENTAGE |
|---|---|---|---|
| 1 | 300 | 250 | 83.33 |
| 2 | 250 | 200 | 80 |
| 3 | 300 | 260 | 86.67 |
| 4 | 400 | 350 | 87.5 |
| 5 | 350 | 300 | 85.71 |
| 6 | 300 | 250 | 83.33 |

```
         7        450        400                 88.89
         8        300        260                 86.67
         9        250        200                    80
        10        200        150                    75

10 rows selected.

SQL> -- Identify underutilized centers
SQL> SELECT cca.Center_ID,
  2       cca.Capacity_Utilization_Percentage
  3  FROM CENTER_CAPACITY_AGGREGATION cca
  4  WHERE cca.Capacity_Utilization_Percentage < 70;

no rows selected

SQL> -- Detect schedules where allocated exceeds registered
SQL> SELECT es.Schedule_ID,
  2       es.Total_Registered_Count,
  3       es.Total_Allocated_Count
  4  FROM EXAM_SCHEDULE es
  5  WHERE es.Total_Allocated_Count > es.Total_Registered_Count;

no rows selected

SQL> -- View allocation status per room
SQL> SELECT ra.Room_ID,
  2       ra.Schedule_ID,
  3       ra.Allocation_Status
  4  FROM ROOM_ALLOCATION ra;

   ROOM_ID SCHEDULE_ID ALLOCATION_STATUS
---------- ----------- --------------------------------------------------
         1           1 ALLOCATED
         2           2 ALLOCATED
```

```
       3         3 ALLOCATED
       4         4 ALLOCATED
       5         5 ALLOCATED
       6         6 ALLOCATED
       7         7 ALLOCATED
       8         8 ALLOCATED
       9         9 ALLOCATED
      10        10 ALLOCATED

10 rows selected.

SQL> -- Get seating details of students
SQL> SELECT ssa.Registration_ID,
  2       ssa.Schedule_ID,
  3       ssa.Room_ID,
  4       ssa.Seat_Number,
  5       ssa.Row_Number,
  6       ssa.Column_Number
  7  FROM STUDENT_SEAT_ALLOCATION ssa;

REGISTRATION_ID SCHEDULE_ID    ROOM_ID SEAT_NUMBER        ROW_NUMBER
--------------- ----------- ---------- -------------------- ----------
COLUMN_NUMBER
-------------
           1         1         1 A1                    1
          1

           2         2         2 A2                    1
          2

           3         3         3 A3                    1
          3
```

```
REGISTRATION_ID SCHEDULE_ID   ROOM_ID SEAT_NUMBER       ROW_NUMBER
--------------- ----------- ---------- -------------------- ----------
COLUMN_NUMBER
-------------
          4        4       4 A4                    1
         4


          5        5       5 A5                    1
         5


          6        6       6 A6                    1
         6


REGISTRATION_ID SCHEDULE_ID   ROOM_ID SEAT_NUMBER       ROW_NUMBER
--------------- ----------- ---------- -------------------- ----------
COLUMN_NUMBER
-------------
          7        7       7 A7                    1
         7


          8        8       8 A8                    1
         8


          9        9       9 A9                    1
         9


REGISTRATION_ID SCHEDULE_ID   ROOM_ID SEAT_NUMBER       ROW_NUMBER
--------------- ----------- ---------- -------------------- ----------
COLUMN_NUMBER
-------------
         10       10       10 A10                   1
        10
```

```
10 rows selected.

SQL> -- Identify schedules with no room allocation
SQL> SELECT es.Schedule_ID
  2  FROM EXAM_SCHEDULE es
  3  WHERE es.Schedule_ID NOT IN (
  4     SELECT ra.Schedule_ID FROM ROOM_ALLOCATION ra
  5  );

no rows selected

SQL> -- Find busiest time slots
SQL> SELECT es.Time_Slot_ID,
  2       COUNT(*) AS total_schedules
  3  FROM EXAM_SCHEDULE es
  4  GROUP BY es.Time_Slot_ID
  5  ORDER BY total_schedules DESC;

TIME_SLOT_ID TOTAL_SCHEDULES
------------ ---------------
           1               1
           2               1
           3               1
           4               1
          10               1
           6               1
           7               1
           8               1
           9               1
           5               1

10 rows selected.
```

```
SQL> -- Track how many allocations done by each admin

SQL> SELECT ra.Allocated_By,

  2       COUNT(*) AS allocations_done

  3  FROM ROOM_ALLOCATION ra

  4  GROUP BY ra.Allocated_By;


ALLOCATED_BY ALLOCATIONS_DONE

------------ ----------------

       101                1

       102                1

       103                1

       104                1

       105                1

       106                1

       107                1

       108                1

       109                1

       110                1


10 rows selected.


SQL> spool off;
```

## IMPLEMENTATION OF MODULE 4 – INVIGILATOR MANAGEMENT MODULE

### (i) DESIGN AND IMPLEMENTATION OF ENTITIES WITH DATA INITIALIZATION:

**This module focuses on the creation and management of invigilator related entities, including the registration of invigilator details, assignment of invigilation duties, and the insertion of essential data required to ensure efficient monitoring and supervision of examinations within the system.**

```
SQL> CREATE TABLE INVIGILATOR (

  2     Invigilator_ID NUMBER PRIMARY KEY,

  3     Employee_Code VARCHAR2(30) UNIQUE NOT NULL,
```

```
  4   First_Name VARCHAR2(100) NOT NULL,
  5   Middle_Name VARCHAR2(100),
  6   Last_Name VARCHAR2(100) NOT NULL,
  7   Gender CHAR(1) NOT NULL,
  8   Date_of_Birth DATE NOT NULL,
  9   Email VARCHAR2(150) UNIQUE NOT NULL,
 10   Phone_Number VARCHAR2(15) NOT NULL,
 11   Alternate_Phone VARCHAR2(15),
 12   Qualification_Level VARCHAR2(100) NOT NULL,
 13   Subject_Specialization VARCHAR2(150),
 14   Employment_Type VARCHAR2(50) NOT NULL,
 15   Years_of_Experience NUMBER NOT NULL,
 16   Center_ID NUMBER,
 17   Is_Active NUMBER(1) DEFAULT 1,
 18   Background_Verification_Status VARCHAR2(50) NOT NULL,
 19   Police_Clearance_Number VARCHAR2(100),
 20   Max_Sessions_Per_Day NUMBER NOT NULL,
 21   FOREIGN KEY (Center_ID) REFERENCES EXAM_CENTER(Center_ID)
 22 );

Table created.

SQL> CREATE TABLE INVIGILATOR_ASSIGNMENT (
  2   Assignment_ID NUMBER PRIMARY KEY,
  3   Schedule_ID NUMBER NOT NULL,
  4   Room_ID NUMBER NOT NULL,
  5   Invigilator_ID NUMBER NOT NULL,
  6   Role_Type VARCHAR2(50) NOT NULL,
  7   Reporting_Time TIMESTAMP NOT NULL,
  8   Duty_Start_Time TIMESTAMP NOT NULL,
  9   Duty_End_Time TIMESTAMP NOT NULL,
 10   Attendance_Status VARCHAR2(50) NOT NULL,
 11   Replacement_Flag NUMBER(1) DEFAULT 0,
 12   Replacement_For_Assignment_ID NUMBER,
```

```
13    Assignment_Status VARCHAR2(50) NOT NULL,
14    Assigned_By NUMBER NOT NULL,
15    Assigned_At TIMESTAMP NOT NULL,
16    Last_Modified_At TIMESTAMP NOT NULL,
17    FOREIGN KEY (Schedule_ID) REFERENCES EXAM_SCHEDULE(Schedule_ID),
18    FOREIGN KEY (Room_ID) REFERENCES ROOM(Room_ID),
19    FOREIGN KEY (Invigilator_ID) REFERENCES INVIGILATOR(Invigilator_ID)
20  );
```

Table created.

```
SQL> CREATE TABLE INVIGILATOR_WORKLOAD (
 2    Workload_ID NUMBER PRIMARY KEY,
 3    Invigilator_ID NUMBER NOT NULL,
 4    Exam_Cycle_ID NUMBER NOT NULL,
 5    Total_Assigned_Sessions NUMBER DEFAULT 0,
 6    Total_Attended_Sessions NUMBER DEFAULT 0,
 7    Total_Hours_Assigned NUMBER(6,2) DEFAULT 0,
 8    Total_Hours_Completed NUMBER(6,2) DEFAULT 0,
 9    Overtime_Hours NUMBER(6,2) DEFAULT 0,
10    Workload_Status VARCHAR2(50) NOT NULL,
11    Max_Limit NUMBER NOT NULL,
12    Last_Updated TIMESTAMP NOT NULL,
13    FOREIGN KEY (Invigilator_ID) REFERENCES INVIGILATOR(Invigilator_ID),
14    FOREIGN KEY (Exam_Cycle_ID) REFERENCES EXAM_CYCLE(Exam_Cycle_ID)
15  );
```

Table created.

```
SQL> CREATE TABLE INVIGILATOR_AVAILABILITY (
 2    Availability_ID NUMBER PRIMARY KEY,
 3    Invigilator_ID NUMBER NOT NULL,
 4    Available_Date DATE NOT NULL,
 5    Time_Slot_ID NUMBER NOT NULL,
```

```
  6  Availability_Status VARCHAR2(50) NOT NULL,
  7  Reason_For_Unavailability VARCHAR2(200),
  8  Declared_At TIMESTAMP NOT NULL,
  9  Approved_By NUMBER,
 10  Approval_Status VARCHAR2(50) NOT NULL,
 11  FOREIGN KEY (Invigilator_ID) REFERENCES INVIGILATOR(Invigilator_ID),
 12  FOREIGN KEY (Time_Slot_ID) REFERENCES TIME_SLOT(Time_Slot_ID)
 13 );

Table created.

SQL> CREATE TABLE INVIGILATOR_PAYMENT (
  2  Payment_ID NUMBER PRIMARY KEY,
  3  Invigilator_ID NUMBER NOT NULL,
  4  Exam_Cycle_ID NUMBER NOT NULL,
  5  Total_Sessions NUMBER NOT NULL,
  6  Base_Rate_Per_Session NUMBER(10,2) NOT NULL,
  7  Overtime_Rate NUMBER(10,2) DEFAULT 0,
  8  Overtime_Hours NUMBER(6,2) DEFAULT 0,
  9  Total_Amount NUMBER(10,2) NOT NULL,
 10  Tax_Deduction NUMBER(10,2) DEFAULT 0,
 11  Net_Payable NUMBER(10,2) NOT NULL,
 12  Payment_Status VARCHAR2(50) NOT NULL,
 13  Transaction_Reference VARCHAR2(100),
 14  Payment_Date TIMESTAMP,
 15  FOREIGN KEY (Invigilator_ID) REFERENCES INVIGILATOR(Invigilator_ID),
 16  FOREIGN KEY (Exam_Cycle_ID) REFERENCES EXAM_CYCLE(Exam_Cycle_ID)
 17 );

Table created.

SQL> CREATE TABLE INVIGILATOR_CENTER_TRANSFER (
  2  Transfer_ID NUMBER PRIMARY KEY,
  3  Invigilator_ID NUMBER NOT NULL,
```

```
  4   Old_Center_ID NUMBER NOT NULL,
  5   New_Center_ID NUMBER NOT NULL,
  6   Transfer_Date DATE NOT NULL,
  7   Reason CLOB NOT NULL,
  8   Approved_By NUMBER NOT NULL,
  9   Approval_Date DATE NOT NULL,
 10    FOREIGN KEY (Invigilator_ID) REFERENCES INVIGILATOR(Invigilator_ID),
 11    FOREIGN KEY (Old_Center_ID) REFERENCES EXAM_CENTER(Center_ID),
 12    FOREIGN KEY (New_Center_ID) REFERENCES EXAM_CENTER(Center_ID)
 13  );

Table created.

SQL> INSERT INTO INVIGILATOR VALUES (1,'EMP1','A',NULL,'B','M',DATE '1980-01-
01','i1@mail.com','9001',NULL,'PG','Math','FULL',10,1,1,'VERIFIED','PC1',3);

1 row created.

SQL> INSERT INTO INVIGILATOR VALUES (2,'EMP2','A',NULL,'B','F',DATE '1981-01-
01','i2@mail.com','9002',NULL,'PG','Sci','PART',8,2,1,'VERIFIED','PC2',3);

1 row created.

SQL> INSERT INTO INVIGILATOR VALUES (3,'EMP3','A',NULL,'B','M',DATE '1982-01-
01','i3@mail.com','9003',NULL,'UG','Eng','FULL',6,3,1,'VERIFIED','PC3',3);

1 row created.

SQL> INSERT INTO INVIGILATOR VALUES (4,'EMP4','A',NULL,'B','F',DATE '1983-01-
01','i4@mail.com','9004',NULL,'PG','Math','FULL',12,4,1,'VERIFIED','PC4',3);

1 row created.

SQL> INSERT INTO INVIGILATOR VALUES (5,'EMP5','A',NULL,'B','M',DATE '1984-01-
01','i5@mail.com','9005',NULL,'UG','Sci','PART',5,5,1,'VERIFIED','PC5',3);
```

1 row created.

SQL> INSERT INTO INVIGILATOR VALUES (6,'EMP6','A',NULL,'B','F',DATE '1985-01-01','i6@mail.com','9006',NULL,'PG','Math','FULL',9,6,1,'VERIFIED','PC6',3);

1 row created.

SQL> INSERT INTO INVIGILATOR VALUES (7,'EMP7','A',NULL,'B','M',DATE '1986-01-01','i7@mail.com','9007',NULL,'UG','Eng','PART',4,7,1,'VERIFIED','PC7',3);

1 row created.

SQL> INSERT INTO INVIGILATOR VALUES (8,'EMP8','A',NULL,'B','F',DATE '1987-01-01','i8@mail.com','9008',NULL,'PG','Sci','FULL',11,8,1,'VERIFIED','PC8',3);

1 row created.

SQL> INSERT INTO INVIGILATOR VALUES (9,'EMP9','A',NULL,'B','M',DATE '1988-01-01','i9@mail.com','9009',NULL,'UG','Math','PART',3,9,1,'VERIFIED','PC9',3);

1 row created.

SQL> INSERT INTO INVIGILATOR VALUES (10,'EMP10','A',NULL,'B','F',DATE '1989-01-01','i10@mail.com','9010',NULL,'PG','Sci','FULL',7,10,1,'VERIFIED','PC10',3);

1 row created.

SQL> INSERT INTO INVIGILATOR_ASSIGNMENT VALUES (1,1,1,1,'CHIEF',SYSTIMESTAMP,SYSTIMESTAMP,SYSTIMESTAMP,'PRESENT',0,NULL,'ACTIVE',101,SYSTIMESTAMP,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_ASSIGNMENT VALUES (2,2,2,2,'ASSIST',SYSTIMESTAMP,SYSTIMESTAMP,SYSTIMESTAMP,'PRESENT',0,NULL,'ACTIVE',102,SYSTIMESTAMP,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_ASSIGNMENT VALUES
(3,3,3,3,'CHIEF',SYSTIMESTAMP,SYSTIMESTAMP,SYSTIMESTAMP,'ABSENT',1,1,'ACTIVE',103,SYSTIMESTAMP,SYSTIMESTA
MP);

1 row created.

SQL> INSERT INTO INVIGILATOR_ASSIGNMENT VALUES
(4,4,4,4,'ASSIST',SYSTIMESTAMP,SYSTIMESTAMP,SYSTIMESTAMP,'PRESENT',0,NULL,'ACTIVE',104,SYSTIMESTAMP,SYSTI
MESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_ASSIGNMENT VALUES
(5,5,5,5,'CHIEF',SYSTIMESTAMP,SYSTIMESTAMP,SYSTIMESTAMP,'PRESENT',0,NULL,'ACTIVE',105,SYSTIMESTAMP,SYSTI
MESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_ASSIGNMENT VALUES
(6,6,6,6,'ASSIST',SYSTIMESTAMP,SYSTIMESTAMP,SYSTIMESTAMP,'PRESENT',0,NULL,'ACTIVE',106,SYSTIMESTAMP,SYSTI
MESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_ASSIGNMENT VALUES
(7,7,7,7,'CHIEF',SYSTIMESTAMP,SYSTIMESTAMP,SYSTIMESTAMP,'ABSENT',0,NULL,'ACTIVE',107,SYSTIMESTAMP,SYSTIM
ESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_ASSIGNMENT VALUES
(8,8,8,8,'ASSIST',SYSTIMESTAMP,SYSTIMESTAMP,SYSTIMESTAMP,'PRESENT',0,NULL,'ACTIVE',108,SYSTIMESTAMP,SYSTI
MESTAMP);

1 row created.

```
SQL> INSERT INTO INVIGILATOR_ASSIGNMENT VALUES
(9,9,9,9,'CHIEF',SYSTIMESTAMP,SYSTIMESTAMP,SYSTIMESTAMP,'PRESENT',0,NULL,'ACTIVE',109,SYSTIMESTAMP,SYSTI
MESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_ASSIGNMENT VALUES
(10,10,10,10,'ASSIST',SYSTIMESTAMP,SYSTIMESTAMP,SYSTIMESTAMP,'PRESENT',0,NULL,'ACTIVE',110,SYSTIMESTAMP,
SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_WORKLOAD VALUES (1,1,1,5,5,15,15,0,'NORMAL',10,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_WORKLOAD VALUES (2,2,1,4,4,12,12,0,'NORMAL',10,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_WORKLOAD VALUES (3,3,1,6,5,18,15,3,'OVERLOAD',10,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_WORKLOAD VALUES (4,4,1,3,3,9,9,0,'NORMAL',10,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_WORKLOAD VALUES (5,5,1,2,2,6,6,0,'NORMAL',10,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_WORKLOAD VALUES (6,6,1,5,5,15,15,0,'NORMAL',10,SYSTIMESTAMP);

1 row created.
```

```
SQL> INSERT INTO INVIGILATOR_WORKLOAD VALUES (7,7,1,4,3,12,9,3,'OVERLOAD',10,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_WORKLOAD VALUES (8,8,1,6,6,18,18,0,'NORMAL',10,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_WORKLOAD VALUES (9,9,1,3,3,9,9,0,'NORMAL',10,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_WORKLOAD VALUES (10,10,1,4,4,12,12,0,'NORMAL',10,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_AVAILABILITY VALUES (1,1,DATE '2026-05-
01',1,'AVAILABLE',NULL,SYSTIMESTAMP,NULL,'APPROVED');

1 row created.

SQL> INSERT INTO INVIGILATOR_AVAILABILITY VALUES (2,2,DATE '2026-05-
01',2,'AVAILABLE',NULL,SYSTIMESTAMP,NULL,'APPROVED');

1 row created.

SQL> INSERT INTO INVIGILATOR_AVAILABILITY VALUES (3,3,DATE '2026-05-
02',3,'UNAVAILABLE','Sick',SYSTIMESTAMP,NULL,'PENDING');

1 row created.

SQL> INSERT INTO INVIGILATOR_AVAILABILITY VALUES (4,4,DATE '2026-05-
02',4,'AVAILABLE',NULL,SYSTIMESTAMP,NULL,'APPROVED');
```

1 row created.

SQL> INSERT INTO INVIGILATOR_AVAILABILITY VALUES (5,5,DATE '2026-05-
03',5,'AVAILABLE',NULL,SYSTIMESTAMP,NULL,'APPROVED');

1 row created.

SQL> INSERT INTO INVIGILATOR_AVAILABILITY VALUES (6,6,DATE '2026-05-
03',6,'AVAILABLE',NULL,SYSTIMESTAMP,NULL,'APPROVED');

1 row created.

SQL> INSERT INTO INVIGILATOR_AVAILABILITY VALUES (7,7,DATE '2026-05-
04',7,'UNAVAILABLE','Leave',SYSTIMESTAMP,NULL,'PENDING');

1 row created.

SQL> INSERT INTO INVIGILATOR_AVAILABILITY VALUES (8,8,DATE '2026-05-
04',8,'AVAILABLE',NULL,SYSTIMESTAMP,NULL,'APPROVED');

1 row created.

SQL> INSERT INTO INVIGILATOR_AVAILABILITY VALUES (9,9,DATE '2026-05-
05',9,'AVAILABLE',NULL,SYSTIMESTAMP,NULL,'APPROVED');

1 row created.

SQL> INSERT INTO INVIGILATOR_AVAILABILITY VALUES (10,10,DATE '2026-05-
05',10,'AVAILABLE',NULL,SYSTIMESTAMP,NULL,'APPROVED');

1 row created.

SQL> INSERT INTO INVIGILATOR_PAYMENT VALUES (1,1,1,5,500,0,0,2500,100,2400,'PAID','TXN1',SYSTIMESTAMP);

1 row created.

```
SQL> INSERT INTO INVIGILATOR_PAYMENT VALUES (2,2,1,4,500,0,0,2000,80,1920,'PAID','TXN2',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_PAYMENT VALUES (3,3,1,6,500,100,3,3300,150,3150,'PENDING',NULL,NULL);

1 row created.

SQL> INSERT INTO INVIGILATOR_PAYMENT VALUES (4,4,1,3,500,0,0,1500,50,1450,'PAID','TXN4',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_PAYMENT VALUES (5,5,1,2,500,0,0,1000,30,970,'PAID','TXN5',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_PAYMENT VALUES (6,6,1,5,500,0,0,2500,100,2400,'PAID','TXN6',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_PAYMENT VALUES (7,7,1,4,500,100,3,2300,90,2210,'PENDING',NULL,NULL);

1 row created.

SQL> INSERT INTO INVIGILATOR_PAYMENT VALUES (8,8,1,6,500,0,0,3000,120,2880,'PAID','TXN8',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_PAYMENT VALUES (9,9,1,3,500,0,0,1500,50,1450,'PAID','TXN9',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO INVIGILATOR_PAYMENT VALUES (10,10,1,4,500,0,0,2000,80,1920,'PAID','TXN10',SYSTIMESTAMP);
```

1 row created.

SQL> INSERT INTO INVIGILATOR_CENTER_TRANSFER VALUES (1,1,1,2,DATE '2026-04-01','Shift',101,DATE '2026-04-02');

1 row created.

SQL> INSERT INTO INVIGILATOR_CENTER_TRANSFER VALUES (2,2,2,3,DATE '2026-04-01','Shift',102,DATE '2026-04-02');

1 row created.

SQL> INSERT INTO INVIGILATOR_CENTER_TRANSFER VALUES (3,3,3,4,DATE '2026-04-01','Shift',103,DATE '2026-04-02');

1 row created.

SQL> INSERT INTO INVIGILATOR_CENTER_TRANSFER VALUES (4,4,4,5,DATE '2026-04-01','Shift',104,DATE '2026-04-02');

1 row created.

SQL> INSERT INTO INVIGILATOR_CENTER_TRANSFER VALUES (5,5,5,6,DATE '2026-04-01','Shift',105,DATE '2026-04-02');

1 row created.

SQL> INSERT INTO INVIGILATOR_CENTER_TRANSFER VALUES (6,6,6,7,DATE '2026-04-01','Shift',106,DATE '2026-04-02');

1 row created.

SQL> INSERT INTO INVIGILATOR_CENTER_TRANSFER VALUES (7,7,7,8,DATE '2026-04-01','Shift',107,DATE '2026-04-02');

1 row created.

```
SQL> INSERT INTO INVIGILATOR_CENTER_TRANSFER VALUES (8,8,8,9,DATE '2026-04-01','Shift',108,DATE '2026-04-02');

1 row created.

SQL> INSERT INTO INVIGILATOR_CENTER_TRANSFER VALUES (9,9,9,10,DATE '2026-04-01','Shift',109,DATE '2026-04-02');

1 row created.

SQL> INSERT INTO INVIGILATOR_CENTER_TRANSFER VALUES (10,10,10,1,DATE '2026-04-01','Shift',110,DATE '2026-04-02');

1 row created.
```

## APPLIED SQL QUESRIES AND BUSINESS USE CASES – INVIGILATOR MANAGEMENT MODULE

1. **Complete Invigilator Profile with Center Details**
2. **Inactive Invigilators Identification**
3. **Invigilators Without Background Verification**
4. **Invigilator Assignments Per Day**
5. **Invigilators Exceeding Maximum Sessions Per Day**
6. **Absent Invigilators Tracking**
7. **Replacement Assignment Identification**
8. **Invigilator Availability for Specific Date and Time Slot**
9. **Unavailable Invigilators with Reasons**
10. **Workload Summary Per Invigilator**
11. **Invigilator Overload Detection**
12. **Pending Payment Identification for Invigilators**
13. **Total Payout Per Exam Cycle**
14. **Highest Earning Invigilators**

15. **Suspicious Overtime Detection**
16. **Invigilator Assignment Schedule Overview**
17. **Invigilator Center Transfer History**
18. **Frequent Transfer Detection**
19. **Assignment Without Availability Declaration**
20. **Double Booking Detection for Invigilators**

```
SQL> -- View invigilator details along with assigned center
SQL> SELECT i.Invigilator_ID,
  2       i.First_Name,
  3       i.Last_Name,
  4       i.Email,
  5       i.Qualification_Level,
  6       i.Years_of_Experience,
  7       ec.Center_Name,
  8       ec.City
  9  FROM INVIGILATOR i
 10  LEFT JOIN EXAM_CENTER ec
 11    ON i.Center_ID = ec.Center_ID;

INVIGILATOR_ID
--------------
FIRST_NAME
--------------------------------------------------------------------------------
LAST_NAME
--------------------------------------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
QUALIFICATION_LEVEL
--------------------------------------------------------------------------------
YEARS_OF_EXPERIENCE
-------------------
CENTER_NAME
```

---

CITY

---

1

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

A

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

B

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

i1@mail.com

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

PG

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---



INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Center1

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Chennai

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

2

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

A

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

B

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

i2@mail.com

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

PG

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

8

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Center2

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Delhi

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

3

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

A

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

B

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

i3@mail.com

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

UG

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

6

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Center3

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Mumbai

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

4

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

A

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

B

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

i4@mail.com

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

PG

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

12

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Center4

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Kolkata

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

5

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

A

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

B

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

i5@mail.com

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

UG

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---



INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Center5

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Bhopal

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---



INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

A

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

B

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

i6@mail.com

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

PG

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

9

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Center6

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Bangalore

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---



INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

A

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

B

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

i7@mail.com

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

UG

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

4

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Center7

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Chandigarh

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

8

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

A

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

B

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

i8@mail.com

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

PG

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

11

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Center8

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Ahmedabad

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

9

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

A

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

B

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

i9@mail.com

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

UG

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---



INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Center9

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Patna

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

A

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

B

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

i10@mail.com

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

PG

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

7

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Center10

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

---

YEARS_OF_EXPERIENCE

---

CENTER_NAME

---

CITY

---

Guwahati

INVIGILATOR_ID

---

FIRST_NAME

---

LAST_NAME

---

EMAIL

---

QUALIFICATION_LEVEL

```
--------------------------------------------------------------------------------
YEARS_OF_EXPERIENCE
-------------------
CENTER_NAME
--------------------------------------------------------------------------------
CITY
--------------------------------------------------------------------------------



10 rows selected.


SQL> -- Identify invigilators who are not active
SQL> SELECT i.Invigilator_ID,
  2       i.First_Name,
  3       i.Last_Name
  4  FROM INVIGILATOR i
  5  WHERE i.Is_Active = 0;

no rows selected


SQL> -- Security check: unverified invigilators
SQL> SELECT i.Invigilator_ID,
  2       i.First_Name,
  3       i.Background_Verification_Status
  4  FROM INVIGILATOR i
  5  WHERE i.Background_Verification_Status <> 'VERIFIED';

no rows selected


SQL> -- Count assignments handled by each invigilator
SQL> SELECT ia.Invigilator_ID,
  2       COUNT(ia.Assignment_ID) AS total_assignments
  3  FROM INVIGILATOR_ASSIGNMENT ia
  4  GROUP BY ia.Invigilator_ID;
```

```
INVIGILATOR_ID TOTAL_ASSIGNMENTS
-------------- -----------------
             1                 1
             2                 1
             3                 1
             4                 1
             5                 1
             6                 1
             7                 1
             8                 1
             9                 1
            10                 1

10 rows selected.

SQL> -- Overworked invigilators (policy violation)
SQL> SELECT i.Invigilator_ID,
  2       COUNT(ia.Assignment_ID) AS assigned_sessions,
  3       i.Max_Sessions_Per_Day
  4  FROM INVIGILATOR i
  5  JOIN INVIGILATOR_ASSIGNMENT ia
  6    ON i.Invigilator_ID = ia.Invigilator_ID
  7  GROUP BY i.Invigilator_ID, i.Max_Sessions_Per_Day
  8  HAVING COUNT(ia.Assignment_ID) > i.Max_Sessions_Per_Day;

no rows selected

SQL> -- Track absentees
SQL> SELECT ia.Assignment_ID,
  2       ia.Invigilator_ID,
  3       ia.Attendance_Status
  4  FROM INVIGILATOR_ASSIGNMENT ia
  5  WHERE ia.Attendance_Status = 'ABSENT';
```

```
ASSIGNMENT_ID INVIGILATOR_ID ATTENDANCE_STATUS
------------- -------------- ------------------------------------------------
            3            3 ABSENT
            7            7 ABSENT

SQL> -- Identify assignments where replacement happened
SQL> SELECT ia.Assignment_ID,
  2       ia.Invigilator_ID,
  3       ia.Replacement_For_Assignment_ID
  4  FROM INVIGILATOR_ASSIGNMENT ia
  5  WHERE ia.Replacement_Flag = 1;

ASSIGNMENT_ID INVIGILATOR_ID REPLACEMENT_FOR_ASSIGNMENT_ID
------------- -------------- ------------------------------
            3            3                              1

SQL> -- Find available invigilators for scheduling
SQL> SELECT ia.Invigilator_ID,
  2       i.First_Name,
  3       i.Last_Name
  4  FROM INVIGILATOR_AVAILABILITY ia
  5  JOIN INVIGILATOR i
  6    ON ia.Invigilator_ID = i.Invigilator_ID
  7  WHERE ia.Available_Date = DATE '2026-05-01'
  8    AND ia.Time_Slot_ID = 1
  9    AND ia.Availability_Status = 'AVAILABLE';

INVIGILATOR_ID
--------------
FIRST_NAME
--------------------------------------------------------------------------------
LAST_NAME
--------------------------------------------------------------------------------
```

```
         1
A
B


SQL> -- Identify unavailable invigilators and reasons
SQL> SELECT ia.Invigilator_ID,
  2       ia.Reason_For_Unavailability
  3  FROM INVIGILATOR_AVAILABILITY ia
  4  WHERE ia.Availability_Status = 'UNAVAILABLE';

INVIGILATOR_ID
--------------
REASON_FOR_UNAVAILABILITY
--------------------------------------------------------------------------------
         3
Sick


         7
Leave


SQL> -- Analyze workload performance
SQL> SELECT iw.Invigilator_ID,
  2       iw.Total_Assigned_Sessions,
  3       iw.Total_Attended_Sessions,
  4       iw.Overtime_Hours,
  5       iw.Workload_Status
  6  FROM INVIGILATOR_WORKLOAD iw;

INVIGILATOR_ID TOTAL_ASSIGNED_SESSIONS TOTAL_ATTENDED_SESSIONS OVERTIME_HOURS
-------------- ------------------------ ------------------------ --------------
WORKLOAD_STATUS
--------------------------------------------------
```

1 5 5 0

NORMAL

2 4 4 0

NORMAL

3 6 5 3

OVERLOAD

INVIGILATOR_ID TOTAL_ASSIGNED_SESSIONS TOTAL_ATTENDED_SESSIONS OVERTIME_HOURS

--- --- --- ---

WORKLOAD_STATUS

---

4 3 3 0

NORMAL

5 2 2 0

NORMAL

6 5 5 0

NORMAL

INVIGILATOR_ID TOTAL_ASSIGNED_SESSIONS TOTAL_ATTENDED_SESSIONS OVERTIME_HOURS

--- --- --- ---

WORKLOAD_STATUS

---

7 4 3 3

OVERLOAD

8 6 6 0

NORMAL

```
         9                 3                 3          0
NORMAL



INVIGILATOR_ID TOTAL_ASSIGNED_SESSIONS TOTAL_ATTENDED_SESSIONS OVERTIME_HOURS
-------------- ----------------------- ----------------------- ---------------
WORKLOAD_STATUS
--------------------------------------------------
         10                4                 4          0
NORMAL



10 rows selected.


SQL> -- Find invigilators with overload status
SQL> SELECT iw.Invigilator_ID,
  2       iw.Overtime_Hours,
  3       iw.Workload_Status
  4  FROM INVIGILATOR_WORKLOAD iw
  5  WHERE iw.Workload_Status = 'OVERLOAD';

INVIGILATOR_ID OVERTIME_HOURS WORKLOAD_STATUS
-------------- -------------- --------------------------------------------------
         3           3 OVERLOAD
         7           3 OVERLOAD


SQL> -- Identify unpaid invigilators
SQL> SELECT ip.Invigilator_ID,
  2       ip.Total_Amount,
  3       ip.Payment_Status
  4  FROM INVIGILATOR_PAYMENT ip
  5  WHERE ip.Payment_Status <> 'PAID';

INVIGILATOR_ID TOTAL_AMOUNT PAYMENT_STATUS
```

```
-------------- ------------ ----------------------------------------------------
         3       3300 PENDING
         7       2300 PENDING

SQL> -- Calculate total payout for exam cycle
SQL> SELECT ip.Exam_Cycle_ID,
  2       SUM(ip.Net_Payable) AS total_payout
  3  FROM INVIGILATOR_PAYMENT ip
  4  GROUP BY ip.Exam_Cycle_ID;

EXAM_CYCLE_ID TOTAL_PAYOUT
------------- ------------
         1       20750

SQL> -- Top paid invigilators
SQL> SELECT ip.Invigilator_ID,
  2       SUM(ip.Net_Payable) AS total_earned
  3  FROM INVIGILATOR_PAYMENT ip
  4  GROUP BY ip.Invigilator_ID
  5  ORDER BY total_earned DESC;

INVIGILATOR_ID TOTAL_EARNED
-------------- ------------
         3       3150
         8       2880
         6       2400
         1       2400
         7       2210
         2       1920
        10       1920
         4       1450
         9       1450
         5        970
```

```
10 rows selected.

SQL> -- Flag abnormal overtime (possible misuse)
SQL> SELECT iw.Invigilator_ID,
  2       iw.Overtime_Hours
  3  FROM INVIGILATOR_WORKLOAD iw
  4  WHERE iw.Overtime_Hours > 5;

no rows selected

SQL> -- Complete duty schedule
SQL> SELECT ia.Assignment_ID,
  2       ia.Invigilator_ID,
  3       ia.Schedule_ID,
  4       ia.Room_ID,
  5       ia.Role_Type,
  6       ia.Duty_Start_Time,
  7       ia.Duty_End_Time
  8  FROM INVIGILATOR_ASSIGNMENT ia;

ASSIGNMENT_ID INVIGILATOR_ID SCHEDULE_ID    ROOM_ID
------------- -------------- ----------- ----------
ROLE_TYPE
--------------------------------------------------
DUTY_START_TIME
---------------------------------------------------------------------------
DUTY_END_TIME
---------------------------------------------------------------------------
         1           1         1         1
CHIEF
14-APR-26 05.26.42.985000 PM
14-APR-26 05.26.42.985000 PM
```

ASSIGNMENT_ID INVIGILATOR_ID SCHEDULE_ID ROOM_ID

---------- ---------- ---------- ----------

ROLE_TYPE

---

DUTY_START_TIME

---

DUTY_END_TIME

---

2 2 2 2

ASSIST

14-APR-26 05.26.43.004000 PM

14-APR-26 05.26.43.004000 PM

ASSIGNMENT_ID INVIGILATOR_ID SCHEDULE_ID ROOM_ID

---------- ---------- ---------- ----------

ROLE_TYPE

---

DUTY_START_TIME

---

DUTY_END_TIME

---

3 3 3 3

CHIEF

14-APR-26 05.26.43.007000 PM

14-APR-26 05.26.43.007000 PM

ASSIGNMENT_ID INVIGILATOR_ID SCHEDULE_ID ROOM_ID

---------- ---------- ---------- ----------

ROLE_TYPE

---

DUTY_START_TIME

---

DUTY_END_TIME

---

4 4 4 4

ASSIST

14-APR-26 05.26.43.011000 PM

14-APR-26 05.26.43.011000 PM

ASSIGNMENT_ID INVIGILATOR_ID SCHEDULE_ID ROOM_ID

---

ROLE_TYPE

---

DUTY_START_TIME

---

DUTY_END_TIME

---

5 5 5 5

CHIEF

14-APR-26 05.26.43.016000 PM

14-APR-26 05.26.43.016000 PM

ASSIGNMENT_ID INVIGILATOR_ID SCHEDULE_ID ROOM_ID

---

ROLE_TYPE

---

DUTY_START_TIME

---

DUTY_END_TIME

---

6 6 6 6

ASSIST

14-APR-26 05.26.43.020000 PM

14-APR-26 05.26.43.020000 PM

ASSIGNMENT_ID INVIGILATOR_ID SCHEDULE_ID ROOM_ID

---------- ---------- ---------- ----------

ROLE_TYPE

---

DUTY_START_TIME

---

DUTY_END_TIME

---

7 7 7 7

CHIEF

14-APR-26 05.26.43.024000 PM

14-APR-26 05.26.43.024000 PM

ASSIGNMENT_ID INVIGILATOR_ID SCHEDULE_ID ROOM_ID

---------- ---------- ---------- ----------

ROLE_TYPE

---

DUTY_START_TIME

---

DUTY_END_TIME

---

8 8 8 8

ASSIST

14-APR-26 05.26.43.028000 PM

14-APR-26 05.26.43.028000 PM

ASSIGNMENT_ID INVIGILATOR_ID SCHEDULE_ID ROOM_ID

---------- ---------- ---------- ----------

ROLE_TYPE

---

```
DUTY_START_TIME
---------------------------------------------------------------------------
DUTY_END_TIME
---------------------------------------------------------------------------
         9          9        9        9
CHIEF
14-APR-26 05.26.43.032000 PM
14-APR-26 05.26.43.032000 PM


ASSIGNMENT_ID INVIGILATOR_ID SCHEDULE_ID    ROOM_ID
------------- -------------- ------------ -----------
ROLE_TYPE
--------------------------------------------------
DUTY_START_TIME
---------------------------------------------------------------------------
DUTY_END_TIME
---------------------------------------------------------------------------
        10         10        10        10
ASSIST
14-APR-26 05.26.43.470000 PM
14-APR-26 05.26.43.470000 PM


10 rows selected.


SQL> -- Track invigilator transfers
SQL> SELECT ict.Invigilator_ID,
  2       ict.Old_Center_ID,
  3       ict.New_Center_ID,
  4       ict.Transfer_Date
  5  FROM INVIGILATOR_CENTER_TRANSFER ict;


INVIGILATOR_ID OLD_CENTER_ID NEW_CENTER_ID TRANSFER_
```

```
-------------- -------------- -------------- ---------
         1          1          2 01-APR-26
         2          2          3 01-APR-26
         3          3          4 01-APR-26
         4          4          5 01-APR-26
         5          5          6 01-APR-26
         6          6          7 01-APR-26
         7          7          8 01-APR-26
         8          8          9 01-APR-26
         9          9         10 01-APR-26
        10         10          1 01-APR-26

10 rows selected.

SQL> -- Detect invigilators frequently transferred
SQL> SELECT ict.Invigilator_ID,
  2       COUNT(*) AS transfer_count
  3  FROM INVIGILATOR_CENTER_TRANSFER ict
  4  GROUP BY ict.Invigilator_ID
  5  HAVING COUNT(*) > 1;

no rows selected

SQL> -- Detect scheduling without availability approval
SQL> SELECT ia.Invigilator_ID
  2  FROM INVIGILATOR_ASSIGNMENT ia
  3  WHERE ia.Invigilator_ID NOT IN (
  4     SELECT av.Invigilator_ID FROM INVIGILATOR_AVAILABILITY av
  5  );

no rows selected

SQL> -- Same invigilator assigned to multiple rooms same time
SQL> SELECT ia.Invigilator_ID,
```

```
  2     ia.Schedule_ID,
  3     COUNT(*) AS assignments
  4 FROM INVIGILATOR_ASSIGNMENT ia
  5 GROUP BY ia.Invigilator_ID, ia.Schedule_ID
  6 HAVING COUNT(*) > 1;

no rows selected

SQL> spool off;
```

## IMPLEMENTATION OF MODULE 5 – STUDENT DISTRIBUTION AND ATTENDANCE MODULE

### (i) DESIGN AND IMPLEMENTATION OF ENTITIES WITH DATA INITIALIZATION:

**This module focuses on the creation and management of entities related to student distribution and attendance, including the allocation of students to examination rooms, tracking of attendance status, and the insertion of essential data required to ensure accurate monitoring and efficient management of student participation during examinations within the system.**

```
SQL> CREATE TABLE SEAT_ALLOCATION (
  2   Allocation_ID NUMBER PRIMARY KEY,
  3   Schedule_ID NUMBER NOT NULL,
  4   Registration_ID NUMBER NOT NULL,
  5   Room_ID NUMBER NOT NULL,
  6   Seat_Number VARCHAR2(20) NOT NULL,
  7   Row_Number NUMBER NOT NULL,
  8   Column_Number NUMBER NOT NULL,
  9   Seat_Label VARCHAR2(20),
 10   Allocation_Method VARCHAR2(50) NOT NULL,
 11   Is_Buffer_Seat NUMBER(1) DEFAULT 0,
 12   Allocation_Timestamp TIMESTAMP NOT NULL,
 13   Allocation_Status VARCHAR2(50) NOT NULL,
 14   Reassigned_From_Seat VARCHAR2(20),
 15   Remarks CLOB,
```

```
16    FOREIGN KEY (Schedule_ID) REFERENCES EXAM_SCHEDULE(Schedule_ID),
17    FOREIGN KEY (Registration_ID) REFERENCES REGISTRATION(Registration_ID),
18    FOREIGN KEY (Room_ID) REFERENCES ROOM(Room_ID)
19  );
```

Table created.

```
SQL> CREATE TABLE ATTENDANCE (
 2    Attendance_ID NUMBER PRIMARY KEY,
 3    Schedule_ID NUMBER NOT NULL,
 4    Registration_ID NUMBER NOT NULL,
 5    Room_ID NUMBER NOT NULL,
 6    Attendance_Status VARCHAR2(50) NOT NULL,
 7    Checking_Time TIMESTAMP,
 8    Checking_Mode VARCHAR2(50) NOT NULL,
 9    Biometric_Verification_Status VARCHAR2(50) NOT NULL,
10    Verified_By_Invigilator_ID NUMBER,
11    Attendance_Device_ID VARCHAR2(100),
12    Attendance_Timestamp TIMESTAMP NOT NULL,
13    Remarks CLOB,
14    FOREIGN KEY (Schedule_ID) REFERENCES EXAM_SCHEDULE(Schedule_ID),
15    FOREIGN KEY (Registration_ID) REFERENCES REGISTRATION(Registration_ID),
16    FOREIGN KEY (Room_ID) REFERENCES ROOM(Room_ID),
17    FOREIGN KEY (Verified_By_Invigilator_ID) REFERENCES INVIGILATOR(Invigilator_ID)
18  );
```

Table created.

```
SQL> CREATE TABLE ROOM_ATTENDANCE_SUMMARY (
 2    Room_ID NUMBER NOT NULL,
 3    Schedule_ID NUMBER NOT NULL,
 4    Total_Allocated NUMBER DEFAULT 0,
 5    Total_Present NUMBER DEFAULT 0,
 6    Total_Late NUMBER DEFAULT 0,
```

```
  7     Total_Malpractice_Cases NUMBER DEFAULT 0,
  8     Total_Absent NUMBER DEFAULT 0,
  9     Last_Updated TIMESTAMP NOT NULL,
 10      PRIMARY KEY (Room_ID, Schedule_ID),
 11      FOREIGN KEY (Room_ID) REFERENCES ROOM(Room_ID),
 12      FOREIGN KEY (Schedule_ID) REFERENCES EXAM_SCHEDULE(Schedule_ID)
 13  );

Table created.

SQL> INSERT INTO SEAT_ALLOCATION VALUES
(1,1,1,1,'A1',1,1,'R1C1','AUTO',0,SYSTIMESTAMP,'ALLOCATED',NULL,NULL);

1 row created.

SQL> INSERT INTO SEAT_ALLOCATION VALUES
(2,2,2,2,'A2',1,2,'R1C2','AUTO',0,SYSTIMESTAMP,'ALLOCATED',NULL,NULL);

1 row created.

SQL> INSERT INTO SEAT_ALLOCATION VALUES
(3,3,3,3,'A3',1,3,'R1C3','AUTO',0,SYSTIMESTAMP,'ALLOCATED',NULL,NULL);

1 row created.

SQL> INSERT INTO SEAT_ALLOCATION VALUES
(4,4,4,4,'A4',1,4,'R1C4','AUTO',0,SYSTIMESTAMP,'ALLOCATED',NULL,NULL);

1 row created.

SQL> INSERT INTO SEAT_ALLOCATION VALUES
(5,5,5,5,'A5',1,5,'R1C5','AUTO',0,SYSTIMESTAMP,'ALLOCATED',NULL,NULL);

1 row created.
```

```
SQL> INSERT INTO SEAT_ALLOCATION VALUES
(6,6,6,6,'A6',1,6,'R1C6','AUTO',0,SYSTIMESTAMP,'ALLOCATED',NULL,NULL);

1 row created.

SQL> INSERT INTO SEAT_ALLOCATION VALUES
(7,7,7,7,'A7',1,7,'R1C7','AUTO',0,SYSTIMESTAMP,'ALLOCATED',NULL,NULL);

1 row created.

SQL> INSERT INTO SEAT_ALLOCATION VALUES
(8,8,8,8,'A8',1,8,'R1C8','AUTO',0,SYSTIMESTAMP,'ALLOCATED',NULL,NULL);

1 row created.

SQL> INSERT INTO SEAT_ALLOCATION VALUES
(9,9,9,9,'A9',1,9,'R1C9','AUTO',0,SYSTIMESTAMP,'ALLOCATED',NULL,NULL);

1 row created.

SQL> INSERT INTO SEAT_ALLOCATION VALUES
(10,10,10,10,'A10',1,10,'R1C10','AUTO',0,SYSTIMESTAMP,'ALLOCATED',NULL,NULL);

1 row created.

SQL> INSERT INTO ATTENDANCE VALUES
(1,1,1,1,'PRESENT',SYSTIMESTAMP,'BIOMETRIC','VERIFIED',1,'DEV1',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO ATTENDANCE VALUES
(2,2,2,2,'PRESENT',SYSTIMESTAMP,'BIOMETRIC','VERIFIED',2,'DEV2',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO ATTENDANCE VALUES
(3,3,3,3,'ABSENT',NULL,'MANUAL','NOT_VERIFIED',3,'DEV3',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO ATTENDANCE VALUES
(4,4,4,4,'LATE',SYSTIMESTAMP,'BIOMETRIC','VERIFIED',4,'DEV4',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO ATTENDANCE VALUES
(5,5,5,5,'PRESENT',SYSTIMESTAMP,'BIOMETRIC','VERIFIED',5,'DEV5',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO ATTENDANCE VALUES
(6,6,6,6,'PRESENT',SYSTIMESTAMP,'BIOMETRIC','VERIFIED',6,'DEV6',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO ATTENDANCE VALUES
(7,7,7,7,'ABSENT',NULL,'MANUAL','NOT_VERIFIED',7,'DEV7',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO ATTENDANCE VALUES
(8,8,8,8,'PRESENT',SYSTIMESTAMP,'BIOMETRIC','VERIFIED',8,'DEV8',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO ATTENDANCE VALUES
(9,9,9,9,'PRESENT',SYSTIMESTAMP,'BIOMETRIC','VERIFIED',9,'DEV9',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO ATTENDANCE VALUES
(10,10,10,10,'PRESENT',SYSTIMESTAMP,'BIOMETRIC','VERIFIED',10,'DEV10',SYSTIMESTAMP,NULL);
```

```
1 row created.

SQL> INSERT INTO ROOM_ATTENDANCE_SUMMARY VALUES (1,1,30,28,1,0,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ROOM_ATTENDANCE_SUMMARY VALUES (2,2,25,24,0,0,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ROOM_ATTENDANCE_SUMMARY VALUES (3,3,30,25,2,1,3,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ROOM_ATTENDANCE_SUMMARY VALUES (4,4,40,38,1,0,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ROOM_ATTENDANCE_SUMMARY VALUES (5,5,35,33,1,0,2,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ROOM_ATTENDANCE_SUMMARY VALUES (6,6,30,29,0,0,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ROOM_ATTENDANCE_SUMMARY VALUES (7,7,45,40,2,1,3,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ROOM_ATTENDANCE_SUMMARY VALUES (8,8,30,28,1,0,1,SYSTIMESTAMP);

1 row created.
```

```
SQL> INSERT INTO ROOM_ATTENDANCE_SUMMARY VALUES (9,9,25,24,0,0,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ROOM_ATTENDANCE_SUMMARY VALUES (10,10,20,19,0,0,1,SYSTIMESTAMP);

1 row created.
```

## APPLIED SQL QUERIES AND BUSINESS USE CASES – STUDENT DISTRIBUTION AND ATTENDANCE MODULE:

1. **Complete Seat Allocation Details**
2. **Unallocated Student Identification**
3. **Duplicate Seat Assignment Detection**
4. **Buffer Seat Usage Identification**
5. **Seat Reassignment Tracking**
6. **Attendance Summary Per Schedule**
7. **Absent Student Identification**
8. **Late Entry Detection**
9. **Biometric Verification Failure Detection**
10. **Attendance Without Invigilator Verification**
11. **Attendance and Allocation Mismatch Detection**
12. **Room Wise Attendance Performance Analysis**
13. **High Absentee Rate Room Identification**
14. **Malpractice Heavy Room Detection**
15. **Real Time Attendance Tracking**
16. **Mode Wise Attendance Analysis**
17. **Device Wise Attendance Tracking**
18. **Fraud Detection Based on Biometric Mismatch**
19. **Seat Occupancy Per Room**

## 20. Room Over Allocation Detection

```
SQL> -- View complete seating allocation details
SQL> SELECT sa.Allocation_ID,
  2       sa.Schedule_ID,
  3       sa.Registration_ID,
  4       sa.Room_ID,
  5       sa.Seat_Number,
  6       sa.Row_Number,
  7       sa.Column_Number,
  8       sa.Allocation_Status
  9  FROM SEAT_ALLOCATION sa;

ALLOCATION_ID SCHEDULE_ID REGISTRATION_ID    ROOM_ID SEAT_NUMBER
------------- ----------- --------------- ---------- --------------------
ROW_NUMBER COLUMN_NUMBER ALLOCATION_STATUS
---------- ------------- --------------------------------------------------
         1           1             1         1 A1
         1           1 ALLOCATED

         2           2             2         2 A2
         1           2 ALLOCATED

         3           3             3         3 A3
         1           3 ALLOCATED


ALLOCATION_ID SCHEDULE_ID REGISTRATION_ID    ROOM_ID SEAT_NUMBER
------------- ----------- --------------- ---------- --------------------
ROW_NUMBER COLUMN_NUMBER ALLOCATION_STATUS
---------- ------------- --------------------------------------------------
         4           4             4         4 A4
         1           4 ALLOCATED
```

```
         5         5            5        5 A5
        1          5 ALLOCATED


         6         6            6        6 A6
        1          6 ALLOCATED



ALLOCATION_ID SCHEDULE_ID REGISTRATION_ID    ROOM_ID SEAT_NUMBER
------------- ----------- --------------- ---------- --------------------
ROW_NUMBER COLUMN_NUMBER ALLOCATION_STATUS
---------- ------------- --------------------------------------------------
         7         7            7        7 A7
        1          7 ALLOCATED


         8         8            8        8 A8
        1          8 ALLOCATED


         9         9            9        9 A9
        1          9 ALLOCATED



ALLOCATION_ID SCHEDULE_ID REGISTRATION_ID    ROOM_ID SEAT_NUMBER
------------- ----------- --------------- ---------- --------------------
ROW_NUMBER COLUMN_NUMBER ALLOCATION_STATUS
---------- ------------- --------------------------------------------------
        10        10           10       10 A10
        1         10 ALLOCATED



10 rows selected.


SQL> -- Students who have no seat assigned
SQL> SELECT r.Registration_ID
```

```
  2  FROM REGISTRATION r
  3  WHERE r.Registration_ID NOT IN (
  4    SELECT sa.Registration_ID FROM SEAT_ALLOCATION sa
  5  );

no rows selected

SQL> -- Same seat assigned to multiple students
SQL> SELECT sa.Schedule_ID,
  2       sa.Room_ID,
  3       sa.Seat_Number,
  4       COUNT(*) AS duplicate_count
  5  FROM SEAT_ALLOCATION sa
  6  GROUP BY sa.Schedule_ID, sa.Room_ID, sa.Seat_Number
  7  HAVING COUNT(*) > 1;

no rows selected

SQL> -- Identify buffer seats usage
SQL> SELECT sa.Allocation_ID,
  2       sa.Seat_Number
  3  FROM SEAT_ALLOCATION sa
  4  WHERE sa.Is_Buffer_Seat = 1;

no rows selected

SQL> -- Students whose seats were reassigned
SQL> SELECT sa.Registration_ID,
  2       sa.Seat_Number,
  3       sa.Reassigned_From_Seat
  4  FROM SEAT_ALLOCATION sa
  5  WHERE sa.Reassigned_From_Seat IS NOT NULL;

no rows selected
```

```
SQL> -- Count attendance status per schedule
SQL> SELECT a.Schedule_ID,
  2       a.Attendance_Status,
  3       COUNT(*) AS total
  4  FROM ATTENDANCE a
  5  GROUP BY a.Schedule_ID, a.Attendance_Status;

SCHEDULE_ID ATTENDANCE_STATUS                          TOTAL
----------- -------------------------------------------------- ----------
          1 PRESENT                                         1
          2 PRESENT                                         1
          3 ABSENT                                          1
          4 LATE                                            1
          5 PRESENT                                         1
          6 PRESENT                                         1
          7 ABSENT                                          1
          8 PRESENT                                         1
          9 PRESENT                                         1
         10 PRESENT                                         1

10 rows selected.

SQL> -- List of absent candidates
SQL> SELECT a.Registration_ID,
  2       a.Schedule_ID
  3  FROM ATTENDANCE a
  4  WHERE a.Attendance_Status = 'ABSENT';

REGISTRATION_ID SCHEDULE_ID
--------------- -----------
              3           3
              7           7
```

```
SQL> -- Students marked late
SQL> SELECT a.Registration_ID,
  2       a.Schedule_ID,
  3       a.Checking_Time
  4  FROM ATTENDANCE a
  5  WHERE a.Attendance_Status = 'LATE';

REGISTRATION_ID SCHEDULE_ID
--------------- -----------
CHECKING_TIME
---------------------------------------------------------------------------
           4         4
14-APR-26 05.31.28.121000 PM


SQL> -- Identify biometric failures
SQL> SELECT a.Registration_ID,
  2       a.Biometric_Verification_Status
  3  FROM ATTENDANCE a
  4  WHERE a.Biometric_Verification_Status <> 'VERIFIED';

REGISTRATION_ID BIOMETRIC_VERIFICATION_STATUS
--------------- -------------------------------------------------
           3 NOT_VERIFIED
           7 NOT_VERIFIED

SQL> -- Missing invigilator verification
SQL> SELECT a.Attendance_ID,
  2       a.Registration_ID
  3  FROM ATTENDANCE a
  4  WHERE a.Verified_By_Invigilator_ID IS NULL;

no rows selected
```

```
SQL> -- Students marked present but no seat allocation
SQL> SELECT a.Registration_ID
  2  FROM ATTENDANCE a
  3  WHERE a.Attendance_Status = 'PRESENT'
  4  AND a.Registration_ID NOT IN (
  5     SELECT sa.Registration_ID FROM SEAT_ALLOCATION sa
  6  );

no rows selected

SQL> -- Analyze attendance per room
SQL> SELECT ras.Room_ID,
  2       ras.Total_Allocated,
  3       ras.Total_Present,
  4       ras.Total_Absent
  5  FROM ROOM_ATTENDANCE_SUMMARY ras;

   ROOM_ID TOTAL_ALLOCATED TOTAL_PRESENT TOTAL_ABSENT
---------- --------------- ------------- ------------
         1              30            28            1
         2              25            24            1
         3              30            25            3
         4              40            38            1
         5              35            33            2
         6              30            29            1
         7              45            40            3
         8              30            28            1
         9              25            24            1
        10              20            19            1

10 rows selected.

SQL> -- Identify problematic rooms
SQL> SELECT ras.Room_ID,
```

```
  2       ras.Total_Absent,
  3       ras.Total_Allocated
  4  FROM ROOM_ATTENDANCE_SUMMARY ras
  5  WHERE ras.Total_Absent > (ras.Total_Allocated * 0.2);

no rows selected

SQL> -- Rooms with malpractice cases
SQL> SELECT ras.Room_ID,
  2       ras.Total_Malpractice_Cases
  3  FROM ROOM_ATTENDANCE_SUMMARY ras
  4  WHERE ras.Total_Malpractice_Cases > 0;

   ROOM_ID TOTAL_MALPRACTICE_CASES
---------- ------------------------
         3                        1
         7                        1

SQL> -- Latest attendance entries
SQL> SELECT a.Attendance_ID,
  2       a.Registration_ID,
  3       a.Attendance_Timestamp
  4  FROM ATTENDANCE a
  5  ORDER BY a.Attendance_Timestamp DESC;

ATTENDANCE_ID REGISTRATION_ID
------------- ---------------
ATTENDANCE_TIMESTAMP
---------------------------------------------------------------------------
           10             10
14-APR-26 05.31.28.486000 PM

            9              9
14-APR-26 05.31.28.142000 PM
```

8 8

14-APR-26 05.31.28.138000 PM

ATTENDANCE_ID REGISTRATION_ID

---------- ----------

ATTENDANCE_TIMESTAMP

---

7 7

14-APR-26 05.31.28.133000 PM

6 6

14-APR-26 05.31.28.129000 PM

5 5

14-APR-26 05.31.28.125000 PM

ATTENDANCE_ID REGISTRATION_ID

---------- ----------

ATTENDANCE_TIMESTAMP

---

4 4

14-APR-26 05.31.28.121000 PM

3 3

14-APR-26 05.31.28.118000 PM

2 2

14-APR-26 05.31.28.114000 PM

ATTENDANCE_ID REGISTRATION_ID

```
------------- ---------------
ATTENDANCE_TIMESTAMP
---------------------------------------------------------------------------
         1           1
14-APR-26 05.31.28.100000 PM



10 rows selected.


SQL> -- Analyze checking modes
SQL> SELECT a.Checking_Mode,
  2       COUNT(*) AS total
  3  FROM ATTENDANCE a
  4  GROUP BY a.Checking_Mode;

CHECKING_MODE                                TOTAL
-------------------------------------------------- ----------
BIOMETRIC                                         8
MANUAL                                            2


SQL> -- Monitor device usage
SQL> SELECT a.Attendance_Device_ID,
  2       COUNT(*) AS usage_count
  3  FROM ATTENDANCE a
  4  GROUP BY a.Attendance_Device_ID;

ATTENDANCE_DEVICE_ID
--------------------------------------------------------------------------------
USAGE_COUNT
-----------
DEV1
        1


DEV2
```

1

DEV3

1

ATTENDANCE_DEVICE_ID

---

USAGE_COUNT

---

DEV4

1

DEV5

1

DEV6

1

ATTENDANCE_DEVICE_ID

---

USAGE_COUNT

---

DEV7

1

DEV8

1

DEV9

1

```
ATTENDANCE_DEVICE_ID
--------------------------------------------------------------------------------
USAGE_COUNT
-----------
DEV10
         1



10 rows selected.


SQL> -- Potential fraud cases
SQL> SELECT a.Registration_ID
  2  FROM ATTENDANCE a
  3  WHERE a.Attendance_Status = 'PRESENT'
  4  AND a.Biometric_Verification_Status <> 'VERIFIED';


no rows selected


SQL> -- Number of seats used per room
SQL> SELECT sa.Room_ID,
  2       COUNT(*) AS occupied_seats
  3  FROM SEAT_ALLOCATION sa
  4  GROUP BY sa.Room_ID;


   ROOM_ID OCCUPIED_SEATS
---------- --------------
         1             1
         2             1
         3             1
         4             1
         5             1
         6             1
         7             1
         8             1
```

```
      9          1
     10          1

10 rows selected.

SQL> -- Allocations exceeding room capacity
SQL> SELECT sa.Room_ID,
  2       COUNT(*) AS allocated_seats,
  3       r.Capacity
  4  FROM SEAT_ALLOCATION sa
  5  JOIN ROOM r ON sa.Room_ID = r.Room_ID
  6  GROUP BY sa.Room_ID, r.Capacity
  7  HAVING COUNT(*) > r.Capacity;

no rows selected

SQL> SPOOL OFF;
```

## IMPLEMENTATION OF MODULE 6 – SECURE EVALUATION MODULE

## (i) DESIGN AND IMPLEMENTATION OF ENTITIES WITH DATA INITIALIZATION:

**This module focuses on the creation and management of entities related to secure evaluation processes, including the recording of student responses, evaluation of answer scripts, assignment of evaluators, and the insertion of essential data required to ensure accuracy, confidentiality, and integrity in the assessment and grading process within the system.**

```
SQL> CREATE TABLE ANSWER_SCRIPT (
  2    Script_ID NUMBER PRIMARY KEY,
  3    Registration_ID NUMBER NOT NULL,
  4    Student_ID NUMBER NOT NULL,
  5    Schedule_ID NUMBER NOT NULL,
  6    Subject_ID NUMBER NOT NULL,
  7    Script_Barcode VARCHAR2(100) UNIQUE NOT NULL,
  8    Anonymous_Code VARCHAR2(100) UNIQUE NOT NULL,
```

```
  9   Script_Version NUMBER DEFAULT 1,
 10   Upload_Mode VARCHAR2(50) NOT NULL,
 11   Encrypted_File_Path VARCHAR2(500) NOT NULL,
 12   Encryption_Key_ID VARCHAR2(100) NOT NULL,
 13   Script_Page_Count NUMBER NOT NULL,
 14   Submission_Timestamp TIMESTAMP NOT NULL,
 15   Decryption_Status VARCHAR2(50) NOT NULL,
 16   Is_Revaluation_Requested NUMBER(1) DEFAULT 0,
 17   Script_Status VARCHAR2(50) NOT NULL,
 18   Created_At TIMESTAMP NOT NULL,
 19   FOREIGN KEY (Registration_ID) REFERENCES REGISTRATION(Registration_ID),
 20   FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
 21   FOREIGN KEY (Schedule_ID) REFERENCES EXAM_SCHEDULE(Schedule_ID),
 22   FOREIGN KEY (Subject_ID) REFERENCES SUBJECT(Subject_ID)
 23 );

Table created.

SQL> CREATE TABLE EVALUATOR (
  2   Evaluator_ID NUMBER PRIMARY KEY,
  3   Employee_Code VARCHAR2(30) UNIQUE NOT NULL,
  4   First_Name VARCHAR2(100) NOT NULL,
  5   Last_Name VARCHAR2(100) NOT NULL,
  6   Email VARCHAR2(150) UNIQUE NOT NULL,
  7   Phone VARCHAR2(15) NOT NULL,
  8   Subject_ID NUMBER NOT NULL,
  9   Qualification_Level VARCHAR2(100) NOT NULL,
 10   Years_of_Experience NUMBER NOT NULL,
 11   Max_Scripts_Per_Day NUMBER NOT NULL,
 12   Max_Scripts_Per_Cycle NUMBER NOT NULL,
 13   Background_Verification_Status VARCHAR2(50) NOT NULL,
 14   FOREIGN KEY (Subject_ID) REFERENCES SUBJECT(Subject_ID)
 15 );
```

Table created.

```
SQL> CREATE TABLE EVALUATION_ASSIGNMENT (
  2    Assignment_ID NUMBER PRIMARY KEY,
  3    Script_ID NUMBER NOT NULL,
  4    Evaluator_ID NUMBER NOT NULL,
  5    Assignment_Round NUMBER NOT NULL,
  6    Assigned_Timestamp TIMESTAMP NOT NULL,
  7    Due_Date DATE NOT NULL,
  8    Assignment_Status VARCHAR2(50) NOT NULL,
  9    Blind_Evaluation_Code VARCHAR2(100) NOT NULL,
 10    Assigned_By NUMBER NOT NULL,
 11    Last_Updated TIMESTAMP NOT NULL,
 12    FOREIGN KEY (Script_ID) REFERENCES ANSWER_SCRIPT(Script_ID),
 13    FOREIGN KEY (Evaluator_ID) REFERENCES EVALUATOR(Evaluator_ID)
 14  );
```

Table created.

```
SQL> CREATE TABLE MARKING_SCHEME (
  2    Scheme_ID NUMBER PRIMARY KEY,
  3    Subject_ID NUMBER NOT NULL,
  4    Question_Number NUMBER NOT NULL,
  5    Max_Marks NUMBER(5,2) NOT NULL,
  6    Negative_Marking_Flag NUMBER(1) DEFAULT 0,
  7    Partial_Marking_Allowed NUMBER(1) DEFAULT 0,
  8    Created_By NUMBER NOT NULL,
  9    Approved_By NUMBER,
 10    Created_At TIMESTAMP NOT NULL,
 11    FOREIGN KEY (Subject_ID) REFERENCES SUBJECT(Subject_ID)
 12  );
```

Table created.

```
SQL> CREATE TABLE EVALUATION_RECORD (
  2    Evaluation_ID NUMBER PRIMARY KEY,
  3    Assignment_ID NUMBER NOT NULL,
  4    Marks_Awarded NUMBER(6,2) NOT NULL,
  5    Remarks CLOB,
  6    Marking_Timestamp TIMESTAMP NOT NULL,
  7    Marking_Device_ID VARCHAR2(100),
  8    Auto_Flag_For_Review NUMBER(1) DEFAULT 0,
  9    Is_Finalized NUMBER(1) DEFAULT 0,
 10    FOREIGN KEY (Assignment_ID) REFERENCES EVALUATION_ASSIGNMENT(Assignment_ID)
 11  );

Table created.

SQL> CREATE TABLE DOUBLE_EVALUATION_COMPARISON (
  2    Comparison_ID NUMBER PRIMARY KEY,
  3    Script_ID NUMBER NOT NULL,
  4    Primary_Evaluator_ID NUMBER NOT NULL,
  5    Secondary_Evaluator_ID NUMBER NOT NULL,
  6    Primary_Total NUMBER(8,2) NOT NULL,
  7    Secondary_Total NUMBER(8,2) NOT NULL,
  8    Absolute_Difference NUMBER(8,2) NOT NULL,
  9    Difference_Percentage NUMBER(5,2) NOT NULL,
 10    Requires_Moderation_Flag NUMBER(1) DEFAULT 0,
 11    Compared_At TIMESTAMP NOT NULL,
 12    FOREIGN KEY (Script_ID) REFERENCES ANSWER_SCRIPT(Script_ID),
 13    FOREIGN KEY (Primary_Evaluator_ID) REFERENCES EVALUATOR(Evaluator_ID),
 14    FOREIGN KEY (Secondary_Evaluator_ID) REFERENCES EVALUATOR(Evaluator_ID)
 15  );

Table created.

SQL> CREATE TABLE MODERATION_RECORD (
  2    Moderation_ID NUMBER PRIMARY KEY,
```

```
  3   Script_ID NUMBER NOT NULL,
  4   Moderator_ID NUMBER NOT NULL,
  5   Previous_Total NUMBER(8,2) NOT NULL,
  6   Revised_Total NUMBER(8,2) NOT NULL,
  7   Moderation_Reason CLOB NOT NULL,
  8   Moderation_Timestamp TIMESTAMP NOT NULL,
  9   Approved_By NUMBER,
 10   Finalized_Flag NUMBER(1) DEFAULT 0,
 11   FOREIGN KEY (Script_ID) REFERENCES ANSWER_SCRIPT(Script_ID)
 12 );

Table created.

SQL> CREATE TABLE REVALUATION_REQUEST (
  2   Request_ID NUMBER PRIMARY KEY,
  3   Script_ID NUMBER NOT NULL,
  4   Student_ID NUMBER NOT NULL,
  5   Request_Type VARCHAR2(50) NOT NULL,
  6   Request_Date DATE NOT NULL,
  7   Fee_Paid NUMBER(10,2) NOT NULL,
  8   Payment_Transaction_ID NUMBER,
  9   Request_Status VARCHAR2(50) NOT NULL,
 10   Finalized_Flag NUMBER(1) DEFAULT 0,
 11   Processed_By NUMBER,
 12   Processed_At TIMESTAMP,
 13   Final_Result_After_Reval NUMBER(8,2),
 14   FOREIGN KEY (Script_ID) REFERENCES ANSWER_SCRIPT(Script_ID),
 15   FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
 16   FOREIGN KEY (Payment_Transaction_ID) REFERENCES PAYMENT_TRANSACTION(Transaction_ID)
 17 );

Table created.

SQL> CREATE TABLE EVALUATOR_WORKLOAD (
```

```
  2    Workload_ID NUMBER PRIMARY KEY,
  3    Evaluator_ID NUMBER NOT NULL,
  4    Exam_Cycle_ID NUMBER NOT NULL,
  5    Assigned_Scripts_Count NUMBER DEFAULT 0,
  6    Completed_Scripts_Count NUMBER DEFAULT 0,
  7    Pending_Scripts_Count NUMBER DEFAULT 0,
  8    Average_Marking_Time NUMBER(6,2),
  9    Overtime_Hours NUMBER(6,2) DEFAULT 0,
 10    Performance_Rating NUMBER(3,2),
 11    Last_Updated TIMESTAMP NOT NULL,
 12    FOREIGN KEY (Evaluator_ID) REFERENCES EVALUATOR(Evaluator_ID),
 13    FOREIGN KEY (Exam_Cycle_ID) REFERENCES EXAM_CYCLE(Exam_Cycle_ID)
 14  );

Table created.

SQL> INSERT INTO ANSWER_SCRIPT VALUES
(1,1,1,1,1,'B1','A1',1,'ONLINE','/path1','KEY1',20,SYSTIMESTAMP,'DECRYPTED',0,'SUBMITTED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ANSWER_SCRIPT VALUES
(2,2,2,2,2,'B2','A2',1,'ONLINE','/path2','KEY2',22,SYSTIMESTAMP,'DECRYPTED',0,'SUBMITTED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ANSWER_SCRIPT VALUES
(3,3,3,3,3,'B3','A3',1,'SCAN','/path3','KEY3',18,SYSTIMESTAMP,'DECRYPTED',1,'UNDER_REVIEW',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ANSWER_SCRIPT VALUES
(4,4,4,4,4,'B4','A4',1,'ONLINE','/path4','KEY4',25,SYSTIMESTAMP,'DECRYPTED',0,'SUBMITTED',SYSTIMESTAMP);

1 row created.
```

```
SQL> INSERT INTO ANSWER_SCRIPT VALUES
(5,5,5,5,5,'B5','A5',1,'SCAN','/path5','KEY5',20,SYSTIMESTAMP,'DECRYPTED',0,'SUBMITTED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ANSWER_SCRIPT VALUES
(6,6,6,6,6,'B6','A6',1,'ONLINE','/path6','KEY6',21,SYSTIMESTAMP,'DECRYPTED',0,'SUBMITTED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ANSWER_SCRIPT VALUES
(7,7,7,7,7,'B7','A7',1,'SCAN','/path7','KEY7',19,SYSTIMESTAMP,'DECRYPTED',1,'UNDER_REVIEW',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ANSWER_SCRIPT VALUES
(8,8,8,8,8,'B8','A8',1,'ONLINE','/path8','KEY8',23,SYSTIMESTAMP,'DECRYPTED',0,'SUBMITTED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ANSWER_SCRIPT VALUES
(9,9,9,9,9,'B9','A9',1,'SCAN','/path9','KEY9',17,SYSTIMESTAMP,'DECRYPTED',0,'SUBMITTED',SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO ANSWER_SCRIPT VALUES
(10,10,10,10,10,'B10','A10',1,'ONLINE','/path10','KEY10',24,SYSTIMESTAMP,'DECRYPTED',0,'SUBMITTED',SYSTIMESTAMP
);

1 row created.

SQL> INSERT INTO EVALUATOR VALUES (1,'EV1','A','B','e1@mail.com','8001',1,'PG',10,50,200,'VERIFIED');

1 row created.
```

```
SQL> INSERT INTO EVALUATOR VALUES (2,'EV2','A','B','e2@mail.com','8002',2,'PG',9,50,200,'VERIFIED');

1 row created.

SQL> INSERT INTO EVALUATOR VALUES (3,'EV3','A','B','e3@mail.com','8003',3,'UG',7,40,150,'VERIFIED');

1 row created.

SQL> INSERT INTO EVALUATOR VALUES (4,'EV4','A','B','e4@mail.com','8004',4,'PG',12,60,250,'VERIFIED');

1 row created.

SQL> INSERT INTO EVALUATOR VALUES (5,'EV5','A','B','e5@mail.com','8005',5,'UG',6,40,150,'VERIFIED');

1 row created.

SQL> INSERT INTO EVALUATOR VALUES (6,'EV6','A','B','e6@mail.com','8006',6,'PG',11,50,200,'VERIFIED');

1 row created.

SQL> INSERT INTO EVALUATOR VALUES (7,'EV7','A','B','e7@mail.com','8007',7,'UG',5,40,150,'VERIFIED');

1 row created.

SQL> INSERT INTO EVALUATOR VALUES (8,'EV8','A','B','e8@mail.com','8008',8,'PG',13,60,250,'VERIFIED');

1 row created.

SQL> INSERT INTO EVALUATOR VALUES (9,'EV9','A','B','e9@mail.com','8009',9,'UG',4,30,120,'VERIFIED');

1 row created.

SQL> INSERT INTO EVALUATOR VALUES (10,'EV10','A','B','e10@mail.com','8010',10,'PG',8,50,200,'VERIFIED');
```

1 row created.

SQL> INSERT INTO EVALUATION_ASSIGNMENT VALUES (1,1,1,1,SYSTIMESTAMP,DATE '2026-05-10','ASSIGNED','BL1',101,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATION_ASSIGNMENT VALUES (2,2,2,1,SYSTIMESTAMP,DATE '2026-05-10','ASSIGNED','BL2',102,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATION_ASSIGNMENT VALUES (3,3,3,1,SYSTIMESTAMP,DATE '2026-05-10','ASSIGNED','BL3',103,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATION_ASSIGNMENT VALUES (4,4,4,1,SYSTIMESTAMP,DATE '2026-05-10','ASSIGNED','BL4',104,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATION_ASSIGNMENT VALUES (5,5,5,1,SYSTIMESTAMP,DATE '2026-05-10','ASSIGNED','BL5',105,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATION_ASSIGNMENT VALUES (6,6,6,1,SYSTIMESTAMP,DATE '2026-05-10','ASSIGNED','BL6',106,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATION_ASSIGNMENT VALUES (7,7,7,1,SYSTIMESTAMP,DATE '2026-05-10','ASSIGNED','BL7',107,SYSTIMESTAMP);

1 row created.

```
SQL> INSERT INTO EVALUATION_ASSIGNMENT VALUES (8,8,8,1,SYSTIMESTAMP,DATE '2026-05-
10','ASSIGNED','BL8',108,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATION_ASSIGNMENT VALUES (9,9,9,1,SYSTIMESTAMP,DATE '2026-05-
10','ASSIGNED','BL9',109,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATION_ASSIGNMENT VALUES (10,10,10,1,SYSTIMESTAMP,DATE '2026-05-
10','ASSIGNED','BL10',110,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO MARKING_SCHEME VALUES (1,1,1,5,0,1,101,201,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO MARKING_SCHEME VALUES (2,2,1,5,1,1,102,202,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO MARKING_SCHEME VALUES (3,3,1,10,0,0,103,203,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO MARKING_SCHEME VALUES (4,4,1,8,1,1,104,204,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO MARKING_SCHEME VALUES (5,5,1,6,0,1,105,205,SYSTIMESTAMP);

1 row created.
```

```
SQL> INSERT INTO MARKING_SCHEME VALUES (6,6,1,5,0,1,106,206,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO MARKING_SCHEME VALUES (7,7,1,7,1,0,107,207,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO MARKING_SCHEME VALUES (8,8,1,9,0,1,108,208,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO MARKING_SCHEME VALUES (9,9,1,4,0,1,109,209,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO MARKING_SCHEME VALUES (10,10,1,5,1,1,110,210,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATION_RECORD VALUES (1,1,80,NULL,SYSTIMESTAMP,'DEV1',0,1);

1 row created.

SQL> INSERT INTO EVALUATION_RECORD VALUES (2,2,75,NULL,SYSTIMESTAMP,'DEV2',0,1);

1 row created.

SQL> INSERT INTO EVALUATION_RECORD VALUES (3,3,60,NULL,SYSTIMESTAMP,'DEV3',1,0);

1 row created.

SQL> INSERT INTO EVALUATION_RECORD VALUES (4,4,85,NULL,SYSTIMESTAMP,'DEV4',0,1);
```

```
1 row created.

SQL> INSERT INTO EVALUATION_RECORD VALUES (5,5,70,NULL,SYSTIMESTAMP,'DEV5',0,1);

1 row created.

SQL> INSERT INTO EVALUATION_RECORD VALUES (6,6,90,NULL,SYSTIMESTAMP,'DEV6',0,1);

1 row created.

SQL> INSERT INTO EVALUATION_RECORD VALUES (7,7,55,NULL,SYSTIMESTAMP,'DEV7',1,0);

1 row created.

SQL> INSERT INTO EVALUATION_RECORD VALUES (8,8,88,NULL,SYSTIMESTAMP,'DEV8',0,1);

1 row created.

SQL> INSERT INTO EVALUATION_RECORD VALUES (9,9,65,NULL,SYSTIMESTAMP,'DEV9',0,1);

1 row created.

SQL> INSERT INTO EVALUATION_RECORD VALUES (10,10,78,NULL,SYSTIMESTAMP,'DEV10',0,1);

1 row created.

SQL> INSERT INTO DOUBLE_EVALUATION_COMPARISON VALUES (1,1,1,2,80,82,2,2.5,0,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO DOUBLE_EVALUATION_COMPARISON VALUES (2,2,2,3,75,70,5,6.6,1,SYSTIMESTAMP);

1 row created.
```

```
SQL> INSERT INTO DOUBLE_EVALUATION_COMPARISON VALUES (3,3,3,4,60,65,5,8.3,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO DOUBLE_EVALUATION_COMPARISON VALUES (4,4,4,5,85,84,1,1.1,0,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO DOUBLE_EVALUATION_COMPARISON VALUES (5,5,5,6,70,72,2,2.8,0,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO DOUBLE_EVALUATION_COMPARISON VALUES (6,6,6,7,90,85,5,5.5,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO DOUBLE_EVALUATION_COMPARISON VALUES (7,7,7,8,55,60,5,9.0,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO DOUBLE_EVALUATION_COMPARISON VALUES (8,8,8,9,88,87,1,1.1,0,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO DOUBLE_EVALUATION_COMPARISON VALUES (9,9,9,10,65,67,2,3.0,0,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO DOUBLE_EVALUATION_COMPARISON VALUES (10,10,10,1,78,80,2,2.5,0,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO MODERATION_RECORD VALUES (1,2,101,75,78,'Recheck',SYSTIMESTAMP,NULL,1);
```

```
1 row created.

SQL> INSERT INTO MODERATION_RECORD VALUES (2,3,102,60,65,'Variance',SYSTIMESTAMP,NULL,1);

1 row created.

SQL> INSERT INTO MODERATION_RECORD VALUES (3,6,103,90,88,'Correction',SYSTIMESTAMP,NULL,1);

1 row created.

SQL> INSERT INTO MODERATION_RECORD VALUES (4,7,104,55,60,'Review',SYSTIMESTAMP,NULL,1);

1 row created.

SQL> INSERT INTO MODERATION_RECORD VALUES (5,5,105,70,72,'Check',SYSTIMESTAMP,NULL,1);

1 row created.

SQL> INSERT INTO MODERATION_RECORD VALUES (6,4,106,85,86,'Adjustment',SYSTIMESTAMP,NULL,1);

1 row created.

SQL> INSERT INTO MODERATION_RECORD VALUES (7,1,107,80,82,'Fix',SYSTIMESTAMP,NULL,1);

1 row created.

SQL> INSERT INTO MODERATION_RECORD VALUES (8,8,108,88,89,'Minor',SYSTIMESTAMP,NULL,1);

1 row created.

SQL> INSERT INTO MODERATION_RECORD VALUES (9,9,109,65,66,'Correction',SYSTIMESTAMP,NULL,1);

1 row created.
```

```
SQL> INSERT INTO MODERATION_RECORD VALUES (10,10,110,78,79,'Review',SYSTIMESTAMP,NULL,1);

1 row created.

SQL> INSERT INTO REVALUATION_REQUEST VALUES (1,3,3,'RECHECK',DATE '2026-05-
15',500,1,'PENDING',0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO REVALUATION_REQUEST VALUES (2,7,7,'RETOTAL',DATE '2026-05-
15',500,2,'PENDING',0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO REVALUATION_REQUEST VALUES (3,2,2,'RECHECK',DATE '2026-05-
15',500,3,'COMPLETED',1,101,SYSTIMESTAMP,78);

1 row created.

SQL> INSERT INTO REVALUATION_REQUEST VALUES (4,5,5,'RETOTAL',DATE '2026-05-
15',500,4,'COMPLETED',1,102,SYSTIMESTAMP,72);

1 row created.

SQL> INSERT INTO REVALUATION_REQUEST VALUES (5,6,6,'RECHECK',DATE '2026-05-
15',500,5,'PENDING',0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO REVALUATION_REQUEST VALUES (6,1,1,'RECHECK',DATE '2026-05-
15',500,6,'COMPLETED',1,103,SYSTIMESTAMP,82);

1 row created.
```

```
SQL> INSERT INTO REVALUATION_REQUEST VALUES (7,4,4,'RETOTAL',DATE '2026-05-
15',500,7,'PENDING',0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO REVALUATION_REQUEST VALUES (8,8,8,'RECHECK',DATE '2026-05-
15',500,8,'COMPLETED',1,104,SYSTIMESTAMP,89);

1 row created.

SQL> INSERT INTO REVALUATION_REQUEST VALUES (9,9,9,'RETOTAL',DATE '2026-05-
15',500,9,'PENDING',0,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO REVALUATION_REQUEST VALUES (10,10,10,'RECHECK',DATE '2026-05-
15',500,10,'COMPLETED',1,105,SYSTIMESTAMP,79);

1 row created.

SQL> INSERT INTO EVALUATOR_WORKLOAD VALUES (1,1,1,50,45,5,12.5,2,4.5,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATOR_WORKLOAD VALUES (2,2,1,40,38,2,13.0,1,4.2,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATOR_WORKLOAD VALUES (3,3,1,30,25,5,14.0,3,3.8,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATOR_WORKLOAD VALUES (4,4,1,60,55,5,11.0,2,4.7,SYSTIMESTAMP);

1 row created.
```

```
SQL> INSERT INTO EVALUATOR_WORKLOAD VALUES (5,5,1,35,30,5,12.0,1,4.0,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATOR_WORKLOAD VALUES (6,6,1,45,42,3,10.5,1,4.6,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATOR_WORKLOAD VALUES (7,7,1,25,20,5,15.0,4,3.5,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATOR_WORKLOAD VALUES (8,8,1,55,50,5,11.5,2,4.8,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATOR_WORKLOAD VALUES (9,9,1,20,18,2,16.0,3,3.2,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO EVALUATOR_WORKLOAD VALUES (10,10,1,40,37,3,13.5,1,4.3,SYSTIMESTAMP);

1 row created.
```

## APPLIED SQL QUERIES AND BUSINESS USE CASES – SECURE EVALUATION MODULE:

1. **Complete Answer Script Details**
2. **Pending Evaluation Scripts Identification**
3. **Evaluator Workload Analysis**
4. **Overloaded Evaluator Detection**
5. **Evaluation Discrepancy Detection**
6. **Scripts Requiring Moderation**

7. **Moderation Change Analysis**
8. **Extreme Moderation Change Detection**
9. **Pending Revaluation Requests**
10. **Revaluation Impact Analysis**
11. **Scripts Marked for Revaluation**
12. **Evaluator Performance Ranking**
13. **Slow Evaluator Detection**
14. **Evaluation Completion Status Tracking**
15. **Non Finalized Evaluation Identification**
16. **Auto Flagged Evaluation Detection**
17. **Evaluator Wise Script Assignment Count**
18. **Evaluator Bias Detection Based on Marks**
19. **Multiple Evaluation Rounds Detection**
20. **Answer Script Encryption Audit**

```
SQL> -- View answer script details with student and subject info

SQL> SELECT ascr.Script_ID,

  2       ascr.Registration_ID,

  3       ascr.Student_ID,

  4       ascr.Subject_ID,

  5       ascr.Script_Status,

  6       ascr.Submission_Timestamp

  7  FROM ANSWER_SCRIPT ascr;


 SCRIPT_ID REGISTRATION_ID STUDENT_ID SUBJECT_ID

---------- --------------- ---------- ----------

SCRIPT_STATUS

--------------------------------------------------

SUBMISSION_TIMESTAMP

-----------------------------------------------------------------------------

       1           1        1        1

SUBMITTED

14-APR-26 05.36.52.003000 PM
```

2 2 2 2

SUBMITTED

14-APR-26 05.36.52.042000 PM

SCRIPT_ID REGISTRATION_ID STUDENT_ID SUBJECT_ID

---------- ---------- ---------- ----------

SCRIPT_STATUS

---

SUBMISSION_TIMESTAMP

---

3 3 3 3

UNDER_REVIEW

14-APR-26 05.36.52.048000 PM

4 4 4 4

SUBMITTED

SCRIPT_ID REGISTRATION_ID STUDENT_ID SUBJECT_ID

---------- ---------- ---------- ----------

SCRIPT_STATUS

---

SUBMISSION_TIMESTAMP

---

14-APR-26 05.36.52.052000 PM

5 5 5 5

SUBMITTED

14-APR-26 05.36.52.056000 PM

6 6 6 6

SCRIPT_ID REGISTRATION_ID STUDENT_ID SUBJECT_ID

---------- ---------- ---------- ----------

SCRIPT_STATUS

---

SUBMISSION_TIMESTAMP

---

SUBMITTED

14-APR-26 05.36.52.063000 PM

7 7 7 7

UNDER_REVIEW

14-APR-26 05.36.52.068000 PM

SCRIPT_ID REGISTRATION_ID STUDENT_ID SUBJECT_ID

---

SCRIPT_STATUS

---

SUBMISSION_TIMESTAMP

---

8 8 8 8

SUBMITTED

14-APR-26 05.36.52.073000 PM

9 9 9 9

SUBMITTED

14-APR-26 05.36.52.079000 PM

SCRIPT_ID REGISTRATION_ID STUDENT_ID SUBJECT_ID

---

SCRIPT_STATUS

---

SUBMISSION_TIMESTAMP

---

10 10 10 10

```
SUBMITTED
14-APR-26 05.36.52.580000 PM



10 rows selected.


SQL> -- Scripts not yet assigned or evaluated
SQL> SELECT ascr.Script_ID
  2  FROM ANSWER_SCRIPT ascr
  3  WHERE ascr.Script_ID NOT IN (
  4    SELECT ea.Script_ID FROM EVALUATION_ASSIGNMENT ea
  5  );


no rows selected


SQL> -- Track evaluator efficiency
SQL> SELECT ew.Evaluator_ID,
  2       ew.Assigned_Scripts_Count,
  3       ew.Completed_Scripts_Count,
  4       ew.Pending_Scripts_Count
  5  FROM EVALUATOR_WORKLOAD ew;


EVALUATOR_ID ASSIGNED_SCRIPTS_COUNT COMPLETED_SCRIPTS_COUNT
------------ ---------------------- ------------------------
PENDING_SCRIPTS_COUNT
---------------------
         1                 50                 45
                5

         2                 40                 38
                2

         3                 30                 25
                5
```

EVALUATOR_ID ASSIGNED_SCRIPTS_COUNT COMPLETED_SCRIPTS_COUNT

---------- ---------- ----------

PENDING_SCRIPTS_COUNT

---

4 60 55

5

5 35 30

5

6 45 42

3

EVALUATOR_ID ASSIGNED_SCRIPTS_COUNT COMPLETED_SCRIPTS_COUNT

---------- ---------- ----------

PENDING_SCRIPTS_COUNT

---

7 25 20

5

8 55 50

5

9 20 18

2

EVALUATOR_ID ASSIGNED_SCRIPTS_COUNT COMPLETED_SCRIPTS_COUNT

---------- ---------- ----------

PENDING_SCRIPTS_COUNT

---

```
        10                40                37
                3


10 rows selected.


SQL> -- Evaluators exceeding workload limits
SQL> SELECT ew.Evaluator_ID,
  2       ew.Assigned_Scripts_Count,
  3       e.Max_Scripts_Per_Cycle
  4  FROM EVALUATOR_WORKLOAD ew
  5  JOIN EVALUATOR e ON ew.Evaluator_ID = e.Evaluator_ID
  6  WHERE ew.Assigned_Scripts_Count > e.Max_Scripts_Per_Cycle;


no rows selected


SQL> -- Detect major discrepancies between evaluators
SQL> SELECT dec.Script_ID,
  2       dec.Primary_Total,
  3       dec.Secondary_Total,
  4       dec.Absolute_Difference,
  5       dec.Difference_Percentage
  6  FROM DOUBLE_EVALUATION_COMPARISON dec
  7  WHERE dec.Difference_Percentage > 10;


no rows selected


SQL> -- Identify scripts flagged for moderation
SQL> SELECT dec.Script_ID
  2  FROM DOUBLE_EVALUATION_COMPARISON dec
  3  WHERE dec.Requires_Moderation_Flag = 1;


 SCRIPT_ID
----------
```

```
      2
      3
      6
      7

SQL> -- Track moderation impact
SQL> SELECT mr.Script_ID,
  2       mr.Previous_Total,
  3       mr.Revised_Total,
  4       mr.Moderation_Reason
  5  FROM MODERATION_RECORD mr;

 SCRIPT_ID PREVIOUS_TOTAL REVISED_TOTAL
---------- --------------- -------------
MODERATION_REASON
--------------------------------------------------------------------------------
      2          75          78
Recheck

      3          60          65
Variance

      6          90          88
Correction


 SCRIPT_ID PREVIOUS_TOTAL REVISED_TOTAL
---------- --------------- -------------
MODERATION_REASON
--------------------------------------------------------------------------------
      7          55          60
Review

      5          70          72
```

Check

4 85 86

Adjustment

SCRIPT_ID PREVIOUS_TOTAL REVISED_TOTAL

---------- ---------- ----------

MODERATION_REASON

---

1 80 82

Fix

8 88 89

Minor

9 65 66

Correction

SCRIPT_ID PREVIOUS_TOTAL REVISED_TOTAL

---------- ---------- ----------

MODERATION_REASON

---

10 78 79

Review

10 rows selected.

SQL> -- Large moderation corrections

SQL> SELECT mr.Script_ID,

2 mr.Previous_Total,

3 mr.Revised_Total

```
  4  FROM MODERATION_RECORD mr
  5  WHERE ABS(mr.Revised_Total - mr.Previous_Total) > 20;

no rows selected

SQL>
SQL> -- Students waiting for revaluation
SQL> SELECT rr.Request_ID,
  2       rr.Student_ID,
  3       rr.Script_ID
  4  FROM REVALUATION_REQUEST rr
  5  WHERE rr.Request_Status = 'PENDING';

REQUEST_ID STUDENT_ID  SCRIPT_ID
---------- ---------- ----------
         1          3          3
         2          7          7
         5          6          6
         7          4          4
         9          9          9

SQL> -- Compare original vs revaluation result
SQL> SELECT rr.Script_ID,
  2       rr.Final_Result_After_Reval
  3  FROM REVALUATION_REQUEST rr
  4  WHERE rr.Finalized_Flag = 1;

 SCRIPT_ID FINAL_RESULT_AFTER_REVAL
---------- -------------------------
         2                        78
         5                        72
         1                        82
         8                        89
        10                        79
```

```
SQL> -- Scripts flagged internally for recheck
SQL> SELECT ascr.Script_ID
  2  FROM ANSWER_SCRIPT ascr
  3  WHERE ascr.Is_Revaluation_Requested = 1;

 SCRIPT_ID
----------
         3
         7

SQL> -- Rank evaluators by performance rating
SQL> SELECT ew.Evaluator_ID,
  2         ew.Performance_Rating
  3  FROM EVALUATOR_WORKLOAD ew
  4  ORDER BY ew.Performance_Rating DESC;

EVALUATOR_ID PERFORMANCE_RATING
------------ ------------------
           8                4.8
           4                4.7
           6                4.6
           1                4.5
          10                 4.3
           2                4.2
           5                  4
           3                3.8
           7                3.5
           9                3.2

10 rows selected.

SQL> -- Identify inefficient evaluators
SQL> SELECT ew.Evaluator_ID,
```

```
  2       ew.Average_Marking_Time
  3  FROM EVALUATOR_WORKLOAD ew
  4  WHERE ew.Average_Marking_Time > 15;

EVALUATOR_ID AVERAGE_MARKING_TIME
------------ --------------------
           9                   16

SQL> -- Track assignment completion
SQL> SELECT ea.Assignment_ID,
  2       ea.Assignment_Status
  3  FROM EVALUATION_ASSIGNMENT ea;

ASSIGNMENT_ID ASSIGNMENT_STATUS
------------- --------------------------------------------------
            1 ASSIGNED
            2 ASSIGNED
            3 ASSIGNED
            4 ASSIGNED
            5 ASSIGNED
            6 ASSIGNED
            7 ASSIGNED
            8 ASSIGNED
            9 ASSIGNED
           10 ASSIGNED

10 rows selected.

SQL> -- Evaluations not finalized
SQL> SELECT er.Evaluation_ID,
  2       er.Assignment_ID
  3  FROM EVALUATION_RECORD er
  4  WHERE er.Is_Finalized = 0;
```

```
EVALUATION_ID ASSIGNMENT_ID
---------- ----------
         3          3
         7          7

SQL> -- System-flagged suspicious evaluations
SQL> SELECT er.Evaluation_ID,
  2       er.Marks_Awarded
  3  FROM EVALUATION_RECORD er
  4  WHERE er.Auto_Flag_For_Review = 1;

EVALUATION_ID MARKS_AWARDED
---------- ----------
         3         60
         7         55

SQL> -- Number of scripts per evaluator
SQL> SELECT ea.Evaluator_ID,
  2       COUNT(*) AS total_scripts
  3  FROM EVALUATION_ASSIGNMENT ea
  4  GROUP BY ea.Evaluator_ID;

EVALUATOR_ID TOTAL_SCRIPTS
---------- ----------
         1          1
         2          1
         3          1
         4          1
         5          1
         6          1
         7          1
         8          1
         9          1
        10          1
```

```
10 rows selected.

SQL> -- Evaluators giving unusually high marks
SQL> SELECT ea.Evaluator_ID,
  2       AVG(er.Marks_Awarded) AS avg_marks
  3  FROM EVALUATION_RECORD er
  4  JOIN EVALUATION_ASSIGNMENT ea
  5    ON er.Assignment_ID = ea.Assignment_ID
  6  GROUP BY ea.Evaluator_ID
  7  HAVING AVG(er.Marks_Awarded) > 85;

EVALUATOR_ID  AVG_MARKS
------------ ----------
         6         90
         8         88

SQL> -- Identify multiple evaluation rounds
SQL> SELECT ea.Script_ID,
  2       COUNT(*) AS evaluation_rounds
  3  FROM EVALUATION_ASSIGNMENT ea
  4  GROUP BY ea.Script_ID
  5  HAVING COUNT(*) > 1;

no rows selected

SQL> -- Check scripts encryption status
SQL> SELECT ascr.Script_ID,
  2       ascr.Encryption_Key_ID,
  3       ascr.Decryption_Status
  4  FROM ANSWER_SCRIPT ascr;

 SCRIPT_ID
----------
```

ENCRYPTION_KEY_ID

---

DECRYPTION_STATUS

---

1

KEY1

DECRYPTED

2

KEY2

DECRYPTED

SCRIPT_ID

---

ENCRYPTION_KEY_ID

---

DECRYPTION_STATUS

---

3

KEY3

DECRYPTED

4

KEY4

SCRIPT_ID

---

ENCRYPTION_KEY_ID

---

DECRYPTION_STATUS

---

DECRYPTED

5

KEY5

DECRYPTED

6

SCRIPT_ID

---

ENCRYPTION_KEY_ID

---

DECRYPTION_STATUS

---

KEY6

DECRYPTED

7

KEY7

DECRYPTED

SCRIPT_ID

---

ENCRYPTION_KEY_ID

---

DECRYPTION_STATUS

---

8

KEY8

DECRYPTED

9

KEY9

DECRYPTED

```
 SCRIPT_ID
----------
ENCRYPTION_KEY_ID
--------------------------------------------------------------------------------
DECRYPTION_STATUS
--------------------------------------------------

      10
KEY10
DECRYPTED



10 rows selected.


SQL> SPOOL off;
```

## IMPLEMENTATION OF MODULE 7 - RESULT PROCESSING MODULE

## (i) DESIGN AND IMPLEMENTATION OF ENTITIES WITH DATA INITIALIZATION:

**This module focuses on the creation and management of entities related to result processing, including the computation of final scores, generation of results, application of grading rules, and the insertion of essential data required to ensure accurate, consistent, and timely declaration of examination results within the system.**

```
SQL> CREATE TABLE RAW_MARKS (
  2    Script_ID NUMBER NOT NULL,
  3    Subject_ID NUMBER NOT NULL,
  4    Raw_Score NUMBER(8,2) NOT NULL,
  5    Max_Marks NUMBER(8,2) NOT NULL,
  6    Evaluation_Round NUMBER NOT NULL,
  7    Evaluation_Completed_At TIMESTAMP NOT NULL,
  8    Is_Finalized NUMBER(1) DEFAULT 0,
  9    Last_Updated TIMESTAMP NOT NULL,
 10     PRIMARY KEY (Script_ID, Subject_ID, Evaluation_Round),
 11     FOREIGN KEY (Script_ID) REFERENCES ANSWER_SCRIPT(Script_ID),
```

```
12     FOREIGN KEY (Subject_ID) REFERENCES SUBJECT(Subject_ID)
13  );
```

Table created.

```
SQL> CREATE TABLE RESULT_MODERATION (
 2     Moderation_ID NUMBER PRIMARY KEY,
 3     Subject_ID NUMBER NOT NULL,
 4     Exam_ID NUMBER NOT NULL,
 5     Moderation_Type VARCHAR2(50) NOT NULL,
 6     Scaling_Factor NUMBER(5,4) DEFAULT 1.0,
 7     Grace_Marks NUMBER(5,2) DEFAULT 0,
 8     Max_Cap_After_Scaling NUMBER(8,2),
 9     Moderation_Reason CLOB NOT NULL,
10     Approval_Date DATE NOT NULL,
11     Approved_By NUMBER NOT NULL,
12     FOREIGN KEY (Subject_ID) REFERENCES SUBJECT(Subject_ID),
13     FOREIGN KEY (Exam_ID) REFERENCES EXAM(Exam_ID)
14  );
```

Table created.

```
SQL> CREATE TABLE SUBJECT_RESULT (
 2     Subject_Result_ID NUMBER PRIMARY KEY,
 3     Registration_ID NUMBER NOT NULL,
 4     Subject_ID NUMBER NOT NULL,
 5     Script_ID NUMBER NOT NULL,
 6     Raw_Score NUMBER(8,2) NOT NULL,
 7     Moderation_ID NUMBER,
 8     Moderated_Score NUMBER(8,2),
 9     Grade VARCHAR2(5),
10     Pass_Flag NUMBER(1) NOT NULL,
11     Result_Computed_At TIMESTAMP NOT NULL,
12     FOREIGN KEY (Registration_ID) REFERENCES REGISTRATION(Registration_ID),
```

```
13    FOREIGN KEY (Subject_ID) REFERENCES SUBJECT(Subject_ID),
14    FOREIGN KEY (Script_ID) REFERENCES ANSWER_SCRIPT(Script_ID),
15    FOREIGN KEY (Moderation_ID) REFERENCES RESULT_MODERATION(Moderation_ID)
16  );
```

Table created.

```
SQL> CREATE TABLE FINAL_RESULT (
 2    Result_ID NUMBER PRIMARY KEY,
 3    Student_ID NUMBER NOT NULL,
 4    Exam_ID NUMBER NOT NULL,
 5    Total_Score NUMBER(10,2) NOT NULL,
 6    Percentile NUMBER(6,4),
 7    Rank NUMBER,
 8    Result_Status VARCHAR2(50) NOT NULL,
 9    Result_Published_At TIMESTAMP NOT NULL,
10    Result_Version NUMBER DEFAULT 1,
11    Is_Final NUMBER(1) DEFAULT 0,
12    FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
13    FOREIGN KEY (Exam_ID) REFERENCES EXAM(Exam_ID)
14  );
```

Table created.

```
SQL> CREATE TABLE RANKING_DISTRIBUTION (
 2    Rank_Distribution_ID NUMBER PRIMARY KEY,
 3    Exam_ID NUMBER NOT NULL,
 4    Score_Range_Min NUMBER(8,2) NOT NULL,
 5    Score_Range_Max NUMBER(8,2) NOT NULL,
 6    Rank_Start NUMBER NOT NULL,
 7    Rank_End NUMBER NOT NULL,
 8    Candidate_Count NUMBER NOT NULL,
 9    Percentile_Min NUMBER(6,4) NOT NULL,
10    Percentile_Max NUMBER(6,4) NOT NULL,
```

```
11     FOREIGN KEY (Exam_ID) REFERENCES EXAM(Exam_ID)
12  );
```

Table created.

```
SQL> CREATE TABLE RESULT_PUBLICATION_LOG (
 2     Publication_ID NUMBER PRIMARY KEY,
 3     Exam_ID NUMBER NOT NULL,
 4     Publication_Phase VARCHAR2(100) NOT NULL,
 5     Published_By NUMBER NOT NULL,
 6     Publication_Time TIMESTAMP NOT NULL,
 7     Total_Candidates NUMBER NOT NULL,
 8     Qualified_Candidates NUMBER NOT NULL,
 9     Publication_Status VARCHAR2(50) NOT NULL,
10     Notification_Status VARCHAR2(50) NOT NULL,
11     FOREIGN KEY (Exam_ID) REFERENCES EXAM(Exam_ID)
12  );
```

Table created.

```
SQL> INSERT INTO RAW_MARKS VALUES (1,1,80,100,1,SYSTIMESTAMP,1,SYSTIMESTAMP);
```

1 row created.

```
SQL> INSERT INTO RAW_MARKS VALUES (2,2,75,100,1,SYSTIMESTAMP,1,SYSTIMESTAMP);
```

1 row created.

```
SQL> INSERT INTO RAW_MARKS VALUES (3,3,60,100,1,SYSTIMESTAMP,1,SYSTIMESTAMP);
```

1 row created.

```
SQL> INSERT INTO RAW_MARKS VALUES (4,4,85,100,1,SYSTIMESTAMP,1,SYSTIMESTAMP);
```

1 row created.

SQL> INSERT INTO RAW_MARKS VALUES (5,5,70,100,1,SYSTIMESTAMP,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO RAW_MARKS VALUES (6,6,90,100,1,SYSTIMESTAMP,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO RAW_MARKS VALUES (7,7,55,100,1,SYSTIMESTAMP,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO RAW_MARKS VALUES (8,8,88,100,1,SYSTIMESTAMP,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO RAW_MARKS VALUES (9,9,65,100,1,SYSTIMESTAMP,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO RAW_MARKS VALUES (10,10,78,100,1,SYSTIMESTAMP,1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO RESULT_MODERATION VALUES (1,1,1,'SCALING',1.05,2,NULL,'Normalization',DATE '2026-05-20',101);

1 row created.

SQL> INSERT INTO RESULT_MODERATION VALUES (2,2,1,'GRACE',1.00,5,NULL,'Grace',DATE '2026-05-20',102);

1 row created.

```
SQL> INSERT INTO RESULT_MODERATION VALUES (3,3,1,'SCALING',1.02,0,NULL,'Adjustment',DATE '2026-05-
20',103);

1 row created.

SQL> INSERT INTO RESULT_MODERATION VALUES (4,4,1,'NONE',1.00,0,NULL,'None',DATE '2026-05-20',104);

1 row created.

SQL> INSERT INTO RESULT_MODERATION VALUES (5,5,1,'GRACE',1.00,3,NULL,'Borderline',DATE '2026-05-20',105);

1 row created.

SQL> INSERT INTO RESULT_MODERATION VALUES (6,6,1,'SCALING',1.04,0,NULL,'Scaling',DATE '2026-05-20',106);

1 row created.

SQL> INSERT INTO RESULT_MODERATION VALUES (7,7,1,'GRACE',1.00,4,NULL,'Low score',DATE '2026-05-20',107);

1 row created.

SQL> INSERT INTO RESULT_MODERATION VALUES (8,8,1,'NONE',1.00,0,NULL,'None',DATE '2026-05-20',108);

1 row created.

SQL> INSERT INTO RESULT_MODERATION VALUES (9,9,1,'SCALING',1.03,0,NULL,'Curve',DATE '2026-05-20',109);

1 row created.

SQL> INSERT INTO RESULT_MODERATION VALUES (10,10,1,'NONE',1.00,0,NULL,'None',DATE '2026-05-20',110);

1 row created.
```

```
SQL> INSERT INTO SUBJECT_RESULT VALUES (1,1,1,1,80,1,86,'A',1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SUBJECT_RESULT VALUES (2,2,2,2,75,2,80,'B',1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SUBJECT_RESULT VALUES (3,3,3,3,60,3,61,'C',1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SUBJECT_RESULT VALUES (4,4,4,4,85,4,85,'A',1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SUBJECT_RESULT VALUES (5,5,5,5,70,5,73,'B',1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SUBJECT_RESULT VALUES (6,6,6,6,90,6,93,'A',1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SUBJECT_RESULT VALUES (7,7,7,7,55,7,59,'C',0,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SUBJECT_RESULT VALUES (8,8,8,8,88,8,88,'A',1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO SUBJECT_RESULT VALUES (9,9,9,9,65,9,67,'B',1,SYSTIMESTAMP);
```

1 row created.

SQL> INSERT INTO SUBJECT_RESULT VALUES (10,10,10,10,78,10,78,'B',1,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO FINAL_RESULT VALUES (1,1,1,86,95.1234,1,'PASS',SYSTIMESTAMP,1,1);

1 row created.

SQL> INSERT INTO FINAL_RESULT VALUES (2,2,1,80,90.0000,2,'PASS',SYSTIMESTAMP,1,1);

1 row created.

SQL> INSERT INTO FINAL_RESULT VALUES (3,3,1,61,70.0000,5,'PASS',SYSTIMESTAMP,1,1);

1 row created.

SQL> INSERT INTO FINAL_RESULT VALUES (4,4,1,85,93.0000,3,'PASS',SYSTIMESTAMP,1,1);

1 row created.

SQL> INSERT INTO FINAL_RESULT VALUES (5,5,1,73,80.0000,4,'PASS',SYSTIMESTAMP,1,1);

1 row created.

SQL> INSERT INTO FINAL_RESULT VALUES (6,6,1,93,98.0000,1,'PASS',SYSTIMESTAMP,1,1);

1 row created.

SQL> INSERT INTO FINAL_RESULT VALUES (7,7,1,59,60.0000,8,'FAIL',SYSTIMESTAMP,1,1);

1 row created.

```
SQL> INSERT INTO FINAL_RESULT VALUES (8,8,1,88,96.0000,2,'PASS',SYSTIMESTAMP,1,1);

1 row created.

SQL> INSERT INTO FINAL_RESULT VALUES (9,9,1,67,75.0000,6,'PASS',SYSTIMESTAMP,1,1);

1 row created.

SQL> INSERT INTO FINAL_RESULT VALUES (10,10,1,78,85.0000,4,'PASS',SYSTIMESTAMP,1,1);

1 row created.

SQL> INSERT INTO RANKING_DISTRIBUTION VALUES (1,1,90,100,1,10,10,99.0000,100.0000);
INSERT INTO RANKING_DISTRIBUTION VALUES (1,1,90,100,1,10,10,99.0000,100.0000)
                                         *
ERROR at line 1:
ORA-01438: value larger than specified precision allowed for this column

SQL> INSERT INTO RANKING_DISTRIBUTION VALUES (2,1,80,89,11,50,40,95.0000,98.9999);

1 row created.

SQL> INSERT INTO RANKING_DISTRIBUTION VALUES (3,1,70,79,51,100,50,90.0000,94.9999);

1 row created.

SQL> INSERT INTO RANKING_DISTRIBUTION VALUES (4,1,60,69,101,200,100,80.0000,89.9999);

1 row created.

SQL> INSERT INTO RANKING_DISTRIBUTION VALUES (5,1,50,59,201,300,100,70.0000,79.9999);

1 row created.
```

```
SQL> INSERT INTO RANKING_DISTRIBUTION VALUES (6,1,40,49,301,400,100,60.0000,69.9999);

1 row created.

SQL> INSERT INTO RANKING_DISTRIBUTION VALUES (7,1,30,39,401,500,100,50.0000,59.9999);

1 row created.

SQL> INSERT INTO RANKING_DISTRIBUTION VALUES (8,1,20,29,501,600,100,40.0000,49.9999);

1 row created.

SQL> INSERT INTO RANKING_DISTRIBUTION VALUES (9,1,10,19,601,700,100,30.0000,39.9999);

1 row created.

SQL> INSERT INTO RANKING_DISTRIBUTION VALUES (10,1,0,9,701,800,100,0.0000,29.9999);

1 row created.

SQL> INSERT INTO RESULT_PUBLICATION_LOG VALUES
(1,1,'PHASE1',101,SYSTIMESTAMP,1000,800,'SUCCESS','SENT');

1 row created.

SQL> INSERT INTO RESULT_PUBLICATION_LOG VALUES
(2,1,'PHASE2',102,SYSTIMESTAMP,1000,820,'SUCCESS','SENT');

1 row created.

SQL> INSERT INTO RESULT_PUBLICATION_LOG VALUES (3,1,'FINAL',103,SYSTIMESTAMP,1000,850,'SUCCESS','SENT');

1 row created.
```

```
SQL> INSERT INTO RESULT_PUBLICATION_LOG VALUES
(4,1,'REVISED',104,SYSTIMESTAMP,1000,860,'SUCCESS','SENT');


1 row created.


SQL> INSERT INTO RESULT_PUBLICATION_LOG VALUES
(5,1,'BACKUP',105,SYSTIMESTAMP,1000,850,'SUCCESS','SENT');


1 row created.


SQL> INSERT INTO RESULT_PUBLICATION_LOG VALUES
(6,1,'PHASE3',106,SYSTIMESTAMP,1000,870,'SUCCESS','SENT');


1 row created.


SQL> INSERT INTO RESULT_PUBLICATION_LOG VALUES
(7,1,'PHASE4',107,SYSTIMESTAMP,1000,880,'SUCCESS','SENT');


1 row created.


SQL> INSERT INTO RESULT_PUBLICATION_LOG VALUES
(8,1,'PHASE5',108,SYSTIMESTAMP,1000,890,'SUCCESS','SENT');


1 row created.


SQL> INSERT INTO RESULT_PUBLICATION_LOG VALUES
(9,1,'PHASE6',109,SYSTIMESTAMP,1000,900,'SUCCESS','SENT');


1 row created.


SQL> INSERT INTO RESULT_PUBLICATION_LOG VALUES
(10,1,'FINAL_LOCK',110,SYSTIMESTAMP,1000,900,'SUCCESS','SENT');


1 row created.
```

## APPLIED SQL QUERIES AND BUSINESS USE CASES – RESULT PROCESSING MODULE:

1. **Subject Wise Result Details for a Student**
2. **Final Result Summary of a Student**
3. **Failed Students List**
4. **Top Rankers in Exam**
5. **Students with Distinction Performance**
6. **Subject Wise Pass Percentage Analysis**
7. **Moderation Impact Analysis**
8. **Subjects with Highest Moderation Increase**
9. **Students Near Pass Boundary**
10. **Rank Distribution Overview**
11. **Result Status Count Analysis**
12. **Students Without Finalized Results**
13. **Average Score Per Exam**
14. **Result Publication Audit Log**
15. **Qualification Rate Analysis**
16. **Abnormal Moderation Detection**
17. **Consistent High Performing Students**
18. **Students Passed in All Subjects**
19. **Students Failed in At Least One Subject**
20. **Raw Score vs Final Score Comparison**

```
SQL> -- View all subject results of a student
SQL> SELECT sr.Registration_ID,
  2     sr.Subject_ID,
  3     sr.Raw_Score,
  4     sr.Moderated_Score,
  5     sr.Grade,
  6     sr.Pass_Flag
```

```
  7  FROM SUBJECT_RESULT sr
  8  WHERE sr.Registration_ID = 1;

REGISTRATION_ID SUBJECT_ID  RAW_SCORE MODERATED_SCORE GRADE  PASS_FLAG
--------------- ---------- ---------- --------------- ----- ----------
              1          1         80            86 A            1

SQL> -- Final result summary of a student
SQL> SELECT fr.Student_ID,
  2         fr.Exam_ID,
  3         fr.Total_Score,
  4         fr.Percentile,
  5         fr.Rank,
  6         fr.Result_Status
  7  FROM FINAL_RESULT fr
  8  WHERE fr.Student_ID = 1;

STUDENT_ID    EXAM_ID TOTAL_SCORE PERCENTILE      RANK
---------- ---------- ----------- ---------- ----------
RESULT_STATUS
--------------------------------------------------
         1          1          86   95.1234         1
PASS


SQL> -- Students who failed in final result
SQL> SELECT fr.Student_ID,
  2         fr.Total_Score
  3  FROM FINAL_RESULT fr
  4  WHERE fr.Result_Status = 'FAIL';

STUDENT_ID TOTAL_SCORE
---------- -----------
         7          59
```

```
SQL> -- Top 10 rank holders
SQL> SELECT fr.Student_ID,
  2       fr.Total_Score,
  3       fr.Rank
  4  FROM FINAL_RESULT fr
  5  WHERE fr.Exam_ID = 1
  6  ORDER BY fr.Rank ASC
  7  FETCH FIRST 10 ROWS ONLY;

STUDENT_ID TOTAL_SCORE      RANK
---------- ----------- ----------
         1          86          1
         6          93          1
         2          80          2
         8          88          2
         4          85          3
         5          73          4
        10          78          4
         3          61          5
         9          67          6
         7          59          8

10 rows selected.

SQL> -- High performers
SQL> SELECT fr.Student_ID,
  2       fr.Total_Score,
  3       fr.Percentile
  4  FROM FINAL_RESULT fr
  5  WHERE fr.Total_Score >= 85;

STUDENT_ID TOTAL_SCORE PERCENTILE
---------- ----------- ----------
```

```
      1        86   95.1234
      4        85       93
      6        93       98
      8        88       96

SQL> -- Pass percentage per subject
SQL> SELECT sr.Subject_ID,
  2       COUNT(*) AS total_students,
  3       SUM(CASE WHEN sr.Pass_Flag = 1 THEN 1 ELSE 0 END) AS passed,
  4       ROUND(
  5         (SUM(CASE WHEN sr.Pass_Flag = 1 THEN 1 ELSE 0 END) * 100.0)
  6         / COUNT(*), 2
  7       ) AS pass_percentage
  8  FROM SUBJECT_RESULT sr
  9  GROUP BY sr.Subject_ID;

SUBJECT_ID TOTAL_STUDENTS     PASSED PASS_PERCENTAGE
---------- -------------- ---------- ---------------
      1           1        1          100
      2           1        1          100
      3           1        1          100
      4           1        1          100
      5           1        1          100
      6           1        1          100
      7           1        0            0
      8           1        1          100
      9           1        1          100
     10           1        1          100

10 rows selected.

SQL> -- Check moderation effect
SQL> SELECT sr.Subject_ID,
  2       sr.Raw_Score,
```

```
  3       sr.Moderated_Score,
  4       (sr.Moderated_Score - sr.Raw_Score) AS increase
  5  FROM SUBJECT_RESULT sr
  6  WHERE sr.Moderation_ID IS NOT NULL;

SUBJECT_ID  RAW_SCORE MODERATED_SCORE   INCREASE
---------- ---------- --------------- ----------
         1         80              86          6
         2         75              80          5
         3         60              61          1
         4         85              85          0
         5         70              73          3
         6         90              93          3
         7         55              59          4
         8         88              88          0
         9         65              67          2
        10         78              78          0

10 rows selected.

SQL> -- Subjects where moderation significantly increased marks
SQL> SELECT sr.Subject_ID,
  2       AVG(sr.Moderated_Score - sr.Raw_Score) AS avg_increase
  3  FROM SUBJECT_RESULT sr
  4  WHERE sr.Moderation_ID IS NOT NULL
  5  GROUP BY sr.Subject_ID
  6  ORDER BY avg_increase DESC;

SUBJECT_ID AVG_INCREASE
---------- ------------
         1            6
         2            5
         7            4
         5            3
```

```
       6          3

       9          2

       3          1

       8          0

      10           0

       4          0


10 rows selected.


SQL> -- Students close to failing

SQL> SELECT sr.Registration_ID,

  2       sr.Subject_ID,

  3       sr.Moderated_Score

  4  FROM SUBJECT_RESULT sr

  5  WHERE sr.Moderated_Score BETWEEN 30 AND 40;


no rows selected


SQL> -- View rank brackets

SQL> SELECT rd.Score_Range_Min,

  2       rd.Score_Range_Max,

  3       rd.Rank_Start,

  4       rd.Rank_End,

  5       rd.Candidate_Count

  6  FROM RANKING_DISTRIBUTION rd

  7  WHERE rd.Exam_ID = 1;


SCORE_RANGE_MIN SCORE_RANGE_MAX RANK_START   RANK_END CANDIDATE_COUNT

--------------- --------------- ---------- ---------- ---------------

          80          89         11         50             40

          70          79         51        100             50

          60          69        101        200            100

          50          59        201        300            100

          40          49        301        400            100
```

```
        30         39     401     500         100
        20         29     501     600         100
        10         19     601     700         100
         0          9     701     800         100

9 rows selected.

SQL> -- PASS vs FAIL count
SQL> SELECT fr.Result_Status,
  2       COUNT(*) AS total
  3  FROM FINAL_RESULT fr
  4  GROUP BY fr.Result_Status;

RESULT_STATUS                                TOTAL
-------------------------------------------------- ----------
PASS                                              9
FAIL                                              1

SQL> -- Results not finalized yet
SQL> SELECT fr.Student_ID,
  2       fr.Exam_ID
  3  FROM FINAL_RESULT fr
  4  WHERE fr.Is_Final = 0;

no rows selected

SQL> -- Overall exam performance
SQL> SELECT fr.Exam_ID,
  2       AVG(fr.Total_Score) AS avg_score
  3  FROM FINAL_RESULT fr
  4  GROUP BY fr.Exam_ID;

   EXAM_ID  AVG_SCORE
---------- ----------
```

```
      1       77

SQL> -- Track result publication history
SQL> SELECT rpl.Exam_ID,
  2       rpl.Publication_Phase,
  3       rpl.Publication_Time,
  4       rpl.Total_Candidates,
  5       rpl.Qualified_Candidates
  6  FROM RESULT_PUBLICATION_LOG rpl
  7  ORDER BY rpl.Publication_Time DESC;

   EXAM_ID
----------
PUBLICATION_PHASE
--------------------------------------------------------------------------------
PUBLICATION_TIME
---------------------------------------------------------------------------
TOTAL_CANDIDATES QUALIFIED_CANDIDATES
---------------- --------------------
      1
FINAL_LOCK
14-APR-26 05.42.27.183000 PM
         1000             900


   EXAM_ID
----------
PUBLICATION_PHASE
--------------------------------------------------------------------------------
PUBLICATION_TIME
---------------------------------------------------------------------------
TOTAL_CANDIDATES QUALIFIED_CANDIDATES
---------------- --------------------
      1
```

PHASE6

14-APR-26 05.42.26.612000 PM

1000 900

EXAM_ID

---

PUBLICATION_PHASE

---

PUBLICATION_TIME

---

TOTAL_CANDIDATES QUALIFIED_CANDIDATES

--- ---

1

PHASE5

14-APR-26 05.42.26.608000 PM

1000 890

EXAM_ID

---

PUBLICATION_PHASE

---

PUBLICATION_TIME

---

TOTAL_CANDIDATES QUALIFIED_CANDIDATES

--- ---

1

PHASE4

14-APR-26 05.42.26.603000 PM

1000 880

EXAM_ID

---

PUBLICATION_PHASE

---

PUBLICATION_TIME

---

TOTAL_CANDIDATES QUALIFIED_CANDIDATES

---- ----

1

PHASE3

14-APR-26 05.42.26.599000 PM

1000 870

EXAM_ID

---

PUBLICATION_PHASE

---

PUBLICATION_TIME

---

TOTAL_CANDIDATES QUALIFIED_CANDIDATES

---- ----

1

BACKUP

14-APR-26 05.42.26.595000 PM

1000 850

EXAM_ID

---

PUBLICATION_PHASE

---

PUBLICATION_TIME

---

TOTAL_CANDIDATES QUALIFIED_CANDIDATES

---------- ----------

1

REVISED

14-APR-26 05.42.26.590000 PM

1000 860

EXAM_ID

---

PUBLICATION_PHASE

---

PUBLICATION_TIME

---

TOTAL_CANDIDATES QUALIFIED_CANDIDATES

---------- ----------

1

FINAL

14-APR-26 05.42.26.586000 PM

1000 850

EXAM_ID

---

PUBLICATION_PHASE

---

PUBLICATION_TIME

---

TOTAL_CANDIDATES QUALIFIED_CANDIDATES

---------- ----------

1

PHASE2

14-APR-26 05.42.26.581000 PM

1000 820

```
  EXAM_ID
----------
PUBLICATION_PHASE
--------------------------------------------------------------------------------
PUBLICATION_TIME
---------------------------------------------------------------------------
TOTAL_CANDIDATES QUALIFIED_CANDIDATES
---------------- --------------------
       1
PHASE1
14-APR-26 05.42.26.571000 PM
         1000              800



10 rows selected.


SQL> -- Percentage of qualified candidates
SQL> SELECT rpl.Exam_ID,
  2       ROUND((rpl.Qualified_Candidates * 100.0) / rpl.Total_Candidates, 2) AS qualification_rate
  3  FROM RESULT_PUBLICATION_LOG rpl;

  EXAM_ID QUALIFICATION_RATE
---------- ------------------
       1              80
       1              82
       1              85
       1              86
       1              85
       1              87
       1              88
       1              89
       1              90
       1              90
```

```
10 rows selected.

SQL> -- Suspicious moderation cases
SQL> SELECT sr.Registration_ID,
  2       sr.Subject_ID,
  3       sr.Raw_Score,
  4       sr.Moderated_Score
  5  FROM SUBJECT_RESULT sr
  6  WHERE (sr.Moderated_Score - sr.Raw_Score) > 20;

no rows selected

SQL> -- Students scoring high across subjects
SQL> SELECT sr.Registration_ID
  2  FROM SUBJECT_RESULT sr
  3  GROUP BY sr.Registration_ID
  4  HAVING AVG(sr.Moderated_Score) > 80;

REGISTRATION_ID
---------------
             1
             4
             6
             8

SQL> -- No failures across subjects
SQL> SELECT sr.Registration_ID
  2  FROM SUBJECT_RESULT sr
  3  GROUP BY sr.Registration_ID
  4  HAVING MIN(sr.Pass_Flag) = 1;

REGISTRATION_ID
---------------
```

```
         1
         2
         3
         4
         5
         6
         8
         9
        10

9 rows selected.

SQL> -- At least one subject failed
SQL> SELECT sr.Registration_ID
  2  FROM SUBJECT_RESULT sr
  3  GROUP BY sr.Registration_ID
  4  HAVING MIN(sr.Pass_Flag) = 0;

REGISTRATION_ID
---------------
         7

SQL> -- Compare raw vs final total
SQL> SELECT fr.Student_ID,
  2       fr.Total_Score,
  3       SUM(sr.Raw_Score) AS total_raw
  4  FROM FINAL_RESULT fr
  5  JOIN SUBJECT_RESULT sr
  6    ON fr.Student_ID = sr.Registration_ID
  7  GROUP BY fr.Student_ID, fr.Total_Score;

STUDENT_ID TOTAL_SCORE  TOTAL_RAW
---------- ----------- ----------
       1        86        80
```

```
 2      80      75
 3      61      60
 4      85      85
 5      73      70
 6      93      90
 7      59      55
 8      88      88
 9      67      65
10      78      78

10 rows selected.

SQL> SPOOL OFF;
```

## IMPLEMENTATION OF MODULE 9 - GRIEVANCE HANDLING MODULE

## (i) DESIGN AND IMPLEMENTATION OF ENTITIES WITH DATA INITIALIZATION:

This module focuses on the creation and management of grievance handling entities, including the submission of student grievances, classification and prioritization of complaints, assignment of grievances to responsible authorities, tracking of resolution status, and the insertion of essential data required to ensure transparent, efficient, and timely resolution of issues within the examination system.

```
SQL> CREATE TABLE GRIEVANCE (
  2  Grievance_ID NUMBER PRIMARY KEY,
  3  Student_ID NUMBER NOT NULL,
  4  Exam_ID NUMBER NOT NULL,
  5  Subject_ID NUMBER,
  6  Grievance_Type VARCHAR2(100) NOT NULL,
  7  Description CLOB NOT NULL,
  8  Evidence_Summary CLOB,
  9  Priority_Level VARCHAR2(50) NOT NULL,
 10  Filed_Channel VARCHAR2(50) NOT NULL,
 11  Filed_Date DATE NOT NULL,
 12  Status VARCHAR2(50) NOT NULL,
```

```
13     Last_Updated TIMESTAMP NOT NULL,
14     Resolution_Deadline DATE,
15     FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
16     FOREIGN KEY (Exam_ID) REFERENCES EXAM(Exam_ID),
17     FOREIGN KEY (Subject_ID) REFERENCES SUBJECT(Subject_ID)
18  );
```

Table created.

```
SQL> CREATE TABLE RESULT_GRIEVANCE (
 2     Grievance_ID NUMBER PRIMARY KEY,
 3     Result_ID NUMBER NOT NULL,
 4     FOREIGN KEY (Grievance_ID) REFERENCES GRIEVANCE(Grievance_ID),
 5     FOREIGN KEY (Result_ID) REFERENCES FINAL_RESULT(Result_ID)
 6  );
```

Table created.

```
SQL> CREATE TABLE SUBJECT_RESULT_GRIEVANCE (
 2     Grievance_ID NUMBER PRIMARY KEY,
 3     Subject_Result_ID NUMBER NOT NULL,
 4     FOREIGN KEY (Grievance_ID) REFERENCES GRIEVANCE(Grievance_ID),
 5     FOREIGN KEY (Subject_Result_ID) REFERENCES SUBJECT_RESULT(Subject_Result_ID)
 6  );
```

Table created.

```
SQL> CREATE TABLE SCRIPT_GRIEVANCE (
 2     Grievance_ID NUMBER PRIMARY KEY,
 3     Script_ID NUMBER NOT NULL,
 4     FOREIGN KEY (Grievance_ID) REFERENCES GRIEVANCE(Grievance_ID),
 5     FOREIGN KEY (Script_ID) REFERENCES ANSWER_SCRIPT(Script_ID)
 6  );
```

```
Table created.

SQL> CREATE TABLE MALPRACTICE_GRIEVANCE (
  2    Grievance_ID NUMBER PRIMARY KEY,
  3    Report_ID NUMBER NOT NULL,
  4    FOREIGN KEY (Grievance_ID) REFERENCES GRIEVANCE(Grievance_ID)
  5  );

Table created.

SQL> CREATE TABLE GRIEVANCE_EVIDENCE (
  2    Evidence_ID NUMBER PRIMARY KEY,
  3    Grievance_ID NUMBER NOT NULL,
  4    Document_Type VARCHAR2(100) NOT NULL,
  5    File_Path VARCHAR2(500) NOT NULL,
  6    File_Hash VARCHAR2(256) NOT NULL,
  7    Uploaded_By NUMBER NOT NULL,
  8    Upload_Timestamp TIMESTAMP NOT NULL,
  9    File_Size NUMBER NOT NULL,
 10    File_Format VARCHAR2(20) NOT NULL,
 11    Verification_Status VARCHAR2(50) NOT NULL,
 12    FOREIGN KEY (Grievance_ID) REFERENCES GRIEVANCE(Grievance_ID)
 13  );

Table created.

SQL> CREATE TABLE GRIEVANCE_ACTION_LOG (
  2    Action_ID NUMBER PRIMARY KEY,
  3    Grievance_ID NUMBER NOT NULL,
  4    Action_Type VARCHAR2(100) NOT NULL,
  5    Action_Description CLOB NOT NULL,
  6    Old_Status VARCHAR2(50) NOT NULL,
  7    New_Status VARCHAR2(50) NOT NULL,
  8    Performed_By NUMBER NOT NULL,
```

```
  9   Officer_Role VARCHAR2(100) NOT NULL,
 10   Action_Timestamp TIMESTAMP NOT NULL,
 11   Remarks CLOB,
 12   FOREIGN KEY (Grievance_ID) REFERENCES GRIEVANCE(Grievance_ID)
 13 );
```

Table created.

```
SQL> CREATE TABLE GRIEVANCE_REVIEW_PANEL (
  2   Panel_ID NUMBER PRIMARY KEY,
  3   Grievance_ID NUMBER NOT NULL,
  4   Panel_Creation_Date DATE NOT NULL,
  5   Panel_Status VARCHAR2(50) NOT NULL,
  6   Chairperson_ID NUMBER NOT NULL,
  7   Total_Members NUMBER NOT NULL,
  8   Meeting_Date DATE,
  9   Decision CLOB,
 10   Decision_Date DATE,
 11   Decision_Document_Path VARCHAR2(500),
 12   FOREIGN KEY (Grievance_ID) REFERENCES GRIEVANCE(Grievance_ID)
 13 );
```

Table created.

```
SQL> CREATE TABLE GRIEVANCE_RESOLUTION (
  2   Resolution_ID NUMBER PRIMARY KEY,
  3   Grievance_ID NUMBER NOT NULL,
  4   Resolution_Type VARCHAR2(100) NOT NULL,
  5   Old_Value CLOB,
  6   New_Value CLOB,
  7   Approved_By NUMBER NOT NULL,
  8   Approval_Date DATE NOT NULL,
  9   Resolution_Status VARCHAR2(50) NOT NULL,
 10   Resolution_Notes CLOB,
```

```
11     FOREIGN KEY (Grievance_ID) REFERENCES GRIEVANCE(Grievance_ID)
12  );
```

Table created.

```
SQL> INSERT INTO GRIEVANCE VALUES (1,1,1,1,'RESULT','Marks mismatch',NULL,'HIGH','ONLINE',DATE '2026-06-01','OPEN',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO GRIEVANCE VALUES (2,2,1,2,'SCRIPT','Unreadable script',NULL,'MEDIUM','PORTAL',DATE '2026-06-02','OPEN',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO GRIEVANCE VALUES (3,3,1,3,'MALPRACTICE','Wrong accusation',NULL,'HIGH','EMAIL',DATE '2026-06-03','OPEN',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO GRIEVANCE VALUES (4,4,1,4,'RESULT','Incorrect grade',NULL,'LOW','ONLINE',DATE '2026-06-04','OPEN',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO GRIEVANCE VALUES (5,5,1,5,'SCRIPT','Missing pages',NULL,'HIGH','PORTAL',DATE '2026-06-05','OPEN',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO GRIEVANCE VALUES (6,6,1,6,'RESULT','Wrong total',NULL,'HIGH','ONLINE',DATE '2026-06-06','OPEN',SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO GRIEVANCE VALUES (7,7,1,7,'SCRIPT','Blurred scan',NULL,'MEDIUM','EMAIL',DATE '2026-06-07','OPEN',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE VALUES (8,8,1,8,'RESULT','Low marks',NULL,'LOW','ONLINE',DATE '2026-06-08','OPEN',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE VALUES (9,9,1,9,'RESULT','Not evaluated',NULL,'HIGH','PORTAL',DATE '2026-06-09','OPEN',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE VALUES (10,10,1,10,'SCRIPT','Wrong script assigned',NULL,'HIGH','EMAIL',DATE '2026-06-10','OPEN',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO RESULT_GRIEVANCE VALUES (1,1);

1 row created.

SQL> INSERT INTO RESULT_GRIEVANCE VALUES (4,4);

1 row created.

SQL> INSERT INTO RESULT_GRIEVANCE VALUES (6,6);

1 row created.

SQL> INSERT INTO RESULT_GRIEVANCE VALUES (8,8);

1 row created.
```

```
SQL> INSERT INTO RESULT_GRIEVANCE VALUES (9,9);

1 row created.

SQL> INSERT INTO SUBJECT_RESULT_GRIEVANCE VALUES (2,2);

1 row created.

SQL> INSERT INTO SUBJECT_RESULT_GRIEVANCE VALUES (5,5);

1 row created.

SQL> INSERT INTO SCRIPT_GRIEVANCE VALUES (3,3);

1 row created.

SQL> INSERT INTO SCRIPT_GRIEVANCE VALUES (7,7);

1 row created.

SQL> INSERT INTO SCRIPT_GRIEVANCE VALUES (10,10);

1 row created.

SQL> INSERT INTO GRIEVANCE_EVIDENCE VALUES
(1,1,'PDF','/e1.pdf','hash1',101,SYSTIMESTAMP,2048,'PDF','VERIFIED');

1 row created.

SQL> INSERT INTO GRIEVANCE_EVIDENCE VALUES
(2,2,'IMG','/e2.jpg','hash2',102,SYSTIMESTAMP,1024,'JPG','PENDING');

1 row created.
```

```
SQL> INSERT INTO GRIEVANCE_EVIDENCE VALUES
(3,3,'PDF','/e3.pdf','hash3',103,SYSTIMESTAMP,2048,'PDF','VERIFIED');

1 row created.

SQL> INSERT INTO GRIEVANCE_EVIDENCE VALUES
(4,4,'PDF','/e4.pdf','hash4',104,SYSTIMESTAMP,2048,'PDF','VERIFIED');

1 row created.

SQL> INSERT INTO GRIEVANCE_EVIDENCE VALUES
(5,5,'IMG','/e5.jpg','hash5',105,SYSTIMESTAMP,1024,'JPG','PENDING');

1 row created.

SQL> INSERT INTO GRIEVANCE_EVIDENCE VALUES
(6,6,'PDF','/e6.pdf','hash6',106,SYSTIMESTAMP,2048,'PDF','VERIFIED');

1 row created.

SQL> INSERT INTO GRIEVANCE_EVIDENCE VALUES
(7,7,'IMG','/e7.jpg','hash7',107,SYSTIMESTAMP,1024,'JPG','PENDING');

1 row created.

SQL> INSERT INTO GRIEVANCE_EVIDENCE VALUES
(8,8,'PDF','/e8.pdf','hash8',108,SYSTIMESTAMP,2048,'PDF','VERIFIED');

1 row created.

SQL> INSERT INTO GRIEVANCE_EVIDENCE VALUES
(9,9,'PDF','/e9.pdf','hash9',109,SYSTIMESTAMP,2048,'PDF','VERIFIED');

1 row created.
```

```
SQL> INSERT INTO GRIEVANCE_EVIDENCE VALUES
(10,10,'IMG','/e10.jpg','hash10',110,SYSTIMESTAMP,1024,'JPG','PENDING');

1 row created.

SQL> INSERT INTO GRIEVANCE_ACTION_LOG VALUES (1,1,'REVIEW','Checked
marks','OPEN','IN_PROGRESS',201,'OFFICER',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_ACTION_LOG VALUES (2,2,'VERIFY','Checked
script','OPEN','IN_PROGRESS',202,'OFFICER',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_ACTION_LOG VALUES (3,3,'INVESTIGATE','Case
review','OPEN','IN_PROGRESS',203,'OFFICER',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_ACTION_LOG VALUES (4,4,'UPDATE','Grade
check','OPEN','RESOLVED',204,'OFFICER',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_ACTION_LOG VALUES (5,5,'REVIEW','Script
missing','OPEN','IN_PROGRESS',205,'OFFICER',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_ACTION_LOG VALUES (6,6,'VERIFY','Total
recheck','OPEN','RESOLVED',206,'OFFICER',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_ACTION_LOG VALUES (7,7,'REVIEW','Scan
check','OPEN','IN_PROGRESS',207,'OFFICER',SYSTIMESTAMP,NULL);
```

1 row created.

SQL> INSERT INTO GRIEVANCE_ACTION_LOG VALUES (8,8,'UPDATE','Marks check','OPEN','RESOLVED',208,'OFFICER',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_ACTION_LOG VALUES (9,9,'VERIFY','Evaluation check','OPEN','IN_PROGRESS',209,'OFFICER',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_ACTION_LOG VALUES (10,10,'REVIEW','Script mismatch','OPEN','IN_PROGRESS',210,'OFFICER',SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_REVIEW_PANEL VALUES (1,1,DATE '2026-06-10','ACTIVE',301,3,NULL,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_REVIEW_PANEL VALUES (2,3,DATE '2026-06-11','ACTIVE',302,4,NULL,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_REVIEW_PANEL VALUES (3,5,DATE '2026-06-12','ACTIVE',303,3,NULL,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_REVIEW_PANEL VALUES (4,7,DATE '2026-06-13','ACTIVE',304,3,NULL,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_REVIEW_PANEL VALUES (5,10,DATE '2026-06-14','ACTIVE',305,5,NULL,NULL,NULL,NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_RESOLUTION VALUES (1,4,'GRADE_UPDATE','B','A',401,DATE '2026-06-20','RESOLVED',NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_RESOLUTION VALUES (2,6,'SCORE_CORRECTION','80','85',402,DATE '2026-06-20','RESOLVED',NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_RESOLUTION VALUES (3,8,'RECHECK','70','75',403,DATE '2026-06-20','RESOLVED',NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_RESOLUTION VALUES (4,1,'RECHECK','86','88',404,DATE '2026-06-20','RESOLVED',NULL);

1 row created.

SQL> INSERT INTO GRIEVANCE_RESOLUTION VALUES (5,2,'SCRIPT_FIX',NULL,NULL,405,DATE '2026-06-20','IN_PROGRESS',NULL);

1 row created.

## (ii) APPLIED SQL QUERIES AND BUSINESS USE CASES – GRIEVANCE HANDLING MODULE

### 1. List All Grievances of a Student

**2. Open Grievances Identification**

**3. High Priority Unresolved Grievances Detection**

**4. SLA Breach Detection for Grievances**

**5. Average Grievance Resolution Time**

**6. Grievance Count by Type**

**7. Grievance Count by Status**

**8. Students with Multiple Grievances**

**9. Result Linked Grievances**

**10. Script Linked Grievances**

**11. Evidence Verification Status Check**

**12. Unverified Evidence Detection**

**13. Grievance Action History Tracking**

**14. Grievances Without Action Taken**

**15. Active Review Panel Details**

**16. Grievances Resolved with Changes**

**17. Grievance Resolution Success Rate**

**18. Pending Grievances Per Exam**

**19. Grievance Officer Workload Analysis**

**20. Malpractice Related Grievance Detection**

```
SQL> -- View all grievances filed by a student
SQL> SELECT g.Grievance_ID,
  2       g.Exam_ID,
  3       g.Subject_ID,
  4       g.Grievance_Type,
  5       g.Status,
  6       g.Filed_Date
  7  FROM GRIEVANCE g
  8  WHERE g.Student_ID = 1;

GRIEVANCE_ID    EXAM_ID SUBJECT_ID
```

```
------------ ---------- ----------
GRIEVANCE_TYPE
--------------------------------------------------------------------------------
STATUS                                FILED_DAT
-------------------------------------------------- ---------
         1        1        1
RESULT
OPEN                                  01-JUN-26


SQL> -- Pending grievances
SQL> SELECT g.Grievance_ID,
  2       g.Student_ID,
  3       g.Priority_Level
  4  FROM GRIEVANCE g
  5  WHERE g.Status = 'OPEN';

GRIEVANCE_ID STUDENT_ID PRIORITY_LEVEL
------------ ---------- ----------------------------------------------------
         1        1 HIGH
         2        2 MEDIUM
         3        3 HIGH
         4        4 LOW
         5        5 HIGH
         6        6 HIGH
         7        7 MEDIUM
         8        8 LOW
         9        9 HIGH
        10       10 HIGH

10 rows selected.

SQL> -- Urgent cases needing attention
SQL> SELECT g.Grievance_ID,
```

```
  2       g.Student_ID,
  3       g.Priority_Level,
  4       g.Status
  5  FROM GRIEVANCE g
  6  WHERE g.Priority_Level = 'HIGH'
  7    AND g.Status <> 'RESOLVED';

GRIEVANCE_ID STUDENT_ID PRIORITY_LEVEL
------------ ---------- --------------------------------------------------
STATUS
--------------------------------------------------
         1          1 HIGH
OPEN

         3          3 HIGH
OPEN

         5          5 HIGH
OPEN


GRIEVANCE_ID STUDENT_ID PRIORITY_LEVEL
------------ ---------- --------------------------------------------------
STATUS
--------------------------------------------------
         6          6 HIGH
OPEN

         9          9 HIGH
OPEN

        10         10 HIGH
OPEN
```

```
6 rows selected.

SQL> -- Grievances past resolution deadline
SQL> SELECT g.Grievance_ID,
  2       g.Student_ID,
  3       g.Resolution_Deadline
  4  FROM GRIEVANCE g
  5  WHERE g.Resolution_Deadline IS NOT NULL
  6    AND g.Resolution_Deadline < SYSDATE
  7    AND g.Status <> 'RESOLVED';

no rows selected

SQL> -- Time taken to resolve grievances
SQL> SELECT AVG(gr.Approval_Date - g.Filed_Date) AS avg_days
  2  FROM GRIEVANCE g
  3  JOIN GRIEVANCE_RESOLUTION gr
  4    ON g.Grievance_ID = gr.Grievance_ID
  5  WHERE gr.Resolution_Status = 'RESOLVED';

  AVG_DAYS
----------
    15.25

SQL> -- Distribution of grievance types
SQL> SELECT g.Grievance_Type,
  2       COUNT(*) AS total
  3  FROM GRIEVANCE g
  4  GROUP BY g.Grievance_Type;

GRIEVANCE_TYPE
--------------------------------------------------------------------------------
    TOTAL
```

```
----------
RESULT
       5

SCRIPT
       4

MALPRACTICE
       1


SQL> -- OPEN vs RESOLVED vs IN_PROGRESS
SQL> SELECT g.Status,
  2       COUNT(*) AS total
  3  FROM GRIEVANCE g
  4  GROUP BY g.Status;

STATUS                                   TOTAL
-------------------------------------------------- ----------
OPEN                                        10

SQL> -- Identify students repeatedly filing complaints
SQL> SELECT g.Student_ID,
  2       COUNT(*) AS grievance_count
  3  FROM GRIEVANCE g
  4  GROUP BY g.Student_ID
  5  HAVING COUNT(*) > 1;

no rows selected

SQL> -- Result-related complaints
SQL> SELECT rg.Grievance_ID,
  2       rg.Result_ID
  3  FROM RESULT_GRIEVANCE rg;
```

```
GRIEVANCE_ID  RESULT_ID
------------ ----------
         1         1
         4         4
         6         6
         8         8
         9         9

SQL> -- Script-level issues
SQL> SELECT sg.Grievance_ID,
  2       sg.Script_ID
  3  FROM SCRIPT_GRIEVANCE sg;

GRIEVANCE_ID  SCRIPT_ID
------------ ----------
         3         3
         7         7
        10        10

SQL> -- Check uploaded evidence status
SQL> SELECT ge.Grievance_ID,
  2       ge.Document_Type,
  3       ge.Verification_Status
  4  FROM GRIEVANCE_EVIDENCE ge;

GRIEVANCE_ID
------------
DOCUMENT_TYPE
--------------------------------------------------------------------------------
VERIFICATION_STATUS
--------------------------------------------------
         1
PDF
```

VERIFIED

2

IMG

PENDING

GRIEVANCE_ID

---

DOCUMENT_TYPE

---

VERIFICATION_STATUS

---

3

PDF

VERIFIED

4

PDF

GRIEVANCE_ID

---

DOCUMENT_TYPE

---

VERIFICATION_STATUS

---

VERIFIED

5

IMG

PENDING

6

GRIEVANCE_ID

---

DOCUMENT_TYPE

---

VERIFICATION_STATUS

---

PDF

VERIFIED

7

IMG

PENDING

GRIEVANCE_ID

---

DOCUMENT_TYPE

---

VERIFICATION_STATUS

---

8

PDF

VERIFIED

9

PDF

VERIFIED

GRIEVANCE_ID

---

DOCUMENT_TYPE

---

VERIFICATION_STATUS

---

```
        10
IMG
PENDING



10 rows selected.


SQL> -- Evidence still not verified
SQL> SELECT ge.Grievance_ID,
  2       ge.File_Path
  3  FROM GRIEVANCE_EVIDENCE ge
  4  WHERE ge.Verification_Status <> 'VERIFIED';


GRIEVANCE_ID
------------
FILE_PATH
--------------------------------------------------------------------------------
         2
/e2.jpg


         5
/e5.jpg


         7
/e7.jpg



GRIEVANCE_ID
------------
FILE_PATH
--------------------------------------------------------------------------------
        10
/e10.jpg
```

```
SQL> -- Evidence still not verified
SQL> SELECT ge.Grievance_ID,
  2       ge.File_Path
  3  FROM GRIEVANCE_EVIDENCE ge
  4  WHERE ge.Verification_Status <> 'VERIFIED';

GRIEVANCE_ID
------------
FILE_PATH
--------------------------------------------------------------------------------
         2
/e2.jpg

         5
/e5.jpg

         7
/e7.jpg


GRIEVANCE_ID
------------
FILE_PATH
--------------------------------------------------------------------------------
        10
/e10.jpg


SQL> -- Full audit trail
SQL> SELECT gal.Grievance_ID,
  2       gal.Action_Type,
  3       gal.Old_Status,
```

```
  4       gal.New_Status,
  5       gal.Action_Timestamp
  6  FROM GRIEVANCE_ACTION_LOG gal
  7  WHERE gal.Grievance_ID = 1
  8  ORDER BY gal.Action_Timestamp;

GRIEVANCE_ID
------------
ACTION_TYPE
--------------------------------------------------------------------------------
OLD_STATUS
--------------------------------------------------
NEW_STATUS
--------------------------------------------------
ACTION_TIMESTAMP
---------------------------------------------------------------------------
         1
REVIEW
OPEN

GRIEVANCE_ID
------------
ACTION_TYPE
--------------------------------------------------------------------------------
OLD_STATUS
--------------------------------------------------
NEW_STATUS
--------------------------------------------------
ACTION_TIMESTAMP
---------------------------------------------------------------------------
IN_PROGRESS
14-APR-26 05.48.06.470000 PM
```

```
SQL> -- No progress cases
SQL> SELECT g.Grievance_ID
  2  FROM GRIEVANCE g
  3  WHERE g.Grievance_ID NOT IN (
  4     SELECT gal.Grievance_ID FROM GRIEVANCE_ACTION_LOG gal
  5  );

no rows selected

SQL> -- Panels currently handling grievances
SQL> SELECT grp.Grievance_ID,
  2       grp.Panel_Status,
  3       grp.Total_Members
  4  FROM GRIEVANCE_REVIEW_PANEL grp
  5  WHERE grp.Panel_Status = 'ACTIVE';

GRIEVANCE_ID PANEL_STATUS                          TOTAL_MEMBERS
------------ ---------------------------------------------------- -------------
           1 ACTIVE                                            3
           3 ACTIVE                                            4
           5 ACTIVE                                            3
           7 ACTIVE                                            3
          10 ACTIVE                                            5

SQL> -- Cases where values changed
SQL> SELECT gr.Grievance_ID,
  2       gr.Old_Value,
  3       gr.New_Value
  4  FROM GRIEVANCE_RESOLUTION gr
  5  WHERE gr.Old_Value IS NOT NULL;

GRIEVANCE_ID
------------
OLD_VALUE
```

---

NEW_VALUE

---

4

B

A

6

80

85

GRIEVANCE_ID

---

OLD_VALUE

---

NEW_VALUE

---

8

70

75

1

86

GRIEVANCE_ID

---

OLD_VALUE

---

NEW_VALUE

---

88

```
SQL> -- % of resolved grievances
SQL> SELECT ROUND(
  2     (SUM(CASE WHEN g.Status = 'RESOLVED' THEN 1 ELSE 0 END) * 100.0)
  3     / COUNT(*), 2
  4  ) AS resolution_rate
  5  FROM GRIEVANCE g;

RESOLUTION_RATE
---------------
              0

SQL> -- Workload per exam
SQL> SELECT g.Exam_ID,
  2       COUNT(*) AS pending
  3  FROM GRIEVANCE g
  4  WHERE g.Status <> 'RESOLVED'
  5  GROUP BY g.Exam_ID;

   EXAM_ID    PENDING
---------- ----------
         1         10

SQL> -- Number of actions handled per officer
SQL> SELECT gal.Performed_By,
  2       COUNT(*) AS actions_count
  3  FROM GRIEVANCE_ACTION_LOG gal
  4  GROUP BY gal.Performed_By;

PERFORMED_BY ACTIONS_COUNT
------------ --------------
         201             1
         202             1
         203             1
         204             1
```

```
205        1
206        1
207        1
208        1
209        1
210        1

10 rows selected.

SQL> -- Malpractice-related complaints
SQL> SELECT mg.Grievance_ID,
  2       mg.Report_ID
  3  FROM MALPRACTICE_GRIEVANCE mg;

no rows selected

SQL> spool off;
```

## IMPLEMENTATION OF MODULE 9 - SECURITY AND MONITORING MODULE

## (i) DESIGN AND IMPLEMENTATION OF ENTITIES WITH DATA INITIALIZATION:

**This module focuses on the creation and management of security and monitoring entities, including access control mechanisms, user authentication and authorization, system activity logging, audit trail maintenance, intrusion detection, and the insertion of essential data required to ensure system integrity, confidentiality, and continuous monitoring of all critical operations within the examination system.**

```
SQL> CREATE TABLE MALPRACTICE_REPORT (
  2    Report_ID NUMBER PRIMARY KEY,
  3    Schedule_ID NUMBER NOT NULL,
  4    Registration_ID NUMBER NOT NULL,
  5    Student_ID NUMBER NOT NULL,
  6    Invigilator_ID NUMBER NOT NULL,
  7    Room_ID NUMBER NOT NULL,
  8    Incident_Time TIMESTAMP NOT NULL,
```

```
  9   Incident_Type VARCHAR2(100) NOT NULL,
 10   Detection_Mode VARCHAR2(50) NOT NULL,
 11   Description CLOB NOT NULL,
 12   Evidence_File_Path VARCHAR2(500),
 13   Severity_Level VARCHAR2(50) NOT NULL,
 14   Immediate_Action_Taken CLOB,
 15   Escalation_Level VARCHAR2(50) NOT NULL,
 16   Committee_Review_Status VARCHAR2(50) NOT NULL,
 17   Final_Decision CLOB,
 18   Penalty_Type VARCHAR2(100),
 19   Penalty_Duration VARCHAR2(100),
 20   Reported_At TIMESTAMP NOT NULL,
 21   Closed_At TIMESTAMP,
 22   FOREIGN KEY (Schedule_ID) REFERENCES EXAM_SCHEDULE(Schedule_ID),
 23   FOREIGN KEY (Registration_ID) REFERENCES REGISTRATION(Registration_ID),
 24   FOREIGN KEY (Student_ID) REFERENCES STUDENT(Student_ID),
 25   FOREIGN KEY (Invigilator_ID) REFERENCES INVIGILATOR(Invigilator_ID),
 26   FOREIGN KEY (Room_ID) REFERENCES ROOM(Room_ID)
 27 );

Table created.

SQL> CREATE TABLE CENTER_HEALTH_STATUS (
  2   Status_ID NUMBER PRIMARY KEY,
  3   Center_ID NUMBER NOT NULL,
  4   Schedule_ID NUMBER NOT NULL,
  5   CCTV_Status VARCHAR2(50) NOT NULL,
  6   Biometric_Device_Status VARCHAR2(50) NOT NULL,
  7   Metal_Detector_Status VARCHAR2(50) NOT NULL,
  8   Network_Connectivity_Status VARCHAR2(50) NOT NULL,
  9   Power_Backup_Status VARCHAR2(50) NOT NULL,
 10   Fire_Safety_Status VARCHAR2(50) NOT NULL,
 11   Overall_Risk_Level VARCHAR2(50) NOT NULL,
 12   Checked_By NUMBER NOT NULL,
```

```
  13    Last_Checked TIMESTAMP NOT NULL,
  14    Remarks CLOB,
  15    FOREIGN KEY (Center_ID) REFERENCES EXAM_CENTER(Center_ID),
  16    FOREIGN KEY (Schedule_ID) REFERENCES EXAM_SCHEDULE(Schedule_ID)
  17  );
```

Table created.

```
SQL> INSERT INTO MALPRACTICE_REPORT VALUES (1,1,1,1,1,1,SYSTIMESTAMP,'CHEATING','CCTV','Using
chits',NULL,'HIGH','Seized material','LEVEL1','PENDING',NULL,NULL,NULL,SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO MALPRACTICE_REPORT VALUES
(2,2,2,2,2,2,SYSTIMESTAMP,'IMPERSONATION','BIOMETRIC','Mismatch detected',NULL,'CRITICAL','Student
detained','LEVEL2','PENDING',NULL,NULL,NULL,SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO MALPRACTICE_REPORT VALUES
(3,3,3,3,3,3,SYSTIMESTAMP,'MOBILE_USAGE','INVIGILATOR','Using phone',NULL,'HIGH','Phone
confiscated','LEVEL1','PENDING',NULL,NULL,NULL,SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO MALPRACTICE_REPORT VALUES (4,4,4,4,4,4,SYSTIMESTAMP,'CHEATING','CCTV','Copying
answers',NULL,'MEDIUM','Warning given','LEVEL1','REVIEWED',NULL,NULL,NULL,SYSTIMESTAMP,NULL);
```

1 row created.

```
SQL> INSERT INTO MALPRACTICE_REPORT VALUES
(5,5,5,5,5,5,SYSTIMESTAMP,'DISTURBANCE','INVIGILATOR','Creating noise',NULL,'LOW','Removed
temporarily','LEVEL0','CLOSED',NULL,NULL,NULL,SYSTIMESTAMP,SYSTIMESTAMP);
```

1 row created.

```
SQL> INSERT INTO MALPRACTICE_REPORT VALUES (6,6,6,6,6,6,SYSTIMESTAMP,'CHEATING','CCTV','Hand
signals',NULL,'HIGH','Monitored','LEVEL1','PENDING',NULL,NULL,NULL,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO MALPRACTICE_REPORT VALUES (7,7,7,7,7,7,SYSTIMESTAMP,'IMPERSONATION','BIOMETRIC','Face
mismatch',NULL,'CRITICAL','Escalated','LEVEL3','PENDING',NULL,NULL,NULL,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO MALPRACTICE_REPORT VALUES
(8,8,8,8,8,8,SYSTIMESTAMP,'MOBILE_USAGE','INVIGILATOR','Hidden
device',NULL,'HIGH','Confiscated','LEVEL1','REVIEWED',NULL,NULL,NULL,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO MALPRACTICE_REPORT VALUES (9,9,9,9,9,9,SYSTIMESTAMP,'CHEATING','CCTV','Answer
exchange',NULL,'HIGH','Separated','LEVEL2','PENDING',NULL,NULL,NULL,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO MALPRACTICE_REPORT VALUES
(10,10,10,10,10,10,SYSTIMESTAMP,'OTHER','INVIGILATOR','Suspicious
behavior',NULL,'LOW','Observed','LEVEL0','CLOSED',NULL,NULL,NULL,SYSTIMESTAMP,SYSTIMESTAMP);

1 row created.

SQL> INSERT INTO CENTER_HEALTH_STATUS VALUES
(1,1,1,'OK','OK','OK','GOOD','AVAILABLE','SAFE','LOW',101,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO CENTER_HEALTH_STATUS VALUES
(2,2,2,'OK','FAIL','OK','GOOD','AVAILABLE','SAFE','MEDIUM',102,SYSTIMESTAMP,NULL);

1 row created.
```

```
SQL> INSERT INTO CENTER_HEALTH_STATUS VALUES
(3,3,3,'FAIL','OK','OK','POOR','AVAILABLE','SAFE','HIGH',103,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO CENTER_HEALTH_STATUS VALUES
(4,4,4,'OK','OK','FAIL','GOOD','AVAILABLE','SAFE','MEDIUM',104,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO CENTER_HEALTH_STATUS VALUES
(5,5,5,'OK','OK','OK','GOOD','FAIL','SAFE','HIGH',105,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO CENTER_HEALTH_STATUS VALUES
(6,6,6,'OK','OK','OK','GOOD','AVAILABLE','FAIL','MEDIUM',106,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO CENTER_HEALTH_STATUS VALUES
(7,7,7,'FAIL','FAIL','OK','POOR','FAIL','FAIL','CRITICAL',107,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO CENTER_HEALTH_STATUS VALUES
(8,8,8,'OK','OK','OK','GOOD','AVAILABLE','SAFE','LOW',108,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO CENTER_HEALTH_STATUS VALUES
(9,9,9,'OK','OK','FAIL','GOOD','AVAILABLE','SAFE','MEDIUM',109,SYSTIMESTAMP,NULL);

1 row created.

SQL> INSERT INTO CENTER_HEALTH_STATUS VALUES
(10,10,10,'OK','OK','OK','GOOD','AVAILABLE','SAFE','LOW',110,SYSTIMESTAMP,NULL);
```

1 row created.

## APPLIED SQL QUERIES AND BUSINESS USE CASES – SECURITY MONITORING MODULE:

1. **List All Malpractice Incidents**
2. **High Severity Malpractice Case Detection**
3. **Unresolved Malpractice Case Identification**
4. **Malpractice Cases Per Exam Schedule**
5. **Students Involved in Multiple Malpractice Cases**
6. **Invigilators Reporting Highest Number of Incidents**
7. **Malpractice Incident Type Distribution**
8. **Detection Mode Effectiveness Analysis**
9. **Average Malpractice Case Resolution Time**
10. **Centers With Highest Malpractice Incidents**
11. **Pending Cases With Escalation Level**
12. **Centers With Critical Infrastructure Issues**
13. **Centers With Device Failures**
14. **Risk Distribution Across Centers**
15. **Power Backup Failure Detection in Centers**
16. **Latest Health Status Per Center**
17. **Centers Not Recently Monitored**
18. **Malpractice Cases Linked With Center Risk**
19. **Cases With Assigned Penalties**
20. **Open Cases Without Final Decision**

```
SQL> -- View all reported malpractice cases
SQL> SELECT mr.Report_ID,
  2       mr.Student_ID,
  3       mr.Incident_Type,
```

```
  4       mr.Severity_Level,
  5       mr.Committee_Review_Status
  6  FROM MALPRACTICE_REPORT mr;

 REPORT_ID STUDENT_ID
---------- ----------
INCIDENT_TYPE
--------------------------------------------------------------------------------
SEVERITY_LEVEL
--------------------------------------------------
COMMITTEE_REVIEW_STATUS
--------------------------------------------------
       1        1
CHEATING
HIGH
PENDING


 REPORT_ID STUDENT_ID
---------- ----------
INCIDENT_TYPE
--------------------------------------------------------------------------------
SEVERITY_LEVEL
--------------------------------------------------
COMMITTEE_REVIEW_STATUS
--------------------------------------------------
       2        2
IMPERSONATION
CRITICAL
PENDING


 REPORT_ID STUDENT_ID
---------- ----------
```

INCIDENT_TYPE

---

SEVERITY_LEVEL

---

COMMITTEE_REVIEW_STATUS

---

3 3

MOBILE_USAGE

HIGH

PENDING

REPORT_ID STUDENT_ID

---

INCIDENT_TYPE

---

SEVERITY_LEVEL

---

COMMITTEE_REVIEW_STATUS

---

4 4

CHEATING

MEDIUM

REVIEWED

REPORT_ID STUDENT_ID

---

INCIDENT_TYPE

---

SEVERITY_LEVEL

---

COMMITTEE_REVIEW_STATUS

---

5 5

DISTURBANCE

LOW

CLOSED

REPORT_ID STUDENT_ID

---------- ----------

INCIDENT_TYPE

---

SEVERITY_LEVEL

---

COMMITTEE_REVIEW_STATUS

---

6 6

CHEATING

HIGH

PENDING

REPORT_ID STUDENT_ID

---------- ----------

INCIDENT_TYPE

---

SEVERITY_LEVEL

---

COMMITTEE_REVIEW_STATUS

---

7 7

IMPERSONATION

CRITICAL

PENDING

REPORT_ID STUDENT_ID

---------- ----------

INCIDENT_TYPE

---

SEVERITY_LEVEL

---

COMMITTEE_REVIEW_STATUS

---

8 8

MOBILE_USAGE

HIGH

REVIEWED

REPORT_ID STUDENT_ID

---------- ----------

INCIDENT_TYPE

---

SEVERITY_LEVEL

---

COMMITTEE_REVIEW_STATUS

---

9 9

CHEATING

HIGH

PENDING

REPORT_ID STUDENT_ID

---------- ----------

INCIDENT_TYPE

---

SEVERITY_LEVEL

---

COMMITTEE_REVIEW_STATUS

---

10 10

OTHER

LOW

CLOSED

10 rows selected.

```
SQL> -- Critical incidents requiring urgent action
SQL> SELECT mr.Report_ID,
  2       mr.Student_ID,
  3       mr.Incident_Type
  4  FROM MALPRACTICE_REPORT mr
  5  WHERE mr.Severity_Level IN ('HIGH','CRITICAL');
```

REPORT_ID STUDENT_ID

---

INCIDENT_TYPE

---

1 1

CHEATING

2 2

IMPERSONATION

3 3

MOBILE_USAGE

REPORT_ID STUDENT_ID

---

INCIDENT_TYPE

```
--------------------------------------------------------------------------------
      6        6
CHEATING

      7        7
IMPERSONATION

      8        8
MOBILE_USAGE



 REPORT_ID STUDENT_ID
---------- ----------
INCIDENT_TYPE
--------------------------------------------------------------------------------
      9        9
CHEATING



7 rows selected.


SQL> -- Cases still under review
SQL> SELECT mr.Report_ID,
  2       mr.Student_ID,
  3       mr.Committee_Review_Status
  4  FROM MALPRACTICE_REPORT mr
  5  WHERE mr.Committee_Review_Status <> 'CLOSED';

 REPORT_ID STUDENT_ID COMMITTEE_REVIEW_STATUS
---------- ---------- ----------------------------------------------------
      1        1 PENDING
      2        2 PENDING
      3        3 PENDING
      4        4 REVIEWED
```

```
       6        6 PENDING
       7        7 PENDING
       8        8 REVIEWED
       9        9 PENDING

8 rows selected.

SQL> -- Identify problematic exam sessions
SQL> SELECT mr.Schedule_ID,
  2       COUNT(*) AS total_cases
  3  FROM MALPRACTICE_REPORT mr
  4  GROUP BY mr.Schedule_ID;

SCHEDULE_ID TOTAL_CASES
----------- -----------
        1         1
        2         1
        3         1
        4         1
        5         1
        6         1
        7         1
        8         1
        9         1
       10         1

10 rows selected.

SQL> -- Repeat offenders
SQL> SELECT mr.Student_ID,
  2       COUNT(*) AS case_count
  3  FROM MALPRACTICE_REPORT mr
  4  GROUP BY mr.Student_ID
  5  HAVING COUNT(*) > 1;
```

```
no rows selected

SQL> -- Active invigilators detecting malpractice
SQL> SELECT mr.Invigilator_ID,
  2       COUNT(*) AS reports_count
  3  FROM MALPRACTICE_REPORT mr
  4  GROUP BY mr.Invigilator_ID
  5  ORDER BY reports_count DESC;

INVIGILATOR_ID REPORTS_COUNT
-------------- -------------
             1             1
             2             1
             3             1
             4             1
            10             1
             6             1
             7             1
             8             1
             9             1
             5             1

10 rows selected.

SQL> -- Active invigilators detecting malpractice
SQL> SELECT mr.Invigilator_ID,
  2       COUNT(*) AS reports_count
  3  FROM MALPRACTICE_REPORT mr
  4  GROUP BY mr.Invigilator_ID
  5  ORDER BY reports_count DESC;

INVIGILATOR_ID REPORTS_COUNT
-------------- -------------
```

```
         1          1
         2          1
         3          1
         4          1
        10           1
         6          1
         7          1
         8          1
         9          1
         5          1

10 rows selected.

SQL> -- Most common malpractice types
SQL> SELECT mr.Incident_Type,
  2       COUNT(*) AS total
  3  FROM MALPRACTICE_REPORT mr
  4  GROUP BY mr.Incident_Type
  5  ORDER BY total DESC;

INCIDENT_TYPE
--------------------------------------------------------------------------------
    TOTAL
----------
CHEATING
       4

IMPERSONATION
       2

MOBILE_USAGE
       2
```

```
INCIDENT_TYPE
--------------------------------------------------------------------------------
   TOTAL
----------
DISTURBANCE
       1


OTHER
       1



SQL> -- Which detection method catches most cases
SQL> SELECT mr.Detection_Mode,
  2       COUNT(*) AS total_cases
  3  FROM MALPRACTICE_REPORT mr
  4  GROUP BY mr.Detection_Mode;

DETECTION_MODE                         TOTAL_CASES
-------------------------------------------------- -----------
CCTV                                             4
BIOMETRIC                                        2
INVIGILATOR                                      4


SQL> -- Efficiency of resolution
SQL> SELECT AVG(mr.Closed_At - mr.Reported_At) AS avg_resolution_time
  2  FROM MALPRACTICE_REPORT mr
  3  WHERE mr.Closed_At IS NOT NULL;
SELECT AVG(mr.Closed_At - mr.Reported_At) AS avg_resolution_time
                 *
ERROR at line 1:
ORA-00932: inconsistent datatypes: expected NUMBER got INTERVAL DAY TO SECOND


SQL> -- Efficiency of resolution
SQL> SELECT AVG(mr.Closed_At - mr.Reported_At) AS avg_resolution_time
```

```
  2  FROM MALPRACTICE_REPORT mr
  3  WHERE mr.Closed_At IS NOT NULL;
SELECT AVG(mr.Closed_At - mr.Reported_At) AS avg_resolution_time
                    *
ERROR at line 1:
ORA-00932: inconsistent datatypes: expected NUMBER got INTERVAL DAY TO SECOND


SQL> -- High-risk centers
SQL> SELECT mr.Room_ID,
  2       COUNT(*) AS cases
  3  FROM MALPRACTICE_REPORT mr
  4  GROUP BY mr.Room_ID
  5  ORDER BY cases DESC;

   ROOM_ID     CASES
---------- ----------
         1          1
         2          1
         3          1
         4          1
        10          1
         6          1
         7          1
         8          1
         9          1
         5          1

10 rows selected.


SQL> -- Escalation monitoring
SQL> SELECT mr.Report_ID,
  2       mr.Escalation_Level,
  3       mr.Committee_Review_Status
  4  FROM MALPRACTICE_REPORT mr
```

```
  5  WHERE mr.Committee_Review_Status = 'PENDING';

 REPORT_ID ESCALATION_LEVEL
---------- ----------------------------------------------------
COMMITTEE_REVIEW_STATUS
--------------------------------------------------
         1 LEVEL1
PENDING

         2 LEVEL2
PENDING

         3 LEVEL1
PENDING

 REPORT_ID ESCALATION_LEVEL
---------- ----------------------------------------------------
COMMITTEE_REVIEW_STATUS
--------------------------------------------------
         6 LEVEL1
PENDING

         7 LEVEL3
PENDING

         9 LEVEL2
PENDING

6 rows selected.

SQL> -- Unsafe exam centers
SQL> SELECT chs.Center_ID,
```

```
  2       chs.Overall_Risk_Level
  3  FROM CENTER_HEALTH_STATUS chs
  4  WHERE chs.Overall_Risk_Level = 'CRITICAL';

 CENTER_ID OVERALL_RISK_LEVEL
---------- --------------------------------------------------
         7 CRITICAL

SQL> -- Hardware failures impacting exam integrity
SQL> SELECT chs.Center_ID,
  2       chs.CCTV_Status,
  3       chs.Biometric_Device_Status,
  4       chs.Network_Connectivity_Status
  5  FROM CENTER_HEALTH_STATUS chs
  6  WHERE chs.CCTV_Status = 'FAIL'
  7    OR chs.Biometric_Device_Status = 'FAIL'
  8    OR chs.Network_Connectivity_Status = 'POOR';

 CENTER_ID CCTV_STATUS
---------- --------------------------------------------------
BIOMETRIC_DEVICE_STATUS
--------------------------------------------------
NETWORK_CONNECTIVITY_STATUS
--------------------------------------------------
         2 OK
FAIL
GOOD

         3 FAIL
OK
POOR

 CENTER_ID CCTV_STATUS
---------- --------------------------------------------------
```

```
BIOMETRIC_DEVICE_STATUS
--------------------------------------------------
NETWORK_CONNECTIVITY_STATUS
--------------------------------------------------

       7 FAIL
FAIL
POOR


SQL> -- Risk-level summary
SQL> SELECT chs.Overall_Risk_Level,
  2       COUNT(*) AS total_centers
  3  FROM CENTER_HEALTH_STATUS chs
  4  GROUP BY chs.Overall_Risk_Level;

OVERALL_RISK_LEVEL                        TOTAL_CENTERS
-------------------------------------------------- -------------
LOW                                                3
MEDIUM                                                 4
HIGH                                               2
CRITICAL                                               1

SQL> -- Critical operational risk
SQL> SELECT chs.Center_ID
  2  FROM CENTER_HEALTH_STATUS chs
  3  WHERE chs.Power_Backup_Status = 'FAIL';

 CENTER_ID
----------
       5
       7

SQL> -- Most recent check per center
```

```
SQL> SELECT chs.Center_ID,
  2       MAX(chs.Last_Checked) AS last_check
  3  FROM CENTER_HEALTH_STATUS chs
  4  GROUP BY chs.Center_ID;

 CENTER_ID
----------
LAST_CHECK
---------------------------------------------------------------------------
       1
14-APR-26 05.52.13.599000 PM

       2
14-APR-26 05.52.13.612000 PM

       3
14-APR-26 05.52.13.618000 PM

 CENTER_ID
----------
LAST_CHECK
---------------------------------------------------------------------------
       4
14-APR-26 05.52.13.622000 PM

       5
14-APR-26 05.52.13.628000 PM

       6
14-APR-26 05.52.13.634000 PM

 CENTER_ID
```

```
----------
LAST_CHECK
---------------------------------------------------------------------------
       7
14-APR-26 05.52.13.642000 PM


       8
14-APR-26 05.52.13.648000 PM


       9
14-APR-26 05.52.13.655000 PM



 CENTER_ID
----------
LAST_CHECK
---------------------------------------------------------------------------
      10
14-APR-26 05.52.13.982000 PM



10 rows selected.


SQL> -- Monitoring gaps
SQL> SELECT chs.Center_ID
  2  FROM CENTER_HEALTH_STATUS chs
  3  WHERE chs.Last_Checked < SYSDATE - 1;


no rows selected


SQL> -- Correlate infra risk with malpractice
SQL> SELECT mr.Report_ID,
  2       chs.Overall_Risk_Level
  3  FROM MALPRACTICE_REPORT mr
```

```
  4  JOIN CENTER_HEALTH_STATUS chs
  5    ON mr.Schedule_ID = chs.Schedule_ID;

 REPORT_ID OVERALL_RISK_LEVEL
---------- --------------------------------------------------
         1 LOW
         2 MEDIUM
         3 HIGH
         4 MEDIUM
         5 HIGH
         6 MEDIUM
         7 CRITICAL
         8 LOW
         9 MEDIUM
        10 LOW

10 rows selected.

SQL> -- Disciplinary actions
SQL> SELECT mr.Report_ID,
  2       mr.Penalty_Type,
  3       mr.Penalty_Duration
  4  FROM MALPRACTICE_REPORT mr
  5  WHERE mr.Penalty_Type IS NOT NULL;

no rows selected

SQL> -- No final decision yet
SQL> SELECT mr.Report_ID,
  2       mr.Student_ID
  3  FROM MALPRACTICE_REPORT mr
  4  WHERE mr.Final_Decision IS NULL;

 REPORT_ID STUDENT_ID
```

```
---------- ----------
         1          1
         2          2
         3          3
         4          4
         5          5
         6          6
         7          7
         8          8
         9          9
        10         10

10 rows selected.

SQL> spool off;
```